CS336 Assignment 1 (basics): Building a Transformer LM

Version 1.0.6

CS336 Staff

Spring 2025

1 Assignment Overview

In this assignment, you will build all the components needed to train a standard Transformer language model (LM) from scratch and train some models.

What you will implement

1. Byte-pair encoding (BPE) tokenizer (§2)
2. Transformer language model (LM) (§3)
3. The cross-entropy loss function and the AdamW optimizer (§4)
4. The training loop, with support for serializing and loading model and optimizer state (§5)

What you will run

1. Train a BPE tokenizer on the TinyStories dataset.
2. Run your trained tokenizer on the dataset to convert it into a sequence of integer IDs.
3. Train a Transformer LM on the TinyStories dataset.
4. Generate samples and evaluate perplexity using the trained Transformer LM.
5. Train models on OpenWebText and submit your attained perplexities to a leaderboard.

What you can use We expect you to build these components from scratch. In particular, you may *not* use any definitions from `torch.nn`, `torch.nn.functional`, or `torch.optim` except for the following:

- `torch.nn.Parameter`
- Container classes in `torch.nn` (e.g., `Module`, `ModuleList`, `Sequential`, etc.)[1]
- The `torch.optim.Optimizer` base class

You may use any other PyTorch definitions. If you would like to use a function or class and are not sure whether it is permitted, feel free to ask on Slack. When in doubt, consider if using it compromises the "from-scratch" ethos of the assignment.

[1] See PyTorch.org/docs/stable/nn.html#containers for a full list.

CS336 作业1（基础）：构建变压器LM

1.0.6版

CS336 员工

2025年春季

1 任务概述

在本任务中，您将从零开始构建训练标准Transformer语言模型（LM）所需的所有组件，并训练若干模型。

你将要实施的

1. 字节对编码（BPE）分词器（§ 2）
2. 变压器语言模型（LM）（§ 3）
3. 交叉熵损失函数与AdamW优化器（§ 4）
4. 训练循环，支持对模型和优化器状态进行序列化和加载（§ 5）

您将运行的内容

1. 在TinyStories数据集上训练 BPE 分词器。
2. 运行训练好的分词器对数据集进行转换，将其转化为整数ID序列。
3. 在TinyStories数据集上训练Transformer语言模型。
4. 使用训练好的Transformer语言模型生成样本并评估困惑度。
5. 在OpenWebText上训练模型，并将获得的困惑度提交至排行榜。

可用工具 我们要求您从头开始构建这些组件。特别说明，除以下内容外，您 *不得* 使用torch.nn、torch.nn.functional或torch.optim中的任何定义：

- `torch.nn.Parameter`
- torch .nn中的容器类（例如，Module、ModuleList、Sequential等）[1]
- 火炬优化器优化器基类

你可以自由使用其他PyTorch定义。若想使用某个函数或类但不确定是否合规，欢迎在Slack上提问。如有疑问，需考虑其使用是否违背本作业的‘从零开始’原则。

[1] 完整列表请参见PyTorch.org/docs/stable/nn.html#containers。

Statement on AI tools Prompting LLMs such as ChatGPT is permitted for low-level programming questions or high-level conceptual questions about language models, but using it directly to solve the problem is prohibited.

We strongly encourage you to disable AI autocomplete (e.g., Cursor Tab, GitHub CoPilot) in your IDE when completing assignments (though non-AI autocomplete, e.g., autocompleting function names is totally fine). We have found that AI autocomplete makes it much harder to engage deeply with the content.

What the code looks like All the assignment code as well as this writeup are available on GitHub at:

`github.com/stanford-cs336/assignment1-basics`

Please `git clone` the repository. If there are any updates, we will notify you so you can `git pull` to get the latest.

1. `cs336_basics/*`: This is where you write your code. Note that there's no code in here—you can do whatever you want from scratch!
2. `adapters.py`: There is a set of functionality that your code must have. For each piece of functionality (e.g., scaled dot product attention), fill out its implementation (e.g., `run_scaled_dot_product_attention`) by simply invoking your code. Note: your changes to `adapters.py` should not contain any substantive logic; this is glue code.
3. `test_*.py`: This contains all the tests that you must pass (e.g., `test_scaled_dot_product_attention`), which will invoke the hooks defined in `adapters.py`. Don't edit the test files.

How to submit You will submit the following files to Gradescope:

- `writeup.pdf`: Answer all the written questions. Please typeset your responses.
- `code.zip`: Contains all the code you've written.

To submit to the leaderboard, submit a PR to:

`github.com/stanford-cs336/assignment1-basics-leaderboard`

See the `README.md` in the leaderboard repository for detailed submission instructions.

Where to get datasets This assignment will use two pre-processed datasets: TinyStories [Eldan and Li, 2023] and OpenWebText [Gokaslan et al., 2019]. Both datasets are single, large plaintext files. If you are doing the assignment with the class, you can find these files at `/data` of any non-head node machine.

If you are following along at home, you can download these files with the commands inside the `README.md`.

Low-Resource/Downscaling Tip: Init

Throughout the course's assignment handouts, we will give advice for working through parts of the assignment with fewer or no GPU resources. For example, we will sometimes suggest **downscaling** your dataset or model size, or explain how to run training code on a MacOS integrated GPU or CPU. You'll find these "low-resource tips" in a blue box (like this one). Even if you are an enrolled Stanford student with access to the course machines, these tips may help you iterate faster and save time, so we recommend you to read them!

关于AI工具的声明允许使用ChatGPT等LLM进行低级编程问题或关于语言模型的高级概念问题的提示，但禁止直接使用它来解决问题。

我们强烈建议您在使用集成开发环境（IDE）完成作业时，关闭AI自动补全功能（如Cursor Tab、GitHub Copilot等）（但保留非AI自动补全功能，例如函数名自动补全则完全可行）。研究发现，AI自动补全会显著降低用户对内容的深度参与度。

代码的具体形式 所有代码和说明文档均可在 GitHub 上获取，地址为：

`github.com/stanford-cs336/assignment1-basics`

请通过git克隆该仓库。如有更新，我们将通知您，以便您通过git pull获取最新版本。

1. cs336 _basics/*: 这里是编写代码的地方。注意这里没有代码——你可以从头开始自由编写任何内容！
2. adapters.py：该文件定义了代码必须具备的功能模块。针对每个功能模块（如缩放点积注意力机制），只需调用runScaled_dot_product_attention等函数即可完成实现。注意：对适配器.py的修改不应包含实质性逻辑，此类代码属于粘合代码。
3. test_* .py：该文件包含所有必须通过的测试（例如，test_scaled_dot_product_attention)，该方法将调用在 adapters.py 中定义的钩子函数。请勿修改测试文件。

如何提交 请将以下文件提交至Gradescope：

- writeup.pdf：请回答所有书面问题。请将您的回答排版。
- zip文件：包含您编写的所有代码。

若要提交至排行榜，请向`github.com/stanford-cs336/assignment1-basics-leaderboard`提交PR

详见排行榜仓库中的readme.md文件获取详细提交说明。

数据集获取途径 本作业将使用两个预处理数据集：TinyStories[Eldan和Li，2023]和OpenWebText[Gokaslan等人，2019]。这两个数据集均为单一大型纯文本文件。若与班级合作完成作业，可在任意非头部节点机器的/data目录下找到这些文件。

若您在家操作，可通过readme.md文件中的命令下载这些文件。

低资源/降尺度提示：启动

在课程作业手册中，我们会为使用GPU资源有限或无GPU的学员提供实用建议。例如，有时会建议**降低数据集或模型规模**，或是讲解如何在MacOS集成GPU或CPU上运行训练代码。这些“低配资源小技巧”会以蓝色方框（如图示）的形式呈现。即使您是斯坦福大学注册学员并能使用课程专用设备，这些技巧也能帮助您加快迭代速度、节省时间，因此我们强烈推荐您仔细阅读！

Low-Resource/Downscaling Tip: Assignment 1 on Apple Silicon or CPU

With the staff solution code, we can train an LM to generate reasonably fluent text on an Apple M3 Max chip with 36 GB RAM, in under 5 minutes on Metal GPU (MPS) and about 30 minutes using the CPU. If these words don't mean much to you, don't worry! Just know that if you have a reasonably up-to-date laptop and your implementation is correct and efficient, you will be able to train a small LM that generates simple children's stories with decent fluency.

Later in the assignment, we will explain what changes to make if you are on CPU or MPS.

低资源/降级提示：Apple Silicon或CPU的作业1

借助员工解决方案代码，我们能在配备36GB内存的苹果M3 Max芯片上训练语言模型（LM），使用Metal GPU（MPS）时不到5分钟即可生成流畅文本，而CPU方案则需约30分钟。若您对这些术语不太熟悉也无需担心！只要您的笔记本电脑配置尚可，且实现方案正确高效，就能训练出能生成流畅度不错的儿童故事的小型语言模型。

在后续任务中，我们将说明若采用CPU或MPS方案时需进行的变更。

2 Byte-Pair Encoding (BPE) Tokenizer

In the first part of the assignment, we will train and implement a byte-level byte-pair encoding (BPE) tokenizer [Sennrich et al., 2016, Wang et al., 2019]. In particular, we will represent arbitrary (Unicode) strings as a sequence of bytes and train our BPE tokenizer on this byte sequence. Later, we will use this tokenizer to encode text (a string) into tokens (a sequence of integers) for language modeling.

2.1 The Unicode Standard

Unicode is a text encoding standard that maps characters to integer *code points*. As of Unicode 16.0 (released in September 2024), the standard defines 154,998 characters across 168 scripts. For example, the character "s" has the code point 115 (typically notated as `U+0073`, where `U+` is a conventional prefix and `0073` is 115 in hexadecimal), and the character "牛" has the code point 29275. In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

Problem (`unicode1`): Understanding Unicode (1 point)

(a) What Unicode character does `chr(0)` return?

Deliverable: A one-sentence response.

(b) How does this character's string representation (`__repr__()`) differ from its printed representation?

Deliverable: A one-sentence response.

(c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

Deliverable: A one-sentence response.

2.2 Unicode Encodings

While the Unicode standard defines a mapping from characters to code points (integers), it's impractical to train tokenizers directly on Unicode codepoints, since the vocabulary would be prohibitively large (around 150K items) and sparse (since many characters are quite rare). Instead, we'll use a Unicode encoding, which converts a Unicode character into a sequence of bytes. The Unicode standard itself defines three encodings: UTF-8, UTF-16, and UTF-32, with UTF-8 being the dominant encoding for the Internet (more than 98% of all webpages).

To encode a Unicode string into UTF-8, we can use the `encode()` function in Python. To access the underlying byte values for a Python `bytes` object, we can iterate over it (e.g., call `list()`). Finally, we can use the `decode()` function to decode a UTF-8 byte string into a Unicode string.

2 字节对编码(BPE)分词器

在任务的第一部分，我们将训练并实现一个字节级字节对编码（BPE）分词器[Sennrich等人，2016，Wang等人，2019]。具体而言，我们将任意（Unicode）字符串表示为字节序列，并在此字节序列上训练我们的BPE 分词器。随后，我们将使用该分词器将文本（字符串）编码为标记（整数序列），用于语言建模。

2.1 Unicode标准

Unicode是一种文本编码标准，将字符映射到整数*代码点*。截至2024年9月发布的Unicode 16.0版本，该标准定义了168种文字系统中的154,998个字符。例如，字符“s”的代码点为115（通常表示为U+0073，其中U+是常规前缀，0073代表十六进制数115），而字符“牛”的代码点为29275。在Python中，可以使用ord()函数将单个Unicode字符转换为其整数表示。chr()函数则将整数Unicode代码点转换为对应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题（Unicode1）：理解Unicode（1分）

(a) chr(0)返回的Unicode字符是什么？

交付物：一句话的答复。

(b) 该字符的字符串表示形式（__repr__（））与其打印形式有何区别？

交付物：一句话的答复。

(c) 当这个字符出现在文本中时会发生什么？不妨在Python解释器中尝试以下操作，看看是否符合预期：

```
>>> chr(0)
>>> 打印 ( 字符 (0 ))
>>> “这是个测试” + chr (0) + “字符串”
>>> print ( “这是个测试” + chr (0) + “字符串” )
```

交付物: 一句话的回复。

2.2 Unicode 编码

虽然Unicode标准定义了字符到代码点（整数）的映射关系，但直接在Unicode代码点上训练分词器并不现实，因为词汇量会大到难以处理（约15万条目），且分布稀疏（许多字符极为罕见）。因此，我们将采用Unicode编码，该编码将Unicode字符转换为字节序列。Unicode标准本身定义了三种编码： UTF -8、UTF -16和 UTF -32，其中 UTF -8是互联网主流编码（超过98%的网页采用该编码）。

将Unicode字符串编码为 UTF -8时，可使用Python的encode()函数。要获取Pythonbytes对象的底层字节值，可遍历该对象（例如调用list()）。最后，使用decode()函数将 UTF -8字节字符串解码为Unicode字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # Get the byte values for the encoded string (integers from 0 to 255).
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # One byte does not necessarily correspond to one Unicode character!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

By converting our Unicode codepoints into a sequence of bytes (e.g., via the UTF-8 encoding), we are essentially taking a sequence of codepoints (integers in the range 0 to 154,997) and transforming it into a sequence of byte values (integers in the range 0 to 255). The 256-length byte vocabulary is *much* more manageable to deal with. When using byte-level tokenization, we do not need to worry about out-of-vocabulary tokens, since we know that *any* input text can be expressed as a sequence of integers from 0 to 255.

Problem (`unicode2`): Unicode Encodings (3 points)

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

Deliverable: A one-to-two sentence response.

(b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

Deliverable: An example input byte string for which `decode_utf8_bytes_to_str_wrong` produces incorrect output, with a one-sentence explanation of why the function is incorrect.

(c) Give a two byte sequence that does not decode to any Unicode character(s).

Deliverable: An example, with a one-sentence explanation.

2.3 Subword Tokenization

While byte-level tokenization can alleviate the out-of-vocabulary issues faced by word-level tokenizers, tokenizing text into bytes results in extremely long input sequences. This slows down model training, since a

```
>>> test_string = "你好！ こんにちは！"
>>> utf8 _encoded = test_string . encode ( "utf-8" )
>>> 打印 (utf8 编码)
b 'hello！ \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf！' >>> print
(type (utf8 _encoded))
<class 'bytes'>
>>> # 获取编码字符串的字节值（0到255的整数）。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # 一个字节并不一定对应一个Unicode字符！
>>> 打印 ( len (test_string))
13
>>> 打印 ( 长度 (utf8 编码))
23
>>> 打印 (utf8 _编码 . 解码 ( "utf-8" ))
你好！ こんにちは！
```

通过将Unicode编码点转换为字节序列（例如采用 UTF -8编码），我们本质上是将编码点序列（0到154,997范围内的整数）转换为字节值序列（0到255范围内的整数）。这种256字节长度的词汇表 *易于* 处理。采用字节级分词时，我们无需担心词汇表外的标记，因为已知 *任何* 输入文本均可表示为0到255范围内的整数序列。

问题（Unicode2）：Unicode编码（3分）

(a) 相较于 UTF -16或 UTF -32，为何更倾向于使用 UTF -8编码字节训练分词器？通过对比不同编码方式对各类输入字符串的输出结果，或许能提供参考依据。

交付物： 一至两句话的答复。

(b) 考虑以下（错误的）函数，该函数旨在将 UTF -8字节字符串解码为Unicode字符串。为何该函数存在错误？请举例说明一个输入字节字符串，其结果不正确。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

交付成果： 一个示例输入字节字符串， decode_utf8 _bytes_to_str_wrong函数会生成错误输出，并附带简要说明该函数为何会产生错误。

(c) 提供一个无法解码为任何Unicode字符的两字节序列。

交付物： 一个例子，附带一句话的解释。

2.3 子词分词

虽然字节级分词技术能缓解词级分词器面临的词汇表外问题，但将文本分割为字节会导致输入序列过长。这会拖慢模型训练速度，因为

sentence with 10 words might only be 10 tokens long in a word-level language model, but could be 50 or more tokens long in a character-level model (depending on the length of the words). Processing these longer sequences requires more computation at each step of the model. Furthermore, language modeling on byte sequences is difficult because the longer input sequences create long-term dependencies in the data.

Subword tokenization is a midpoint between word-level tokenizers and byte-level tokenizers. Note that a byte-level tokenizer's vocabulary has 256 entries (byte values are 0 to 225). A subword tokenizer trades-off a larger vocabulary size for better compression of the input byte sequence. For example, if the byte sequence `b'the'` often occurs in our raw text training data, assigning it an entry in the vocabulary would reduce this 3-token sequence to a single token.

How do we select these subword units to add to our vocabulary? Sennrich et al. [2016] propose to use byte-pair encoding (BPE; Gage, 1994), a compression algorithm that iteratively replaces ("merges") the most frequent pair of bytes with a single, new unused index. Note that this algorithm adds subword tokens to our vocabulary to maximize the compression of our input sequences—if a word occurs in our input text enough times, it'll be represented as a single subword unit.

Subword tokenizers with vocabularies constructed via BPE are often called BPE tokenizers. In this assignment, we'll implement a byte-level BPE tokenizer, where the vocabulary items are bytes or merged sequences of bytes, which give us the best of both worlds in terms of out-of-vocabulary handling and manageable input sequence lengths. The process of constructing the BPE tokenizer vocabulary is known as "training" the BPE tokenizer.

2.4 BPE Tokenizer Training

The BPE tokenizer training procedure consists of three main steps.

Vocabulary initialization The tokenizer vocabulary is a one-to-one mapping from bytestring token to integer ID. Since we're training a byte-level BPE tokenizer, our initial vocabulary is simply the set of all bytes. Since there are 256 possible byte values, our initial vocabulary is of size 256.

Pre-tokenization Once you have a vocabulary, you could, in principle, count how often bytes occur next to each other in your text and begin merging them starting with the most frequent pair of bytes. However, this is quite computationally expensive, since we'd have to go take a full pass over the corpus each time we merge. In addition, directly merging bytes across the corpus may result in tokens that differ only in punctuation (e.g., `dog!` vs. `dog.`). These tokens would get completely different token IDs, even though they are likely to have high semantic similarity (since they differ only in punctuation).

To avoid this, we *pre-tokenize* the corpus. You can think of this as a coarse-grained tokenization over the corpus that helps us count how often pairs of characters appear. For example, the word `'text'` might be a pre-token that appears 10 times. In this case, when we count how often the characters 't' and 'e' appear next to each other, we will see that the word 'text' has 't' and 'e' adjacent and we can increment their count by 10 instead of looking through the corpus. Since we're training a byte-level BPE model, each pre-token is represented as a sequence of UTF-8 bytes.

The original BPE implementation of Sennrich et al. [2016] pre-tokenizes by simply splitting on whitespace (i.e., `s.split(" ")`). In contrast, we'll use a regex-based pre-tokenizer (used by GPT-2; Radford et al., 2019) from `github.com/openai/tiktoken/pull/234/files`:

```
>>> PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

It may be useful to interactively split some text with this pre-tokenizer to get a better sense of its behavior:

```
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

在词级语言模型中，一个10个词的句子可能仅由10个词元组成，但在字符级模型中（取决于词的长度）可能长达50个或更多词元。处理这些较长的序列需要模型在每个步骤进行更多计算。此外，字节序列上的语言建模较为困难，因为较长的输入序列会在数据中产生长期依赖关系。

子词分词技术是介于词级分词器和字节级分词器之间的折中方案。字节级分词器的词汇表包含256个条目（字节值范围0-225），而子词分词器则通过扩大词汇量来提升输入字节序列的压缩效率。例如，若原始训练文本中频繁出现字节序列 'b' the '，将其纳入词汇表后，这个由三个词组成的序列就能被压缩为单个分词。

我们如何选择这些子词单元来扩充词汇量？Sennrich等人2016] 提议采用字节对编码（BPE；Gage，1994），这是一种迭代替换（"合并"）最频繁字节对为单一新未使用索引的压缩算法。需注意该算法通过向词汇表添加子词标记来最大化输入序列的压缩效果——若某个词在输入文本中出现足够多次，则会被表示为单一子词单元。

通过 BPE 构建词汇表的子词分词器通常被称为 BPE 分词器。本任务中，我们将实现一个字节级 BPE 分词器，其词汇表项为字节或字节合并序列，这种设计在处理超出词表范围和管理输入序列长度方面实现了两全其美。构建 BPE 分词器词汇表的过程被称为 '训练' BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练流程包含三个主要步骤。

词汇表初始化分词器词汇表是字节字符串分词符到整数ID的一一映射。由于我们训练的是字节级 BPE 分词器，初始词汇表就是所有字节的集合。由于存在256种可能的字节值，初始词汇表的大小为256。

预分词化 一旦确定词汇表，理论上可以统计文本中字节相邻出现的频率，并从出现频率最高的字节对开始进行合并。但这种方法计算成本较高，因为每次合并都需要对语料库进行完整遍历。此外，直接合并语料库中的字节可能会产生仅在标点符号上存在差异的标记（例如dog！与dog）。这些标记虽然可能具有高度语义相似性（仅因标点不同），但会获得完全不同的标记ID。

为避免这种情况，我们对语料库进行*预分词处理*。这种处理相当于对语料库进行粗粒度分词，有助于统计字符对的出现频率。例如，单词 'text' 可能是一个出现10次的预分词。此时当我们统计字符 't' 和 'e' 相邻出现的次数时，会发现单词 'text' 中存在相邻的 't' 和 'e'，因此可以直接将它们的计数增加10，而无需遍历整个语料库。由于我们训练的是字节级 BPE 模型，每个预分词都以 UTF -8字节的序列形式呈现。

Sennrich等人的原始 BPE 实现2016] 通过简单地按空格分割来预分词（即s.split（""））。相比之下，我们将使用基于正则表达式的预分词器（由GPT-2使用；Radford等人，2019），该工具来自`github.com/openai/tiktoken/pull/234/files`：

```
>>> PAT = r""" '(?:[sdmt]|ll|ve|re)|?\p{L}+|?\p{N}+|?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

使用该预分词器对部分文本进行交互式分割可能有助于更好地理解其行为：

```
>>> # 需要`regex`包
>>> 导入 正则表达式 作为 re
>>> re.findall(PAT, "需要预分词的文本")
['some', '文本', 'that', '我', "'将', '前', '-', '分词']
```

When using it in your code, however, you should use `re.finditer` to avoid storing the pre-tokenized words as you construct your mapping from pre-tokens to their counts.

Compute BPE merges Now that we've converted our input text into pre-tokens and represented each pre-token as a sequence of UTF-8 bytes, we can compute the BPE merges (i.e., train the BPE tokenizer). At a high level, the BPE algorithm iteratively counts every pair of bytes and identifies the pair with the highest frequency ("A", "B"). Every occurrence of this most frequent pair ("A", "B") is then *merged*, i.e., replaced with a new token "AB". This new merged token is added to our vocabulary; as a result, the final vocabulary after BPE training is the size of the initial vocabulary (256 in our case), plus the number of BPE merge operations performed during training. For efficiency during BPE training, we do not consider pairs that cross pre-token boundaries.[2] When computing merges, deterministically break ties in pair frequency by *preferring the lexicographically greater pair*. For example, if the pairs ("A", "B"), ("A", "C"), ("B", "ZZ"), and ("BA", "A") all have the highest frequency, we'd merge ("BA", "A"):

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

Special tokens Often, some strings (e.g., `<|endoftext|>`) are used to encode metadata (e.g., boundaries between documents). When encoding text, it's often desirable to treat some strings as "special tokens" that should never be split into multiple tokens (i.e., will always be preserved as a single token). For example, the end-of-sequence string `<|endoftext|>` should always be preserved as a single token (i.e., a single integer ID), so we know when to stop generating from the language model. These special tokens must be added to the vocabulary, so they have a corresponding fixed token ID.

Algorithm 1 of Sennrich et al. [2016] contains an inefficient implementation of BPE tokenizer training (essentially following the steps that we outlined above). As a first exercise, it may be useful to implement and test this function to test your understanding.

Example (`bpe_example`): BPE training example

Here is a stylized example from Sennrich et al. [2016]. Consider a corpus consisting of the following text

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

and the vocabulary has a special token `<|endoftext|>`.

Vocabulary We initialize our vocabulary with our special token `<|endoftext|>` and the 256 byte values.

Pre-tokenization For simplicity and to focus on the merge procedure, we assume in this example that pretokenization simply splits on whitespace. When we pretokenize and count, we end up with the frequency table.

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

[2]Note that the original BPE formulation [Sennrich et al., 2016] specifies the inclusion of an end-of-word token. We do not add an end-of-word-token when training byte-level BPE models because all bytes (including whitespace and punctuation) are included in the model's vocabulary. Since we're explicitly representing spaces and punctuation, the learned BPE merges will naturally reflect these word boundaries.

但是，当在代码中使用它时，您应该使用re . finditer来避免在构建从预标记到其计数的映射时存储预标记化的单词。

计算 BPE 合并 现在我们已将输入文本转换为预标记，并将每个预标记表示为 UTF -8字节序列，可以计算 BPE 合并（即训练 BPE 分词器）。从宏观层面看， BPE 算法会迭代统计每对字节，并识别出现频率最高的组合（“A”、“B”）。每当出现这种高频组合（“A”、“B”）时，就会进行*合并*，即用新标记“AB”替换。这个新合并标记会被添加到词典中，因此 BPE 训练后的最终词典大小等于初始词典大小（本例中为256）加上训练期间执行的 BPE 合并操作次数。为提升 BPE 训练效率，我们不考虑跨越预标记边界的组合。[2] 在计算合并时，通过*优先选择字典序较大的组合*来确定性解决频率并列问题。例如，若组合（“A”、“B”）、（“A”、“C”）、（“B”、“ZZ”）和（“BA”、“A”）都出现频率最高，我们会合并（“BA”、“A”）：

```
>>> max ([( "A" , "B" ), ( "A" , "C" ), ( "B" , "ZZ" ), ( "BA" , "A" )])
('BA', 'A')
```

特殊标记 通常，某些字符串（例如（endofctext））用于编码元数据（例如文档之间的边界）。在文本编码过程中，我们往往希望将某些字符串视为“特殊标记”，这些标记不应被拆分为多个独立标记（即应始终保持为单一标记）。例如，序列结束字符串（endofctext）必须始终作为单一标记（即单一整数ID）保留，这样我们就能明确何时停止从语言模型生成内容。这些特殊标记需要被添加到词汇表中，因此它们拥有对应的固定标记ID。

Sennrich等人的算法1[2016] 包含对 BPE 分词器训练的低效实现（本质上遵循我们之前概述的步骤）。作为首次练习，建议实现并测试该功能以检验理解程度。

示例 (bpe_example)： BPE 训练示例

Sennrich等人的风格化示例如下2016] 请考虑一个包含以下文本的语料库

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

词汇具有特殊的标记（内嵌文本）。

词汇表 我们使用特殊标记（endoftext）和256字节值初始化词汇表。

预分词化 为简化操作流程并聚焦合并过程，本示例假设预分词化仅对空白字符进行分割。完成预分词化与统计后，最终生成词频表。

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

[2] 需要注意的是，原始 BPE 公式[Sennrich等人，2016]规定了包含词尾标记。我们在训练字节级 BPE 模型时不会添加词尾标记，因为模型词汇表已包含所有字节（包括空白和标点）。由于我们明确表示了空格和标点，所学 BPE 的合并结果自然会反映这些词边界。

It is convenient to represent this as a `dict[tuple[bytes], int]`, e.g. {`(l,o,w)`: 5 …}. Note that even a single byte is a `bytes` object in Python. There is no `byte` type in Python to represent a single byte, just as there is no `char` type in Python to represent a single character.

Merges We first look at every successive pair of bytes and sum the frequency of the words where they appear {`lo`: 7, `ow`: 7, `we`: 8, `er`: 2, `wi`: 3, `id`: 3, `de`: 3, `es`: 9, `st`: 9, `ne`: 6, `ew`: 6}. The pair `('es')` and `('st')` are tied, so we take the lexicographically greater pair, `('st')`. We would then merge the pre-tokens so that we end up with {`(l,o,w)`: 5, `(l,o,w,e,r)`: 2, `(w,i,d,e,st)`: 3, `(n,e,w,e,st)`: 6}.

In the second round, we see that `(e, st)` is the most common pair (with a count of 9) and we would merge into {`(l,o,w)`: 5, `(l,o,w,e,r)`: 2, `(w,i,d,est)`: 3, `(n,e,w,est)`: 6}. Continuing this, the sequence of merges we get in the end will be `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']`.

If we take 6 merges, we have `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']` and our vocabulary elements would be `[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]`.

With this vocabulary and set of merges, the word `newest` would tokenize as `[ne, west]`.

2.5 Experimenting with BPE Tokenizer Training

Let's train a byte-level BPE tokenizer on the TinyStories dataset. Instructions to find / download the dataset can be found in Section 1. Before you start, we recommend taking a look at the TinyStories dataset to get a sense of what's in the data.

Parallelizing pre-tokenization You will find that a major bottleneck is the pre-tokenization step. You can speed up pre-tokenization by parallelizing your code with the built-in library `multiprocessing`. Concretely, we recommend that in parallel implementations of pre-tokenization, you chunk the corpus while ensuring your chunk boundaries occur at the beginning of a special token. You are free to use the starter code at the following link verbatim to obtain chunk boundaries, which you can then use to distribute work across your processes:

`https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py`

This chunking will always be valid, since we never want to merge across document boundaries. For the purposes of the assignment, you can always split in this way. Don't worry about the edge case of receiving a very large corpus that does not contain `<|endoftext|>`.

Removing special tokens before pre-tokenization Before running pre-tokenization with the regex pattern (using `re.finditer`), you should strip out all special tokens from your corpus (or your chunk, if using a parallel implementation). Make sure that you **split** on your special tokens, so that no merging can occur across the text they delimit. For example, if you have a corpus (or chunk) like `[Doc 1]<|endoftext|>[Doc 2]`, you should split on the special token `<|endoftext|>`, and pre-tokenize `[Doc 1]` and `[Doc 2]` separately, so that no merging can occur across the document boundary. This can be done using `re.split` with `"|".join(special_tokens)` as the delimiter (with careful use of `re.escape` since `|` may occur in the special tokens). The test `test_train_bpe_special_tokens` will test for this.

Optimizing the merging step The naïve implementation of BPE training in the stylized example above is slow because for every merge, it iterates over all byte pairs to identify the most frequent pair. However, the only pair counts that change after each merge are those that overlap with the merged pair. Thus, BPE training speed can be improved by indexing the counts of all pairs and incrementally updating these counts, rather than explicitly iterating over each pair of bytes to count pair frequencies. You can get significant speedups with this caching procedure, though we note that the merging part of BPE training is *not* parallelizable in Python.

将此表示为字典[元组[字节]，整数]是方便的，例如{(l，o，w):5...}。注意，即使是一个字节在Python中也是一个字节对象。Python中没有表示单个字节的字节类型，就像Python中没有表示单个字符的char类型一样。

合并 我们首先逐对分析每个字节对，并统计它们出现的词频{lo: 7，ow: 7，we: 8，er: 2，wi: 3，id: 3，de: 3，es: 9，st: 9，ne: 6，ew: 6}。由于对(‘es’)和(‘st’)并列，我们选择字典序较大的对(‘st’)。随后对预标记进行合并，最终得到{(l，o，w): 5，(l，o，w，e，r) : 2，(w，i，d，e，st) : 3，(n，e，w，e，st): 6}。

在第二轮合并中，我们发现(e，st)是最常见的组合（出现9次），因此将其归并为{(l，o，w):5，(l，o，w，e，r):2，(w，i，d，est):3，(n，e，w，est):6}。继续合并后，最终得到的序列将为[‘s t’，‘e st’，‘o w’，‘l ow’，‘w est’，‘n e’，‘ne west’，‘w i’，‘wi d’，‘wid est’，‘low e’，‘lowe r’]。

若进行6次合并，则得到[‘s t’，‘e st’，‘o w’，‘l ow’，‘w est’，‘n e’]，此时词汇元素将为[(endoftext)，[...256 BYTE chars]，st，est，ow，low，west，ne]。

使用该词汇表及合并规则，单词“newest”将被分词为[ne，west]。

2.5 BPE 分词器训练的实验研究

我们将在TinyStories数据集上训练一个字节级 BPE 分词器。获取/下载该数据集的说明详见第1节。开始前建议先浏览TinyStories数据集，以便了解其内容。

并行化预分词处理 你可能会发现预分词步骤是主要瓶颈。通过使用内置的 multiprocessing 库对代码进行并行化处理，可以显著提升预分词效率。具体来说，我们建议在并行化预分词实现时，将语料库分割成多个块，并确保块边界位于特殊标记的起始位置。你可以直接照搬下方链接中的示例代码获取块边界，然后利用这些边界将任务分配到各个进程：

`https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py`

这种分块方式始终有效，因为我们从不希望跨越文档边界进行合并。在作业要求中，你可以随时采用这种方式拆分。对于接收到非常大的语料库且其中不包含（内嵌文本）的情况，无需担心边缘情况。

预分词化前移除特殊标记 在使用正则表达式模式（通过 re. finditer）进行预分词之前，应先从语料库（或使用并行实现时的块）中移除所有特殊标记。确保对特殊标记进行**分割**，以避免它们所界定的文本之间发生合并。例如，若语料库（或块）包含[文档1](endoftext)[文档2]，则应以特殊标记(endoftext)为界进行分割，并分别对[文档1]和[文档2]进行预分词，以防止文档边界处发生合并。此操作可通过 re. split 并使用 “|”.join（特殊标记）作为分隔符来实现（需谨慎使用 re. escape，因特殊标记中可能包含|）。测试 test_train_bpe_special_tokens 将对此进行验证。

优化合并步骤 上述风格化示例中 BPE 训练的原始实现速度较慢，因为每次合并时都需要遍历所有字节对来识别出现频率最高的组合。但每次合并后发生变化的计数仅限于与合并后字节对重叠的那些。因此，通过建立所有字节对的计数索引并进行增量更新，而非显式遍历每个字节对来统计频率，可以显著提升 BPE 训练速度。虽然这种缓存机制能带来明显加速效果，但需要指出的是， BPE 训练中的合并环节在Python中 *无法* 实现并行化。

Low-Resource/Downscaling Tip: Profiling

You should use profiling tools like `cProfile` or `scalene` to identify the bottlenecks in your implementation, and focus on optimizing those.

Low-Resource/Downscaling Tip: "Downscaling"

Instead of jumping to training your tokenizer on the full TinyStories dataset, we recommend you first train on a small subset of the data: a "debug dataset". For example, you could train your tokenizer on the TinyStories validation set instead, which is 22K documents instead of 2.12M. This illustrates a general strategy of downscaling whenever possible to speed up development: for example, using smaller datasets, smaller model sizes, etc. Choosing the size of the debug dataset or hyperparameter config requires careful consideration: you want your debug set to be large enough to have the same bottlenecks as the full configuration (so that the optimizations you make will generalize), but not so big that it takes forever to run.

Problem (`train_bpe`): BPE Tokenizer Training (15 points)

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

`input_path: str` Path to a text file with BPE tokenizer training data.

`vocab_size: int` A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).

`special_tokens: list[str]` A list of strings to add to the vocabulary. These special tokens do not otherwise affect BPE training.

Your BPE training function should return the resulting vocabulary and merges:

`vocab: dict[int, bytes]` The tokenizer vocabulary, a mapping from `int` (token ID in the vocabulary) to `bytes` (token bytes).

`merges: list[tuple[bytes, bytes]]` A list of BPE merges produced from training. Each list item is a `tuple` of `bytes` (`<token1>, <token2>`), representing that `<token1>` was merged with `<token2>`. The merges should be ordered by order of creation.

To test your BPE training function against our provided tests, you will first need to implement the test adapter at `[adapters.run_train_bpe]`. Then, run `uv run pytest tests/test_train_bpe.py`. Your implementation should be able to pass all tests. Optionally (this could be a large time-investment), you can implement the key parts of your training method using some systems language, for instance C++ (consider `cppyy` for this) or Rust (using PyO3). If you do this, be aware of which operations require copying vs reading directly from Python memory, and make sure to leave build instructions, or make sure it builds using only `pyproject.toml`. Also note that the GPT-2 regex is not well-supported in most regex engines and will be too slow in most that do. We have verified that Oniguruma is reasonably fast and supports negative lookahead, but the `regex` package in Python is, if anything, even faster.

低资源/降尺度技巧:剖面分析

建议使用cProfile或scalene等性能分析工具来找出系统实现中的瓶颈，并重点优化这些环节。

资源匮乏/规模缩减提示：“规模缩减”

与其直接在完整的TinyStories数据集上训练分词器，我们建议先在小规模数据子集上进行训练——即所谓的“调试数据集”。例如，可以使用TinyStories验证集（包含2.2万篇文档而非完整的212万篇）来训练模型。这体现了开发加速的通用策略：尽可能采用降维方案，比如使用更小的数据集、更精简的模型架构等。选择调试数据集的规模或超参数配置需要谨慎考量：既要确保调试集足够大以保留完整配置的瓶颈（使优化效果具有普适性），又不能过大导致运行耗时过长。

问题（train_bpe）： BPE 分词器训练（15分）

交付成果：编写一个函数，该函数接收输入文本文件的路径，用于训练（字节级）BPE 分词器。 BPE 训练函数应至少处理以下输入参数：

input_path: str BPE 分词器训练数据的文本文件路径。

vocab_size: int 一个正整数，用于定义最终词汇表的最大规模（包含初始字节词汇表、合并生成的词汇项以及任何特殊标记）。

special_tokens: list[str] 用于扩充词汇表的字符串列表。这些特殊标记不会影响 BPE 训练。

您的 BPE 训练功能应返回生成的词汇表及合并结果：

vocab: dict[int，bytes] 词表，将int（词表中的标记ID）映射到bytes（标记字节）。

merges: list[tuple[bytes，bytes]] 一个由训练产生的 BPE 合并列表。每个列表项是 元组 的 bytes (<token1>，<token2>)，表示 <token1> 与 <token2> 合并。合并应按创建顺序排列。

要测试 BPE 训练功能是否符合我们提供的测试标准，您需要先在[adapters.run_train_bpe]中实现测试适配器。接着运行uv run pytest tests/test_train_bpe.py脚本，您的实现应该能通过所有测试。可选方案（这可能需要投入大量时间）是使用系统语言（例如C++[推荐使用cppyy]或Rust[配合PyO3]）实现训练方法的核心部分。若选择此方案，请注意哪些操作需要从Python内存中直接复制数据而非读取，并确保保留构建指令或仅使用pyproject.toml进行构建。需注意的是，GPT-2正则表达式在多数正则引擎中支持不足，且在大多数支持该功能的引擎中速度会过慢。我们已验证Oniguruma速度尚可且支持负向预读，但Python自带的正则表达式包速度甚至更快。

Problem (`train_bpe_tinystories`): BPE Training on TinyStories (2 points)

(a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How many hours and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 30 minutes (no GPUs), ≤ 30GB RAM

Hint You should be able to get under 2 minutes for BPE training using `multiprocessing` during pretokenization and the following two facts:

(a) The `<|endoftext|>` token delimits documents in the data files.

(b) The `<|endoftext|>` token is handled as a special case before the BPE merges are applied.

Deliverable: A one-to-two sentence response.

(b) Profile your code. What part of the tokenizer training process takes the most time?

Deliverable: A one-to-two sentence response.

Next, we'll try training a byte-level BPE tokenizer on the OpenWebText dataset. As before, we recommend taking a look at the dataset to better understand its contents.

Problem (`train_bpe_expts_owt`): BPE Training on OpenWebText (2 points)

(a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 12 hours (no GPUs), ≤ 100GB RAM

Deliverable: A one-to-two sentence response.

(b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Deliverable: A one-to-two sentence response.

2.6 BPE Tokenizer: Encoding and Decoding

In the previous part of the assignment, we implemented a function to train a BPE tokenizer on input text to obtain a tokenizer vocabulary and a list of BPE merges. Now, we will implement a BPE tokenizer that loads a provided vocabulary and list of merges and uses them to encode and decode text to/from token IDs.

2.6.1 Encoding text

The process of encoding text by BPE mirrors how we train the BPE vocabulary. There are a few major steps.

Step 1: Pre-tokenize. We first pre-tokenize the sequence and represent each pre-token as a sequence of UTF-8 bytes, just as we did in BPE training. We will be merging these bytes within each pre-token into vocabulary elements, handling each pre-token independently (no merges across pre-token boundaries).

Step 2: Apply the merges. We then take the sequence of vocabulary element merges created during BPE training, and apply it to our pre-tokens *in the same order of creation*.

问题（train_bpe_tinystories）：在TinyStories数据集上进行 BPE 训练（2分）

(a) 在TinyStories数据集上训练一个字节级 BPE 分词器，最大词典规模设为10,000。请确保将TinyStories（endotext）特殊标记添加到词典中。将生成的词典序列化并合并到磁盘以供后续检查。训练耗时多少小时？占用多少内存？词典中最长的标记是什么？这合理吗？

资源要求： ≤ 30分钟（无GPU），≤ 30GB内存

提示： 在预标记阶段采用多处理技术进行 BPE 训练时，结合以下两点事实，你应能将时间控制在2分钟以内：

(a) （内文）标记用于界定数据文件中的文档。

(b) 在应用 BPE 合并前，（内文）标记会被视为特殊情况处理。**交付成果：** 需提供一至两句话的回复。

(b) 分析代码性能。在分词器训练过程中，哪个环节耗时最长？

交付物： 一至两句话的答复。

接下来，我们将尝试在OpenWebText数据集上训练一个字节级 BPE 分词器。与之前一样，建议先查看数据集内容，以便更好地理解其特性。

问题（train_bpe_expts_owt）： BPE 在OpenWebText上的训练（2分）

(a) 在OpenWebText数据集上训练一个字节级 BPE 分词器，最大词表大小为32,000。将生成的词表序列化并合并到磁盘以供后续检查。词表中最长的标记是什么？它是否合理？

资源要求： ≤ 12小时（无GPU），≤ 100GB内存

交付物： 一至两句话的答复。

(b) 对比分析在TinyStories和OpenWebText数据集上训练得到的分词器性能差异。

交付物： 一至两句话的答复。

2.6 BPE 分词器:编码与解码

在作业的前一部分，我们实现了一个函数，用于在输入文本上训练 BPE 分词器，以获取分词器词汇表和 BPE 合并列表。现在，我们将实现一个 BPE 分词器，该分词器加载提供的词汇表和合并列表，并利用它们将文本编码和解码为/从tokenID。

2.6.1 文本编码

BPE 对文本进行编码的过程，与我们训练 BPE 词汇表的方式类似，主要包含几个关键步骤。

步骤1：预分词。 我们首先对序列进行预分词处理，将每个预分词表示为 UTF -8字节的序列，这与 BPE 训练时的操作方式相同。随后，我们将每个预分词内部的字节合并为词汇元素，并对每个预分词进行独立处理（不跨越预分词边界进行合并）。

步骤2：应用合并操作。 我们随后将 BPE 训练期间创建的词汇元素合并序列，*按照创建顺序* 应用于预标记。

Example (bpe_encoding): BPE encoding example

For example, suppose our input string is `'the cat ate'`, our vocabulary is `{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}`, and our learned merges are `[(b't', b'h'), (b' ', b'c'), (b' ', 'a'), (b'th', b'e'), (b' a', b't')]`. First, our pre-tokenizer would split this string into `['the', ' cat', ' ate']`. Then, we'll look at each pre-token and apply the BPE merges.
The first pre-token `'the'` is initially represented as `[b't', b'h', b'e']`. Looking at our list of merges, we identify the first applicable merge to be `(b't', b'h')`, and use that to transform the pre-token into `[b'th', b'e']`. Then, we go back to the list of merges and identify the next applicable merge to be `(b'th', b'e')`, which transforms the pre-token into `[b'the']`. Finally, looking back at the list of merges, we see that there are no more that apply to the string (since the entire pre-token has been merged into a single token), so we are done applying the BPE merges. The corresponding integer sequence is `[9]`.
Repeating this process for the remaining pre-tokens, we see that the pre-token `' cat'` is represented as `[b' c', b'a', b't']` after applying the BPE merges, which becomes the integer sequence `[7, 1, 5]`. The final pre-token `' ate'` is `[b' at', b'e']` after applying the BPE merges, which becomes the integer sequence `[10, 3]`. Thus, the final result of encoding our input string is `[9, 7, 1, 5, 10, 3]`.

Special tokens. Your tokenizer should be able to properly handle user-defined special tokens when encoding text (provided when constructing the tokenizer).

Memory considerations. Suppose we want to tokenize a large text file that we cannot fit in memory. To efficiently tokenize this large file (or any other stream of data), we need to break it up into manageable chunks and process each chunk in-turn, so that the memory complexity is constant as opposed to linear in the size of the text. In doing so, we need to make sure that a token doesn't cross chunk boundaries, else we'll get a different tokenization than the naïve method of tokenizing the entire sequence in-memory.

2.6.2 Decoding text

To decode a sequence of integer token IDs back to raw text, we can simply look up each ID's corresponding entries in the vocabulary (a byte sequence), concatenate them together, and then decode the bytes to a Unicode string. Note that input IDs are not guaranteed to map to valid Unicode strings (since a user could input any sequence of integer IDs). In the case that the input token IDs do not produce a valid Unicode string, you should replace the malformed bytes with the official Unicode replacement character `U+FFFD`.[3] The `errors` argument of `bytes.decode` controls how Unicode decoding errors are handled, and using `errors='replace'` will automatically replace malformed data with the replacement marker.

Problem (tokenizer): Implementing the tokenizer (15 points)

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren't already there). We recommend the following interface:

```
def __init__(self, vocab, merges, special_tokens=None)
```
Construct a tokenizer from a given vocabulary, list of merges, and (optionally) a list of special tokens. This function should accept

[3] See en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character for more information about the Unicode replacement character.

示例 (bpe_encoding)： BPE 编码示例

例如，假设输入字符串为‘the cat ate’，词表包含{0: b‘ ’，1: b‘a’，2: b‘c’，3: b‘e’，4: b‘h’，5: b‘t’，6: b‘th’，7: b‘c’，8: b‘a’，9: b‘the’，10: b‘at’}，学习到的合并规则为[(b‘t’，b‘h’)，(b‘ ’，b‘c’)，(b‘ ’，‘a’)，(b‘th’，b‘e’)，(b‘a’，b‘t)]。首先，预分词器会将该字符串拆分为[‘the‘，‘cat‘，‘ate‘]。
接下来，我们将逐一分析每个前缀标记，并应用 BPE 合并操作。
前缀‘the’最初表示为[b‘t’，b‘h’，b‘e’]。通过分析合并列表，我们确定首个适用合并为(b‘t’，b‘h’)，并据此将前缀转换为[b‘th’，b‘e’]。随后再次检查合并列表，发现下一个适用合并为(b‘th’，b‘e’)，将前缀转换为[b‘the’]。最终回看合并列表时发现，由于整个前缀已合并为单一标记，不再有其他合并可施加，至此完成 BPE 合并操作。对应的整数序列为[9]。
对剩余前缀进行相同处理后，我们发现前缀‘cat’在应用 BPE 合并后表示为[b‘c’，b‘a’，b‘t’]，对应整数序列[7,1,5]。前缀‘ate’在应用 BPE 合并后表示为[b‘at’，b‘e’]，对应整数序列[10,3]。因此，输入字符串的最终编码结果为[9,7,1,5,10,3]。

特殊标记。 你的分词器在编码文本时应能正确处理用户定义的特殊标记（需在构建分词器时提供）。

内存优化策略。 假设需要对无法完全加载内存的大文本文件进行分词处理。为高效处理这类海量数据流，我们需要将文本拆分为若干可管理的分块，逐个进行处理，从而将内存复杂度控制在常量级而非与文本长度呈线性关系。在此过程中，必须确保分词结果不会跨越分块边界，否则会导致分词结果与直接在内存中对整个序列进行分词的简单方法产生显著差异。

2.6.2 解码文本

要将整数令牌ID序列解码回原始文本，我们只需在字典（字节序列）中查找每个ID对应的条目，将它们连接起来，再将这些字节解码为Unicode字符串。需要注意的是，输入的ID并不保证能映射为有效的Unicode字符串（因为用户可能输入任意整数ID序列）。当输入的令牌ID无法生成有效Unicode字符串时，应使用官方Unicode替换字符U+ FFFD 替换异常字节。[3] bytes.decode函数的errors参数控制Unicode解码错误的处理方式，使用errors=‘replace’会自动将异常数据替换为替换标记。

问题（分词器）：实现分词器（15分）

交付成果： 实现一个Tokenizer类，该类接收词汇表和合并列表，将文本编码为整数ID，并将整数ID解码为文本。Tokenizer还应支持用户提供的特殊标记（若词汇表中尚未包含，则将其追加到词汇表中）。我们推荐以下接口：

def init (self，vocab，merges，special_tokens=None) 根据给定的词汇表、合并列表（可选）和特殊标记列表构建分词器。该函数应接受

[3] 有关Unicode替换字符的更多信息，请参阅en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character。

the following parameters:

```
vocab: dict[int, bytes]
merges: list[tuple[bytes, bytes]]
special_tokens: list[str] | None = None
```

`def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)` Class method that constructs and return a `Tokenizer` from a serialized vocabulary and list of merges (in the same format that your BPE training code output) and (optionally) a list of special tokens. This method should accept the following additional parameters:

```
vocab_filepath: str
merges_filepath: str
special_tokens: list[str] | None = None
```

`def encode(self, text: str) -> list[int]` Encode an input text into a sequence of token IDs.

`def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]` Given an iterable of strings (e.g., a Python file handle), return a generator that lazily yields token IDs. This is required for memory-efficient tokenization of large files that we cannot directly load into memory.

`def decode(self, ids: list[int]) -> str` Decode a sequence of token IDs into text.

To test your `Tokenizer` against our provided tests, you will first need to implement the test adapter at `[adapters.get_tokenizer]`. Then, run `uv run pytest tests/test_tokenizer.py`. Your implementation should be able to pass all tests.

2.7 Experiments

Problem (`tokenizer_experiments`): Experiments with tokenizers (4 points)

(a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Deliverable: A one-to-two sentence response.

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Deliverable: A one-to-two sentence response.

(c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Deliverable: A one-to-two sentence response.

(d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

以下参数：

词汇： 字典 [整数 ， 字节]
合并： 列表 [元组 [字节 ， 字节]]
special_tokens: 列表 [str] | 无 = 无

def from_files (cls，vocabfilepath，mergesfilepath，special_tokens=None) 类方法，用于根据序列化词汇表、合并列表（格式与 BPE 训练代码输出一致）以及（可选）特殊标记列表，构建并返回一个分词器。该方法需接受以下附加参数：

`vocab_filepath: str`
merges_filepath: 字符串
special_tokens: 列表 [str] | 无 = 无

`def encode(self, text: str) -> list[int]` Encode an input text into a sequence of token IDs.

def encode_iterable (self，iterable: Iterable[str]) -> Iterator[int] 接受一个字符串迭代器（例如Python文件句柄），返回一个生成器，该生成器以惰性方式生成标记ID。此方法用于对无法直接加载到内存中的大文件进行内存高效的标记化处理。

`def decode(self, ids: list[int]) -> str` Decode a sequence of token IDs into text.

要测试你的Tokenizer是否符合我们提供的测试要求，首先需要在[adapters.get_tokenizer]处实现测试适配器。接着运行uv run pytest tests/test_tokenizer.py。你的实现应该能够通过所有测试。

2.7 实验

问题（tokenizer_experiments）：词元化器实验（4分）

(a) 从TinyStories和OpenWebText中抽取10份文档样本，使用预先训练的TinyStories和OpenWebText分词器（词表规模分别为10K和32K），将这些样本文档编码为整数ID。求各分词器的压缩比（字节/词）。
交付物： 一至两句话的答复。

(b) 若使用TinyStories分词器对OpenWebText样本进行分词处理，会发生什么变化？请比较压缩比并/或定性描述其效果。
交付物： 一至两句话的答复。

(c) 估算您的分词器吞吐量（例如，以字节/秒为单位）。对Pile数据集（825GB文本）进行分词需要多长时间？
交付物： 一至两句话的答复。

(d) 请使用TinyStories和OpenWebText分词器，将训练集和开发集分别编码为整数标记ID序列。该序列将用于后续语言模型训练。建议将标记ID序列化为NumPy的uint16型数组。为何选择uint16作为数据类型？

Deliverable: A one-to-two sentence response.

交付物：一至两句话的答复。

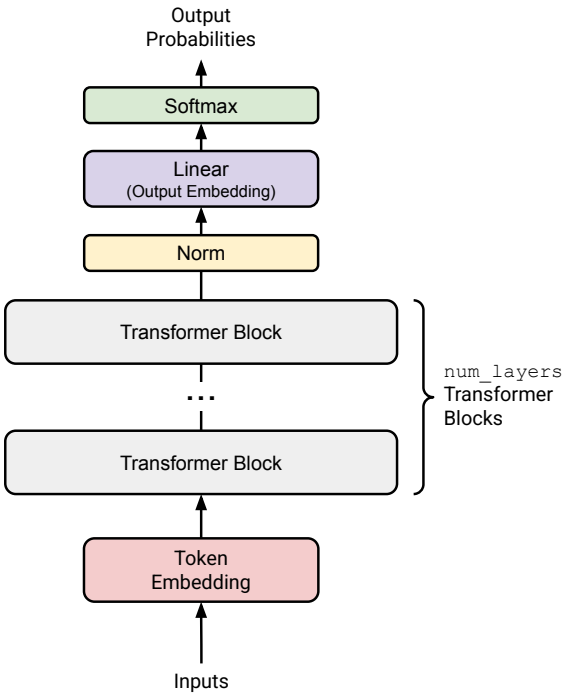

Figure 1: An overview of our Transformer language model.

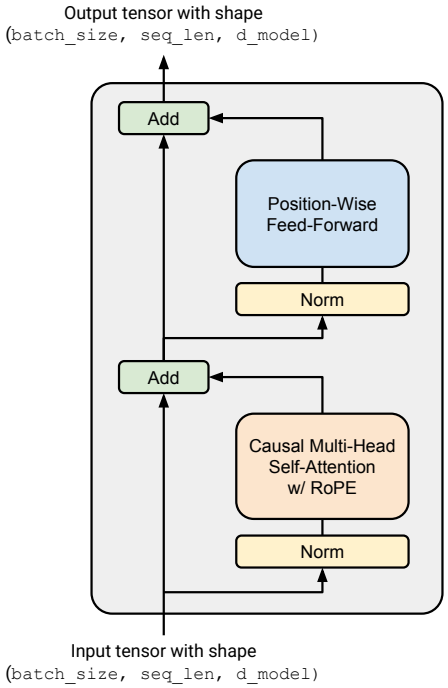

Figure 2: A pre-norm Transformer block.

3 Transformer Language Model Architecture

A language model takes as input a batched sequence of integer token IDs (i.e., `torch.Tensor` of shape `(batch_size, sequence_length)`), and returns a (batched) normalized probability distribution over the vocabulary (i.e., a PyTorch Tensor of shape `(batch_size, sequence_length, vocab_size)`), where the predicted distribution is over the next word for each input token. When training the language model, we use these next-word predictions to calculate the cross-entropy loss between the actual next word and the predicted next word. When generating text from the language model during inference, we take the predicted next-word distribution from the final time step (i.e., the last item in the sequence) to generate the next token in the sequence (e.g., by taking the token with the highest probability, sampling from the distribution, etc.), add the generated token to the input sequence, and repeat.

In this part of the assignment, you will build this Transformer language model from scratch. We will begin with a high-level description of the model before progressively detailing the individual components.

3.1 Transformer LM

Given a sequence of token IDs, the Transformer language model uses an input embedding to convert token IDs to dense vectors, passes the embedded tokens through `num_layers` Transformer blocks, and then applies a learned linear projection (the "output embedding" or "LM head") to produce the predicted next-token logits. See Figure 1 for a schematic representation.

3.1.1 Token Embeddings

In the very first step, the Transformer *embeds* the (batched) sequence of token IDs into a sequence of vectors containing information on the token identity (red blocks in Figure 1).

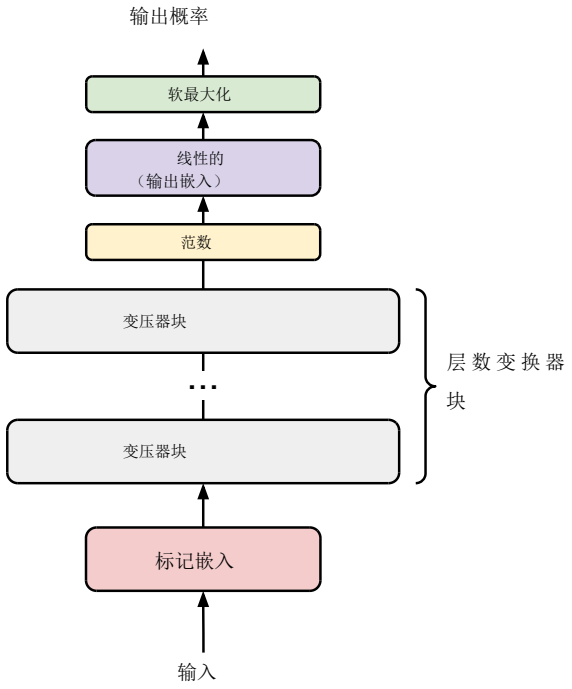

图1：我们的Transformer语言模型概述。

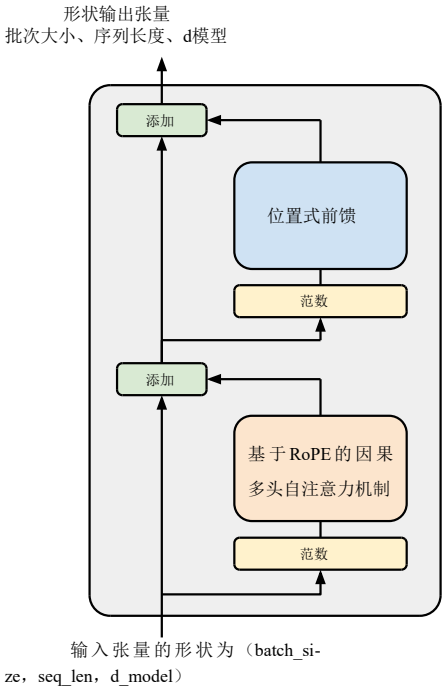

图2：预归一化Transformer模块。

3 Transformer语言模型架构

语言模型接收以整数形式的标记ID序列（即torch。张量，形状为（batch_size，sequence_length））作为输入，输出词汇表上的（批量）归一化概率分布（即PyTorch张量，形状为（batch_size，sequence_length，vocab_size）），其中预测分布对应每个输入标记的下一个词。在训练语言模型时，我们利用这些下词预测值计算实际下词与预测下词之间的交叉熵损失。在推理阶段生成文本时，我们从最终时间步（即序列末尾项）获取预测的下词分布来生成序列中的下一个标记（例如通过选取概率最高的标记、从分布中采样等方式），将生成的标记添加到输入序列后重复该过程。

在本任务模块中，您将从零开始构建Transformer语言模型。我们将先从模型的宏观架构入手，再逐步深入解析各个组件的具体实现。

3.1 变压器LM

给定一组标记ID序列，Transformer语言模型首先通过输入嵌入将标记ID转换为高维向量，随后将这些嵌入向量输入Transformer网络的num_layers层，最后通过训练得到的线性投影（即“输出嵌入”或“语言模型头”）生成预测的下一个标记的logits。具体示意图可参考图1。

3.1.1 标记嵌入

在第一步中，Transformer将（分批的）token ID序列*嵌入*到一个向量序列中，该向量序列包含关于token身份的信息（图1中的红色方块）。

More specifically, given a sequence of token IDs, the Transformer language model uses a token embedding layer to produce a sequence of vectors. Each embedding layer takes in a tensor of integers of shape `(batch_size, sequence_length)` and produces a sequence of vectors of shape `(batch_size, sequence_length, d_model)`.

3.1.2 Pre-norm Transformer Block

After embedding, the activations are processed by several identically structured neural net layers. A standard decoder-only Transformer language model consists of `num_layers` identical layers (commonly called Transformer "blocks"). Each Transformer block takes in an input of shape `(batch_size, sequence_length, d_model)` and returns an output of shape `(batch_size, sequence_length, d_model)`. Each block aggregates information across the sequence (via self-attention) and non-linearly transforms it (via the feed-forward layers).

3.2 Output Normalization and Embedding

After `num_layers` Transformer blocks, we will take the final activations and turn them into a distribution over the vocabulary.

We will implement the "pre-norm" Transformer block (detailed in §3.5), which additionally requires the use of layer normalization (detailed below) after the final Transformer block to ensure its outputs are properly scaled.

After this normalization, we will use a standard learned linear transformation to convert the output of the Transformer blocks into predicted next-token logits (see, e.g., Radford et al. [2018] equation 2).

3.3 Remark: Batching, Einsum and Efficient Computation

Throughout the Transformer, we will be performing the same computation applied to many batch-like inputs. Here are a few examples:

- **Elements of a batch**: we apply the same Transformer `forward` operation on each batch element.
- **Sequence length**: the "position-wise" operations like RMSNorm and feed-forward operate identically on each position of a sequence.
- **Attention heads**: the attention operation is batched across attention heads in a "multi-headed" attention operation.

It is useful to have an ergonomic way of performing such operations in a way that fully utilizes the GPU, and is easy to read and understand. Many PyTorch operations can take in excess "batch-like" dimensions at the start of a tensor and repeat/broadcast the operation across these dimensions efficiently.

For instance, say we are doing a position-wise, batched operation. We have a "data tensor" D of shape `(batch_size, sequence_length, d_model)`, and we would like to do a batched vector-matrix multiply against a matrix A of shape `(d_model, d_model)`. In this case, `D @ A` will do a batched matrix multiply, which is an efficient primitive in PyTorch, where the `(batch_size, sequence_length)` dimensions are batched over.

Because of this, it is helpful to assume that your functions may be given additional batch-like dimensions and to keep those dimensions at the start of the PyTorch shape. To organize tensors so they can be batched in this manner, they might need to be shaped using many steps of `view`, `reshape` and `transpose`. This can be a bit of a pain, and it often gets hard to read what the code is doing and what the shapes of your tensors are.

A more ergonomic option is to use *einsum notation* within `torch.einsum`, or rather use framework agnostic libraries like `einops` or `einx`. The two key ops are `einsum`, which can do tensor contractions with arbitrary dimensions of input tensors, and `rearrange`, which can reorder, concatenate, and split arbitrary

具体而言，当给定一组标记ID时，Transformer语言模型通过标记嵌入层生成向量序列。每个嵌入层接收一个形状为（batch_size，sequence_length）的整数张量作为输入，最终输出形状为（batch_size，sequence_length，d_model）的向量序列。

3.1.2 预归一化变换器模块

激活向量经嵌入后，会依次通过多个结构相同的神经网络层进行处理。标准的Transformer语言模型仅包含解码器，由num_layers个相同结构的层组成（通常称为Transformer“模块”）。每个模块接收输入数据（包含batch_size、sequence_length和d_model三个维度），并输出相同维度的输出结果。模块通过自注意力机制对序列信息进行聚合，并借助前馈层实现非线性变换。

3.2 输出归一化与嵌入

在经过num_layers个Transformer模块后，我们将获取最终激活值，并将其转化为词汇表上的分布。

我们将实现“预归一化”Transformer模块（详见第3.5节），该模块在最终Transformer模块后还需使用层归一化（详见下文），以确保其输出被正确缩放。

完成标准化处理后，我们将采用标准学习线性变换将Transformer模块的输出转换为预测的下一个标记logits（参见例如Radford等人的研究）。2018] 公式2)。

3.3 批处理、求和与高效计算

在Transformer模型中，我们将对多个批处理类输入执行相同的计算操作。以下列举若干示例：

- **批量元素**：我们对每个批次元素应用相同的Transformer前向操作。
- **序列长度**：诸如RMSNorm和前馈等“逐位”操作对序列的每个位置均以相同方式执行。
- **注意力头**：注意力操作通过“多头”注意力机制对多个注意力头进行批量处理。

采用符合人体工学的操作方式能充分发挥GPU性能，同时确保操作流程清晰易懂。PyTorch的多数操作支持在张量起始时接收超批量维度，并能高效地沿这些维度重复或广播执行。

举个例子，假设我们正在执行逐位置批量操作。我们有一个形状为（batch_size，sequence_length，d_model）的“数据张量” D，并希望对形状为（d_model，d_model）的矩阵 A 进行批量向量-矩阵乘法运算。此时，D @ A将执行批量矩阵乘法运算——这是PyTorch中一种高效的底层运算，其中（batch_size，sequence_length）维度会被批量处理。

因此，建议将函数视为可能包含额外的批处理维度，并将这些维度保留在PyTorch结构的起始位置。若要实现这种批量处理，可能需要通过多次视图变换、重塑和转置操作来调整张量结构。这种操作过程相当繁琐，且代码逻辑和张量结构往往难以清晰呈现。

更符合人体工程学的选择是在torch.einsum中使用*einsum表示法*，或者更准确地说，使用框架无关的库如einops或einx。两个关键操作是einsum，它可以对任意维度的输入张量进行张量收缩，以及rearrange，它可以重新排序、连接和分割任意张量。

dimensions. It turns out almost all operations in machine learning are some combination of dimension juggling and tensor contraction with the occasional (usually pointwise) nonlinear function. This means that a lot of your code can be more readable and flexible when using einsum notation.

We **strongly** recommend learning and using einsum notation for the class. Students who have not been exposed to einsum notation before should use `einops` (docs here), and students who are already comfortable with `einops` should learn the more general `einx` (here).[4] Both packages are already installed in the environment we've supplied.

Here we give some examples of how einsum notation can be used. These are a supplement to the documentation for einops, which you should read first.

Example (`einstein_example1`): Batched matrix multiplication with `einops.einsum`

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#                           D                A     ->          Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

Example (`einstein_example2`): Broadcasted operations with `einops.rearrange`

We have a batch of images, and for each image we want to generate 10 dimmed versions based on some scaling factor:

```
images = torch.randn(64, 128, 128, 3)  # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by,    "dim_value              -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

[4] It's worth noting that while `einops` has a great amount of support, `einx` is not as battle-tested. You should feel free to fall back to using `einops` with some more plain PyTorch if you find any limitations or bugs in `einx`.

维度。事实证明，机器学习中的几乎所有操作都包含维度调整和张量收缩的组合，偶尔还会涉及（通常逐点）非线性函数。这意味着使用einsum符号时，代码的可读性和灵活性会显著提升。

我们 **强烈** 建议学习和使用 einsum 符号表示法。对于之前没有接触过 einsum 符号表示法的学生，应使用 einops（文档 在这里）；而对于已经熟悉 einops 的学生，则应学习更通用的 einx（在这里）。[4] 两个软件包都已安装在我们提供的环境中。

本文列举了einsum符号的若干应用实例，作为einsops文档的补充说明，建议读者先阅读einsops文档。

示例（einstein_example1）：使用einops进行批量矩阵乘法运算。einsum

```
进口火炬
从 einops 导入 重排，einsum

## Basic implementation
Y = D @ A.T
 # 很难分辨输入和输出的形状及其含义。
 # D和A可以呈现哪些形状？这些形状中是否存在意外表现？

## Einsum is self-documenting and robust
#                          D                A     ->          Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

 或者，D可以具有任何前导维度，但A受到约束的批量版本。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例（einstein_example2）：使用 einops .rearrange 进行广播操作

我们有一批图像，针对每张图像，我们希望基于某个缩放因子生成10个渐变版本：

```
images = torch.randn(64, 128, 128, 3)  # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by,    "dim_value              -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
     “批次高度宽度通道，dim_value -> 批次 dim_value 高度宽度通道”  )
```

[4]值得注意的是，虽然einops拥有强大的支持体系，但einx的实战测试经验相对不足。若在einx中发现任何功能限制或漏洞，建议您不妨退而求其次，改用更基础的PyTorch来使用einops。

Example (einstein_example3): Pixel mixing with einops.rearrange

Suppose we have a batch of images represented as a tensor of shape (batch, height, width, channel), and we want to perform a linear transformation across all pixels of the image, but this transformation should happen independently for each channel. Our linear transformation is represented as a matrix B of shape (height $\times$ width, height $\times$ width).

```
channels_last = torch.randn(64, 32, 32, 3)  # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)

channels_first_flat_transformed = channels_first_flat @ B.T

channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)

channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

Instead, using einops:

```
height = width = 32
## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

Or, if you're feeling crazy: all in one go using einx.dot (einx equivalent of einops.einsum)

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

The first implementation here could be improved by placing comments before and after to indicate

示例（einstein_example3）：使用 einops .rearrange 进行像素混合

假设我们有一批图像，表示为形状为（批次，高度，宽度，通道）的张量，我们希望对图像的所有像素执行线性变换，但该变换应针对每个通道独立进行。我们的线性变换表示为形状为（高度×宽度，高度×宽度）的矩阵B。

```
channels_last = torch.randn(64, 32, 32, 3)  # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

 ## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)

channels_first_flat_transformed = channels_first_flat @ B.T

channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)

channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

相反，使用einops：

```
height = width = 32
 ## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
```

高度= 高度，宽度= 宽度）

或者，如果你觉得太疯狂了：直接用einx . dot（相当于einops . einsum的einx）一次性搞定

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

此处的首个实施方案可通过在前后添加注释以示说明而得到改进

what the input and output shapes are, but this is clunky and susceptible to bugs. With einsum notation, documentation *is* implementation!

Einsum notation can handle arbitrary input batching dimensions, but also has the key benefit of being *self-documenting*. It's much clearer what the relevant shapes of your input and output tensors are in code that uses einsum notation. For the remaining tensors, you can consider using Tensor type hints, for instance using the `jaxtyping` library (not specific to Jax).

We will talk more about the performance implications of using einsum notation in assignment 2, but for now know that they're almost always better than the alternative!

3.3.1 Mathematical Notation and Memory Ordering

Many machine learning papers use row vectors in their notation, which result in representations that mesh well with the row-major memory ordering used by default in NumPy and PyTorch. With row vectors, a linear transformation looks like

$$y = xW^\top, \tag{1}$$

for row-major $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and row-vector $x \in \mathbb{R}^{1 \times d_{\text{in}}}$.

In linear algebra it's generally more common to use column vectors, where linear transformations look like

$$y = Wx, \tag{2}$$

given a row-major $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and column-vector $x \in \mathbb{R}^{d_{\text{in}}}$. **We will use column vectors** for mathematical notation in this assignment, as it is generally easier to follow the math this way. You should keep in mind that if you want to use plain matrix multiplication notation, you will have to apply matrices using the row vector convention, since PyTorch uses row-major memory ordering. If you use `einsum` for your matrix operations, this should be a non-issue.

3.4 Basic Building Blocks: Linear and Embedding Modules

3.4.1 Parameter Initialization

Training neural networks effectively often requires careful initialization of the model parameters—bad initializations can lead to undesirable behavior such as vanishing or exploding gradients. Pre-norm transformers are unusually robust to initializations, but they can still have a siginificant impact on training speed and convergence. Since this assignment is already long, we will save the details for assignment 3, and instead give you some approximate initializations that should work well for most cases. For now, use:

- Linear weights: $\mathcal{N}\left(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$ truncated at $[-3\sigma, 3\sigma]$.
- Embedding: $\mathcal{N}\left(\mu = 0, \sigma^2 = 1\right)$ truncated at $[-3, 3]$
- RMSNorm: $\mathbb{1}$

You should use `torch.nn.init.trunc_normal_` to initialize the truncated normal weights.

3.4.2 Linear Module

Linear layers are a fundamental building block of Transformers and neural nets in general. First, you will implement your own `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation:

$$y = Wx. \tag{3}$$

Note that we do not include a bias term, following most modern LLMs.

虽然能显示输入输出的形状，但操作笨拙且容易出错。采用einsum符号表示法时，文档本身 *就是* 实现！

einsum表示法可以处理任意输入批量维度，但也有一个关键优势，即 *自动文档化* 。在使用einsum表示法的代码中，输入和输出张量的相关形状会更加清晰。对于剩余的张量，可以考虑使用Tensor类型提示，例如使用 jaxtyping库（非Jax专用）。

我们将在作业2中深入探讨使用einsum符号对性能的影响，但目前可以确定的是，这种符号几乎总是优于其他替代方案！

3.3.1 数学符号与记忆排序

许多机器学习论文采用行向量的符号表示法，这种表示方式与NumPy和PyTorch默认采用的行优先内存顺序完美契合。对于行向量而言，线性变换可表示为

$$y = xW^{\top}, \tag{1}$$

对于行优先 $W \in \mathrm{R}^{d_{\text{out}} \times d_{\text{in}}}$ 和行向量 $x \in \mathrm{R}^{1 \times d_{\text{in}}}$ 。

在线性代数中，通常更常用列向量，此时线性变换的形式为

$$y = Wx, \tag{2}$$

给定行优先$W \in \mathrm{R}^{d_{\text{out}} \times d_{\text{in}}}$ 和列向量$x \in \mathrm{R}^{d_{\text{in}}}$。**本作业中我们将使用列向量** 进行数学符号表示，因为这种方式通常更便于理解数学逻辑。需要特别注意的是，若要使用标准矩阵乘法符号，必须采用行向量约定来操作矩阵，因为PyTorch采用行优先内存排序。若使用einsum进行矩阵运算，则无需考虑该问题。

3.4 基本构件:线性与嵌入模块

3.4.1 参数初始化

要有效训练神经网络，通常需要对模型参数进行谨慎初始化——错误的初始化可能导致梯度消失或爆炸等不良行为。预归一化变换器对初始化异常稳健，但仍可能显著影响训练速度和收敛效果。由于本作业篇幅较长，我们将把具体细节留到第三题，这里先给出几个适用于大多数情况的近似初始化方案。目前建议使用：

- 线性权重: $\mathcal{N}\left(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$ 截断于 $[-3\sigma, 3\sigma]$。
- 嵌入：$N(\mu = 0, \sigma^2 = 1)$ 在 $[-3, 3]$ 处截断
- RMSNorm：1

你应该使用 torch . nn . init . trunc_normal_to 来初始化截断正态权重。

3.4.2 线性模块

线性层是Transformer模型及神经网络的基础构建模块。首先，您需要实现一个继承自torch.nn.Module的线性类，用于执行线性变换：

$$y = Wx. \tag{3}$$

需注意，我们未包含偏置项，这与大多数现代大语言模型（LLM）的做法一致。

Problem (linear): Implementing the linear module (1 point)

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch's built-in `nn.Linear` module, except for not having a `bias` argument or parameter. We recommend the following interface:

`def __init__(self, in_features, out_features, device=None, dtype=None)` Construct a linear transformation module. This function should accept the following parameters:

`in_features: int` final dimension of the input

`out_features: int` final dimension of the output

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

`def forward(self, x: torch.Tensor) -> torch.Tensor` Apply the linear transformation to the input.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- construct and store your parameter as W (not $W^\top$) for memory ordering reasons, putting it in an `nn.Parameter`
- of course, don't use `nn.Linear` or `nn.functional.linear`

For initializations, use the settings from above along with `torch.nn.init.trunc_normal_` to initialize the weights.
To test your `Linear` module, implement the test adapter at `[adapters.run_linear]`. The adapter should load the given weights into your `Linear` module. You can use `Module.load_state_dict` for this purpose. Then, run `uv run pytest -k test_linear`.

3.4.3 Embedding Module

As discussed above, the first layer of the Transformer is an embedding layer that maps integer token IDs into a vector space of dimension `d_model`. We will implement a custom `Embedding` class that inherits from `torch.nn.Module` (so you should not use `nn.Embedding`). The `forward` method should select the embedding vector for each token ID by indexing into an embedding matrix of shape `(vocab_size, d_model)` using a `torch.LongTensor` of token IDs with shape `(batch_size, sequence_length)`.

Problem (embedding): Implement the embedding module (1 point)

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch's built-in `nn.Embedding` module. We recommend the following interface:

`def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)` Construct an embedding module. This function should accept the following parameters:

`num_embeddings: int` Size of the vocabulary

问题（线性）：实现线性模块（1分）

交付成果：实现一个继承自torch.nn.Module的线性类，用于执行线性变换。该实现应遵循PyTorch内置nn Linear模块的接口，但不包含偏置参数。推荐采用以下接口：

def init (self，in_features，out_features，device=None，dtype=None) 构建线性变换模块。该函数应接受以下参数：

in_features: int 输入的最终维度

out_features: int 输出的最终维度

device: torch .device | None = None 用于存储参数的设备

dtype: torch dtype | None = None 参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` Apply the linear transformation to the input.

务必做到：

- 子类 nn . 模块
- 调用超类构造函数
- 出于内存排序原因，将参数构造并存储为W（而非$W^\top$），并将其放入nn.参数中。
- 当然，不要使用nn.线性或nn.函数.线性

对于初始化，使用上述设置以及torch.nn.init.trunc_normal_to来初始化权重。
为测试线性模块，请在[adapters.run_linear]处实现测试适配器。该适配器需将指定权重加载至线性模块，可使用模块.load_state_dict完成此操作。随后运行uv run pytest -k test_linear。

3.4.3 嵌入式模块

如前所述，Transformer的第一层是嵌入层，其将整数标记ID映射到维度为d_model的向量空间。我们将实现一个自定义Embedding类，该类继承自torch.nn.Module（因此不应使用nn.Embedding）。前向方法应通过使用形状为(batch_size，sequence_length)的标记ID长张量，对形状为vocab_size，d_model的嵌入矩阵进行索引，从而为每个标记ID选择对应的嵌入向量。

问题（嵌入）：实现嵌入模块（1分）

可交付成果：实现继承自torch.nn.Module的Embedding类并执行嵌入查找。您的实现应遵循PyTorch内置接口规范nn. 嵌入模块。我们推荐以下接口：

def init (self，num_embeddings，embedding_dim，device=None，dtype=None) 构建一个嵌入模块。该函数应接受以下参数：

num_embeddings: int 词汇表大小

`embedding_dim: int` Dimension of the embedding vectors, i.e., d_{model}

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

`def forward(self, token_ids: torch.Tensor) -> torch.Tensor` Lookup the embedding vectors for the given token IDs.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as a `nn.Parameter`
- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.
To test your implementation, implement the test adapter at `[adapters.run_embedding]`. Then, run `uv run pytest -k test_embedding`.

3.5 Pre-Norm Transformer Block

Each Transformer block has two sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network (Vaswani et al., 2017, section 3.1).

In the original Transformer paper, the model uses a residual connection around each of the two sub-layers, followed by layer normalization. This architecture is commonly known as the "post-norm" Transformer, since layer normalization is applied to the sublayer output. However, a variety of work has found that moving layer normalization from the output of each sub-layer to the input of each sub-layer (with an additional layer normalization after the final Transformer block) improves Transformer training stability [Nguyen and Salazar, 2019, Xiong et al., 2020]—see Figure 2 for a visual representation of this "pre-norm" Transformer block. The output of each Transformer block sub-layer is then added to the sub-layer input via the residual connection (Vaswani et al., 2017, section 5.4). An intuition for pre-norm is that there is a clean "residual stream" without any normalization going from the input embeddings to the final output of the Transformer, which is purported to improve gradient flow. This pre-norm Transformer is now the standard used in language models today (e.g., GPT-3, LLaMA, PaLM, etc.), so we will implement this variant. We will walk through each of the components of a pre-norm Transformer block, implementing them in sequence.

3.5.1 Root Mean Square Layer Normalization

The original Transformer implementation of Vaswani et al. [2017] uses layer normalization [Ba et al., 2016] to normalize activations. Following Touvron et al. [2023], we will use root mean square layer normalization (RMSNorm; Zhang and Sennrich, 2019, equation 4) for layer normalization. Given a vector $a \in \mathbb{R}^{d_{\text{model}}}$ of activations, RMSNorm will rescale each activation a_i as follows:

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \tag{4}$$

where $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$. Here, g_i is a learnable "gain" parameter (there are `d_model` such parameters total), and ε is a hyperparameter that is often fixed at 1e-5.

embedding_dim: int 嵌入向量的维度，即·， d_{model}

device: torch .device | None = None 用于存储参数的设备

dtype: torch dtype | None = None 参数的数据类型

`def forward(self, token_ids: torch.Tensor) -> torch.Tensor` Lookup the embedding vectors for the given token IDs.

务必做到：

- 子类 nn . 模块
- 调用超类构造函数
- 将嵌入矩阵初始化为一个nn。参数
- 存储嵌入矩阵，其中d_model为最终维度
- 当然，不要使用nn.嵌入或nn.函数.嵌入

同样，使用上面的设置进行初始化，并使用torch . nn . init . trunc_normal_to初始化权重。
为测试您的实现，请在[adapters.run_embedding]处部署测试适配器，随后执行uv run pytest -k test_embedding命令。

3.5 预归一化变换器模块

每个Transformer模块包含两个子层：多头自注意力机制和位置级前馈网络（Vaswani等人，2017，第3.1节）。

在原始Transformer论文中，模型在两个子层的每个位置使用残差连接，随后进行层归一化。这种架构通常被称为“后归一化”Transformer，因为层归一化应用于子层输出。然而，多项研究发现，将层归一化从每个子层的输出移至输入（并在最终Transformer模块后增加一个额外的层归一化）可以提高Transformer的训练稳定性[Nguyen和 Salazar，2019，Xiong等人，2020]——参见图2中该“前归一化”Transformer模块的可视化表示。每个Transformer模块子层的输出随后通过残差连接与子层输入相加（Vaswani等人，2017，第5.4节）。前归一化的直观理解是：从输入嵌入到Transformer最终输出存在一条未经归一化的“残差流”，这被认为能改善梯度流动。这种预归一化Transformer现已成为当前语言模型（如GPT-3、LLaMA、PaLM等）的标准架构，因此我们将实现该变体。我们将按顺序逐步实现预归一化Transformer模块的各个组件。

3.5.1 均方根层归一化

Vaswani等人的原始Transformer实现2017] 采用层归一化[Ba等人，2016]对激活值进行归一化处理。遵循Touvron等人[2023]，我们将使用均方根层归一化（RMSNorm；Zhang和Sennrich，2019，公式4）进行层归一化。给定一个激活向量$a \in \mathrm{R}^{d_{\text{model}}}$，RMSNorm将按如下方式重新缩放每个激活值$a_i$：

$$\mathrm{RMSNorm}(a_i) = \frac{a_i}{\mathrm{RMS}(a)} g_i, \tag{4}$$

均方根值$(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$. 此处，$g_i$是一个可学习的“增益”参数（总共有d_model个此类参数），而ε是一个通常固定为1e-5的超参数。

You should upcast your input to `torch.float32` to prevent overflow when you square the input. Overall, your `forward` method should look like:

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Deliverable: Implement RMSNorm as a `torch.nn.Module`. We recommend the following interface:

`def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)` Construct the RMSNorm module. This function should accept the following parameters:

`d_model: int` Hidden dimension of the model

`eps: float = 1e-5` Epsilon value for numerical stability

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

`def forward(self, x: torch.Tensor) -> torch.Tensor` Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.
To test your implementation, implement the test adapter at `[adapters.run_rmsnorm]`. Then, run `uv run pytest -k test_rmsnorm`.

应将输入提升为torch.float32类型，以避免平方运算时发生溢出。总体而言，前向方法应呈现如下形式：

```
in_dtype = x.dtype
x = x.to(torch.float32)

此处为执行均方根归一化（RMSNorm）的代码
...
结果= ...

# 返回原始数据类型的返回结果
返回 结果 。 到 (in_dtype)
```

问题（rmsnorm）：均方根层归一化（1分）

交付成果：将RMSNorm实现为torch.nn.Module。推荐使用以下接口：

def init (self ， d_model: int ， eps: float = 1e-5，device= None ， dtype= None)

构造RMSNorm模块。该函数应接受以下参数：**d_model: int** 模型的隐藏维度

eps: float = 1e-5 数值稳定性的ε值

device: torch .device | None = None 用于存储参数的设备

dtype: torch dtype | None = None 参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

注意： 请记得在执行归一化操作前（以及后续将数据降级为原始数据类型时），将输入数据提升为 torch.float 32 类型，如上所述。
要测试您的实现，请在 [adapters.run_rmsnorm] 处实现测试适配器。然后，运行 uv run pytest -k test_rmsnorm。

3.5.2 Position-Wise Feed-Forward Network

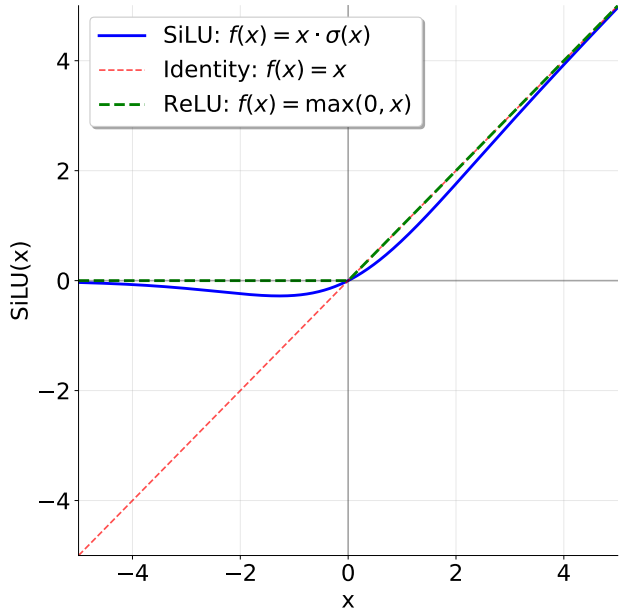

Figure 3: Comparing the SiLU (aka Swish) and ReLU activation functions.

In the original Transformer paper (section 3.3 of Vaswani et al. [2017]), the Transformer feed-forward network consists of two linear transformations with a ReLU activation ($\mathrm{ReLU}(x) = \max(0, x)$) between them. The dimensionality of the inner feed-forward layer is typically 4x the input dimensionality.

However, modern language models tend to incorporate two main changes compared to this original design: they use another activation function and employ a gating mechanism. Specifically, we will implement the "SwiGLU" activation function adopted in LLMs like Llama 3 [Grattafiori et al., 2024] and Qwen 2.5 [Yang et al., 2024], which combines the SiLU (often called Swish) activation with a gating mechanism called a Gated Linear Unit (GLU). We will also omit the bias terms sometimes used in linear layers, following most modern LLMs since PaLM [Chowdhery et al., 2022] and LLaMA [Touvron et al., 2023].

The SiLU or Swish activation function [Hendrycks and Gimpel, 2016, Elfwing et al., 2017] is defined as follows:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{5}$$

As can be seen in Figure 3, the SiLU activation function is similar to the ReLU activation function, but is smooth at zero.

Gated Linear Units (GLUs) were originally defined by Dauphin et al. [2017] as the element-wise product of a linear transformation passed through a sigmoid function and another linear transformation:

$$\mathrm{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \tag{6}$$

where $\odot$ represents element-wise multiplication. Gated Linear Units are suggested to "reduce the vanishing gradient problem for deep architectures by providing a linear path for the gradients while retaining non-linear capabilities."

Putting the SiLU/Swish and GLU together, we get the SwiGLU, which we will use for our feed-forward networks:

$$\mathrm{FFN}(x) = \mathrm{SwiGLU}(x, W_1, W_2, W_3) = W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x), \tag{7}$$

where $x \in \mathbb{R}^{d_{\text{model}}}$, $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, and canonically, $d_{\text{ff}} = \frac{8}{3} d_{\text{model}}$.

3.5.2 位置信息前馈网络

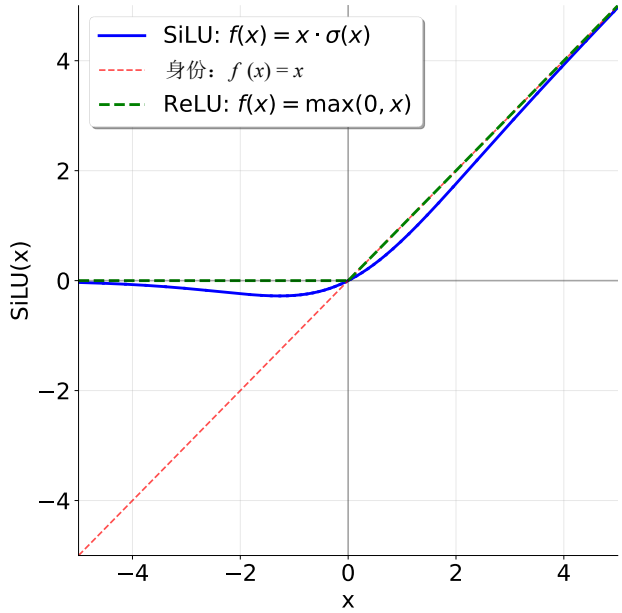

图3：SiLU（又称Swish）与ReLU激活函数的比较。

在原始Transformer论文中（Vaswani等人的第3.3节）2017]），Transformer前馈网络由两个线性变换组成，中间采用ReLU激活函数（ReLU（x）= max（0，x））。内部前馈层的维度通常是输入维度的4倍。

然而，与这一原始设计相比，现代语言模型往往包含两个主要变化：它们使用另一种激活函数并采用门控机制。具体而言，我们将实现类似Llama 3[Grattafiori等人，2024]和Qwen 2.5[Yang等人，2024]等大语言模型采用的“SwiGLU”激活函数，该函数将SiLU（通常称为Swish）激活与称为门控线性单元（GLU）的门控机制相结合。我们还将省略线性层中有时使用的偏置项，这是自PaLM[Chowdhery等人，2022]和LLaMA[Touvron等人，2023]以来大多数现代大语言模型的做法。

SiLU或Swish激活函数[Hendrycks和Gimpel，2016，Elfwing等人，2017]定义如下：

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{5}$$

如图3所示，SiLU激活函数与ReLU激活函数相似，但在零点处平滑。

门控线性单元（GLUs）最初由Dauphin等人定义[2017] 作为通过S型函数的线性变换与另一线性变换的逐元素乘积：

$$\mathrm{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \tag{6}$$

其中$\odot$表示逐元素乘法。门控线性单元（Gated Linear Units）被建议用于“通过提供梯度的线性路径同时保留非线性能力，从而解决深度架构中的梯度消失问题”。

将SiLU/Swish与GLU结合，我们得到SwiGLU，该网络将用于我们的前馈网络：

$$\mathrm{FFN}(x) = \mathrm{SwiGLU}(x, W_1, W_2, W_3) = W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x), \tag{7}$$

其中$x \in \mathrm{R}^{d_{\mathrm{model}}}$，$W_1$，$W_3 \in \mathrm{R}^{d_{\mathrm{ff}} \times d_{\mathrm{model}}}$，$W_2 \in \mathrm{R}^{d_{\mathrm{model}} \times d_{\mathrm{ff}}}$，且规范地，$d_{\mathrm{ff}} = \frac{8}{3} d_{\text{模型}}$。

Shazeer [2020] first proposed combining the SiLU/Swish activation with GLUs and conducted experiments showing that SwiGLU outperforms baselines like ReLU and SiLU (without gating) on language modeling tasks. Later in the assignment, you will compare SwiGLU and SiLU. Though we've mentioned some heuristic arguments for these components (and the papers provide more supporting evidence), it's good to keep an empirical perspective: a now famous quote from Shazeer's paper is

> We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

Problem (`positionwise_feedforward`): Implement the position-wise feed-forward network (2 points)

Deliverable: Implement the SwiGLU feed-forward network, composed of a SiLU activation function and a GLU.

Note: in this particular case, you should feel free to use `torch.sigmoid` in your implementation for numerical stability.

You should set d_{ff} to approximately $\frac{8}{3} \times d_{\text{model}}$ in your implementation, while ensuring that the dimensionality of the inner feed-forward layer is a multiple of 64 to make good use of your hardware. To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_swiglu]`. Then, run `uv run pytest -k test_swiglu` to test your implementation.

3.5.3 Relative Positional Embeddings

To inject positional information into the model, we will implement Rotary Position Embeddings [Su et al., 2021], often called RoPE. For a given query token $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$ at token position i, we will apply a pairwise rotation matrix R^i, giving us $q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}$. Here, R^i will rotate pairs of embedding elements $q^{(i)}_{2k-1:2k}$ as 2d vectors by the angle $\theta_{i,k} = \frac{i}{\Theta^{(2k-2)/d}}$ for $k \in \{1, \ldots, d/2\}$ and some constant Θ. Thus, we can consider R^i to be a block-diagonal matrix of size $d \times d$, with blocks R^i_k for $k \in \{1, \ldots, d/2\}$, with

$$R^i_k = \begin{bmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{bmatrix}. \tag{8}$$

Thus we get the full rotation matrix

$$R^i = \begin{bmatrix} R^i_1 & 0 & 0 & \ldots & 0 \\ 0 & R^i_2 & 0 & \ldots & 0 \\ 0 & 0 & R^i_3 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & R^i_{d/2} \end{bmatrix}, \tag{9}$$

where 0s represent 2×2 zero matrices. While one could construct the full $d \times d$ matrix, a good solution should use the properties of this matrix to implement the transformation more efficiently. Since we only care about the relative rotation of tokens within a given sequence, we can reuse the values we compute for $\cos(\theta_{i,k})$ and $\sin(\theta_{i,k})$ across layers, and different batches. If you would like to optimize it, you may use a single RoPE module referenced by all layers, and it can have a 2d pre-computed buffer of sin and cos values created during init with `self.register_buffer(persistent=False)`, instead of a `nn.Parameter` (because we do not want to learn these fixed cosine and sine values). The exact same rotation process we did for our $q^{(i)}$ is then done for $k^{(j)}$, rotating by the corresponding R^j. Notice that this layer has no learnable parameters.

Shazeer [2020] 首次提出将SiLU/Swish激活函数与GLU结合，并通过实验表明SwiGLU在语言模型任务中优于ReLU和SiLU（无门控机制）等基线方法。在后续任务中，您将比较SwiGLU与SiLU。尽管我们已对这些组件提出了一些启发式论据（论文中提供了更多支持证据），但保持实证视角仍具价值：Shazeer论文中一句如今广为人知的引述是

> 我们无法解释为何这些架构似乎有效；与其他一切因素一样，我们将它们的成功归因于神的恩典。

问题（位置级前馈）：实现位置级前馈网络（2分）

可交付成果：实现由SiLU激活函数和GLU组成的SwiGLU前馈网络。

注：在本特定情况下，为确保数值稳定性，您可自由采用。Sigmoid型火炬函数。

您应该将d_{ff}设置为大约$\frac{8}{3} \times d_{\text{模型}}$在您的实现中，同时确保内部前馈层的维度是64的倍数以充分利用硬件。为了用我们提供的测试来测试您的实现，您需要在[adapters.run_swiglu] 实现测试适配器。然后，运行 uv run pytest -k test_swiglu 来测试您的实现。

3.5.3 相对位置嵌入

为将位置信息注入模型，我们将实现旋转位置嵌入[Su等人，2021]，通常称为RoPE。对于给定的查询标记$q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$在标记位置$i$，我们将应用成对旋转矩阵$R^i$，得到$q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}$。此处$R^i$将通过角度$\theta_{i,k}$=将成对的嵌入元素$q_{2(ik)-1:2k}$旋转为二维向量$\frac{i}{\Theta^{(2k-2)/d}}$对于$k \in \{1, \ldots, d/2\}$以及某个常数$\Theta$。因此，我们可以将$R^i$视为一个大小为$d \times d$的块对角矩阵，其块$R_{ik}$对应于$k \in \{1, \ldots, d/2\}$，且

$$R_k^i = \begin{bmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{bmatrix}. \tag{8}$$

由此我们得到完整的旋转矩阵

$$R^i = \begin{bmatrix} R_1^i & 0 & 0 & \ldots & 0 \\ 0 & R_2^i & 0 & \ldots & 0 \\ 0 & 0 & R_3^i & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & R_{d/2}^i \end{bmatrix}, \tag{9}$$

其中0s代表2×2零矩阵。虽然可以构建完整的$d \times$矩阵，但一个好的解决方案应利用该矩阵的特性来更高效地实现变换。由于我们只关心给定序列中标记的相对旋转，可以跨层和不同批次复用计算的$\cos(\theta_{i,k})$和$\sin(\theta_{i,k})$值。若需优化，可使用所有层引用的单一RoPE模块，并在初始化时通过self.register_buffer（持久化=**False**）创建二维预计算的sin和cos值缓冲区，而非nn.参数（因为我们不想学习这些固定的余弦和正弦值）。随后对$k^{(j)}$执行与$q^{(i)}$完全相同的旋转过程，通过对应的R^j进行旋转。注意该层没有可学习参数。

Problem (rope): Implement RoPE (2 points)

Deliverable: Implement a class `RotaryPositionalEmbedding` that applies RoPE to the input tensor. The following interface is recommended:

`def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)` Construct the RoPE module and create buffers if needed.

`theta: float` Θ value for the RoPE

`d_k: int` dimension of query and key vectors

`max_seq_len: int` Maximum sequence length that will be inputted

`device: torch.device | None = None` Device to store the buffer on

`def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor` Process an input tensor of shape `(..., seq_len, d_k)` and return a tensor of the same shape. Note that you should tolerate x with an arbitrary number of batch dimensions. You should assume that the token positions are a tensor of shape `(..., seq_len)` specifying the token positions of x along the sequence dimension.

You should use the token positions to slice your (possibly precomputed) cos and sin tensors along the sequence dimension.

To test your implementation, complete `[adapters.run_rope]` and make sure it passes `uv run pytest -k test_rope`.

3.5.4 Scaled Dot-Product Attention

We will now implement scaled dot-product attention as described in Vaswani et al. [2017] (section 3.2.1). As a preliminary step, the definition of the Attention operation will make use of softmax, an operation that takes an unnormalized vector of scores and turns it into a normalized distribution:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^{n} \exp(v_j)}. \tag{10}$$

Note that $\exp(v_i)$ can become `inf` for large values (then, `inf/inf` = `NaN`). We can avoid this by noticing that the softmax operation is invariant to adding any constant c to all inputs. We can leverage this property for numerical stability—typically, we will subtract the largest entry of o_i from all elements of o_i, making the new largest entry 0. You will now implement softmax, using this trick for numerical stability.

Problem (softmax): Implement softmax (1 point)

Deliverable: Write a function to apply the softmax operation on a tensor. Your function should take two parameters: a tensor and a *dimension* i, and apply softmax to the i-th dimension of the input tensor. The output tensor should have the same shape as the input tensor, but its i-th dimension will now have a normalized probability distribution. Use the trick of subtracting the maximum value in the i-th dimension from all elements of the i-th dimension to avoid numerical stability issues.

To test your implementation, complete `[adapters.run_softmax]` and make sure it passes `uv run pytest -k test_softmax_matches_pytorch`.

We can now define the Attention operation mathematically as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q^\top K}{\sqrt{d_k}}\right) V \tag{11}$$

问题（绳索）：实施RoPE（2分）

可交付成果：实现一个名为 RotaryPositionalEmbedding 的类，该类将旋转位置嵌入（RoPE）应用于输入张量。推荐使用以下接口：

def init (self，theta: float，d_k: int，max_seq_len: int，device=None) 构建RoPE模块并根据需要创建缓冲区。

theta: float RoPE的 Θ 值

d_k: int 查询向量和键向量的维度

max_seq_len: int 最大输入序列长度

device: torch .device | None = None 用于存储缓冲区的设备

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

处理形状为(...，seq_len，d_k)的输入张量，并返回形状相同的张量。注意应能容忍具有任意批次维度的x。应假设标记位置是一个形状为(...，seq_len)的张量，用于指定x在序列维度上的标记位置。

应利用标记位置沿序列维度切分（可能已预计算的）余弦和正弦张量。

为测试您的实现，请完成 [adapters.run_rope] 并确保其通过 uv run pytest -k test_rope。

3.5.4 标度点积注意力

我们将按照Vaswani等人所述方法实现缩放点积注意力机制。2017] （第3.2.1节）。作为初步步骤，注意力操作的定义将采用softmax函数，该操作将未归一化的分数向量转换为归一化分布：

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^{n} \exp(v_j)}. \tag{10}$$

注意 exp (v_i) 在取值较大时可能变为无穷大（此时 inf/inf = NaN）。我们可以通过注意到 softmax 操作对向所有输入添加任意常数 c 具有不变性来避免这种情况。我们可以利用这一特性来提高数值稳定性——通常，我们会从 o_i 的所有元素中减去 o_i 的最大值，使新的最大值变为 0。现在您将使用这一技巧来实现 softmax，以提高数值稳定性。

问题（softmax）：实现softmax函数（1分）

可交付成果：编写一个函数对张量应用softmax操作。该函数应接受两个参数：一个张量和一个*维度i*，并对输入张量的第i维进行softmax操作。输出张量应与输入张量具有相同形状，但其第i维将具有归一化概率分布。使用从第i维所有元素中减去第i维最大值的技巧以避免数值稳定性问题。

为测试您的实现，请完成[adapters.run_softmax]并确保其通过uv run pytest -k test_softmax_matches_pytorch。

现可将注意力操作从数学角度定义如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q^\top K}{\sqrt{d_k}}\right) V \tag{11}$$

where $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, and $V \in \mathbb{R}^{m \times d_v}$. Here, Q, K and V are all inputs to this operation—note that these are not the learnable parameters. If you're wondering why this isn't $QK^\top$, see 3.3.1.

Masking: It is sometimes convenient to *mask* the output of an attention operation. A mask should have the shape $M \in \{\texttt{True}, \texttt{False}\}^{n \times m}$, and each row i of this boolean matrix indicates which keys the query i should attend to. Canonically (and slightly confusingly), a value of `True` at position (i, j) indicates that the query i *does* attend to the key j, and a value of `False` indicates that the query *does not* attend to the key. In other words, "information flows" at (i, j) pairs with value `True`. For example, consider a 1×3 mask matrix with entries [[`True`, `True`, `False`]]. The single query vector attends only to the first two keys.

Computationally, it will be much more efficient to use masking than to compute attention on subsequences, and we can do this by taking the pre-softmax values $\left(\frac{Q^\top K}{\sqrt{d_k}}\right)$ and adding a $-\infty$ in any entry of the mask matrix that is False.

Problem (`scaled_dot_product_attention`): Implement scaled dot-product attention (5 points)

Deliverable: Implement the scaled dot-product attention function. Your implementation should handle keys and queries of shape `(batch_size, ..., seq_len, d_k)` and values of shape `(batch_size, ..., seq_len, d_v)`, where `...` represents any number of other batch-like dimensions (if provided). The implementation should return an output with the shape `(batch_size, ..., d_v)`. See section 3.3 for a discussion on batch-like dimensions.
Your implementation should also support an optional user-provided boolean mask of shape `(seq_len, seq_len)`. The attention probabilities of positions with a mask value of `True` should collectively sum to 1, and the attention probabilities of positions with a mask value of `False` should be zero.
To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_scaled_dot_product_attention]`.
`uv run pytest -k test_scaled_dot_product_attention` tests your implementation on third-order input tensors, while `uv run pytest -k test_4d_scaled_dot_product_attention` tests your implementation on fourth-order input tensors.

3.5.5 Causal Multi-Head Self-Attention

We will implement multi-head self-attention as described in section 3.2.2 of Vaswani et al. [2017]. Recall that, mathematically, the operation of applying multi-head attention is defined as follows:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \tag{12}$$

$$\text{for head}_i = \text{Attention}(Q_i, K_i, V_i) \tag{13}$$

with Q_i, K_i, V_i being slice number $i \in \{1, \ldots, h\}$ of size d_k or d_v of the embedding dimension for Q, K, and V respectively. With Attention being the scaled dot-product attention operation defined in §3.5.4. From this we can form the multi-head *self*-attention operation:

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \tag{14}$$

Here, the learnable parameters are $W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$, $W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$, $W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$, and $W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$. Since the Qs, K, and Vs are sliced in the multi-head attention operation, we can think of W_Q, W_K and W_V as being separated for each head along the output dimension. When you have this working, you should be computing the key, value, and query projections in a total of three matrix multiplies.[5]

[5] As a stretch goal, try combining the key, query, and value projections into a single weight matrix so you only need a single matrix multiply.

其中$Q \in \mathrm{R}^{n\times d_k}$，$K \in \mathrm{R}^{m\times d_k}$，且$V \in \mathrm{R}^{m\times d_v}$。这里$Q$、$K$和$V$均为该操作的输入——注意这些并非可学习参数。若你疑惑为何这不是$QK^\top$，请参见3.3.1。

掩码： 有时需要对注意力操作的输出进行*掩码*处理。掩码应具有$M \in \{\text{True}, \text{False}\}^{n\times m}$的形状，该布尔矩阵的每一行$i$表示查询$i$应关注哪些键。规范上（略显令人困惑的是），位置（$i$，$j$）处的 True 值表示查询$i$ *确实* 关注键j，而 False 值则表示查询*不关注*该键。换言之，（i，j）处的 True 值对应“信息流”。例如，考虑一个 1 × 3 的掩码矩阵，其条目为[[True，True，False]]。单个查询向量仅关注前两个键。

从计算效率来看，采用掩码技术比在子序列上计算注意力更为高效，我们可通过获取预softmax值来实现这一目标。$\left(\frac{Q^\top K}{\sqrt{d_k}}\right)$ 并在掩码矩阵中任何为False的条目处添加一个$-\infty$。

问题（缩放点积注意力机制）：实现缩放点积注意力机制（5分）

交付成果：实现缩放点积注意力函数。该实现需处理形状为（batch_size，...，seq_len，d_k）的键值对和形状为（batch_size，...，seq_len，d_v）的值对，其中...表示其他批处理维度（如存在）。实现应返回形状为（batch_size，...，d_v）的输出。关于批处理维度的讨论，请参阅3.3节。

您的实现还应支持用户可选的布尔型掩码（seq_len，seq_len）。掩码值为True的位点注意力概率之和应为1，而掩码值为False的位点注意力概率应为零。

为测试您的实现是否符合我们提供的测试要求，您需要在[adapters.runScaled.Dot_Product_attention]处实现测试适配器。

uvrun pytest-ktestScaled.Dot_Product_attention在三阶输入张量上测试你的实现，而uvrun pytest-k test_4 d_scaled_dot_product_attention 在四阶输入张量上测试你的实现。

3.5.5 因果多头自注意力机制

我们将按照Vaswani等人第3.2.2节所述方法实现多头自注意力机制。2017]需注意的是，从数学角度而言，多头注意力机制的操作定义如下：

$$\text{MultiHead}(Q, K, V) = \text{连接}(\text{head}_1, \ldots, \text{head}_h) \tag{12}$$

$$\text{对于头部}_i = \text{注意力}(Q_i, K_i, V_i) \tag{13}$$

其中Q_i、K_i、V_i分别表示切片编号$i \in \{1, \ldots, h\}$的大小为d_k或d_v的嵌入维度，对应Q、K、 和V。Attention 为第3.5.4 节定义的缩放点积注意力操作。由此可构建多头*自* 注意力操作：

$$\text{MultiHeadSelfAttention}(x) = W_O\text{MultiHead}(W_Q x, W_K x, W_V x) \tag{14}$$

此处，可学习的参数为$W_Q \in \mathrm{R}^{hd_k\times d_{\text{model}}}$、$W_K \in \mathrm{R}^{hd_k\times d_{\text{model}}}$、$W_V \in \mathrm{R}^{hd_v\times d_{\text{model}}}$以及$W_O \in \mathrm{R}^{d_{\text{model}}\times hd_v}$。由于$Q$、$K$和$V$在多头注意力操作中被切片，我们可以将$W_Q$、$W_K$和$W_V$视为沿输出维度为每个头分离的。当这一机制生效时，您应通过总共三次矩阵乘法来计算键、值和查询投影。[5]

[5] 作为扩展目标，尝试将键、查询和值的投影合并为单一权重矩阵，这样只需进行一次矩阵乘法运算。

Causal masking. Your implementation should prevent the model from attending to future tokens in the sequence. In other words, if the model is given a token sequence $t_1, \ldots, t_n$, and we want to calculate the next-word predictions for the prefix $t_1, \ldots, t_i$ (where $i < n$), the model should *not* be able to access (attend to) the token representations at positions $t_{i+1}, \ldots, t_n$ since it will not have access to these tokens when generating text during inference (and these future tokens leak information about the identity of the true next word, trivializing the language modeling pre-training objective). For an input token sequence $t_1, \ldots, t_n$ we can naively prevent access to future tokens by running multi-head self-attention n times (for the n unique prefixes in the sequence). Instead, we'll use causal attention masking, which allows token i to attend to all positions $j \leq i$ in the sequence. You can use `torch.triu` or a broadcasted index comparison to construct this mask, and you should take advantage of the fact that your scaled dot-product attention implementation from §3.5.4 already supports attention masking.

Applying RoPE. RoPE should be applied to the query and key vectors, but not the value vectors. Also, the head dimension should be handled as a batch dimension, because in multi-head attention, attention is being applied independently for each head. This means that precisely the same RoPE rotation should be applied to the query and key vectors for each head.

Problem (`multihead_self_attention`): Implement causal multi-head self-attention (5 points)

Deliverable: Implement causal multi-head self-attention as a `torch.nn.Module`. Your implementation should accept (at least) the following parameters:

`d_model: int` Dimensionality of the Transformer block inputs.

`num_heads: int` Number of heads to use in multi-head self-attention.

Folllowing Vaswani et al. [2017], set $d_k = d_v = d_{\text{model}}/h$. To test your implementation against our provided tests, implement the test adapter at `[adapters.run_multihead_self_attention]`. Then, run `uv run pytest -k test_multihead_self_attention` to test your implementation.

3.6 The Full Transformer LM

Let's begin by assembling the Transformer block (it will be helpful to refer back to Figure 2). A Transformer block contains two 'sublayers', one for the multihead self attention, and another for the feed-forward network. In each sublayer, we first perform RMSNorm, then the main operation (MHA/FF), finally adding in the residual connection.

To be concrete, the first half (the first 'sub-layer') of the Transformer block should be implementing the following set of updates to produce an output y from an input x,

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \tag{15}$$

Problem (`transformer_block`): Implement the Transformer block (3 points)

Implement the pre-norm Transformer block as described in §3.5 and illustrated in Figure 2. Your Transformer block should accept (at least) the following parameters.

`d_model: int` Dimensionality of the Transformer block inputs.

`num_heads: int` Number of heads to use in multi-head self-attention.

`d_ff: int` Dimensionality of the position-wise feed-forward inner layer.

因果掩蔽。 您的实现应防止模型关注序列中的未来标记。换言之，若给定标记序列$t_1, \ldots, t_n$，且需计算前缀$t_1, \ldots, t_i$（其中$i < n$）的下一个词预测，模型应*不* 能访问（关注）位置$t_{i+1}, \ldots, t_n$处的标记表示，因为在推理生成文本时无法访问这些标记（这些未来标记会泄露真实下一个词的身份信息，使语言模型预训练目标失效）。对于输入标记序列$t_1, \ldots, t_n$，我们可简单地通过运行n次多头自注意力（针对序列中的n个唯一前缀）来防止对未来标记的访问。相反，我们将使用因果注意力掩码，它允许标记i关注序列中所有$j \leq i$的位置。你可以使用torch.triu或广播索引比较来构建这个掩码，并且你应该利用你从第3.5.4节中实现的缩放点积注意力已经支持注意力掩码的事实。

应用RoPE。RoPE应应用于查询向量和键向量，但不应用于值向量。此外，头维度应作为批处理维度处理，因为在多头注意力机制中，注意力是针对每个头独立应用的。这意味着每个头的查询向量和键向量都应应用完全相同的RoPE旋转。

问题（多头自注意力机制）：实现因果性多头自注意力机制（5分）

可交付成果：将因果多头自注意力机制实现为torch.nn.Module。实现时需至少接受以下参数：

d_model: int Transformer模块输入的维度。

num_heads: int 多头自注意力机制中使用的头数量。

Folllowing Vaswani等人[2017]，设置$d_k = d_v = d_{\text{model}}/h$。为测试您的实现是否符合我们提供的测试要求，请在[adapters.run_multihead_self_attention]处实现测试适配器。随后，运行uv run pytest -k test_multihead_self_attention以测试您的实现。

3.6 全变压器LM

首先组装Transformer模块（参考图2会很有帮助）。该模块包含两个子层：一个用于多头自注意力机制，另一个用于前馈网络。在每个子层中，我们首先执行RMSNorm归一化，接着进行主操作（多头自注意力/前馈网络），最后添加残差连接。

具体来说，Transformer模块的前半部分（第一个“子层”）应该实现以下更新集，以从输入x生成输出y，

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \tag{15}$$

问题（Transformer模块）：实现Transformer模块（3分）

按照第3.5节所述并结合图2所示，实现预归一化Transformer模块。该模块应至少接受以下参数。

d_model: int Transformer模块输入的维度。

num_heads: int 多头自注意力机制中使用的头数量。

d_ff: int 位置级前馈内层的维度。

To test your implementation, implement the adapter [adapters.run_transformer_block]. Then run `uv run pytest -k test_transformer_block` to test your implementation.

Deliverable: Transformer block code that passes the provided tests.

Now we put the blocks together, following the high level diagram in Figure 1. Follow our description of the embedding in Section 3.1.1, feed this into `num_layers` Transformer blocks, and then pass that into the three output layers to obtain a distribution over the vocabulary.

Problem (`transformer_lm`): Implementing the Transformer LM (3 points)

Time to put it all together! Implement the Transformer language model as described in §3.1 and illustrated in Figure 1. At minimum, your implementation should accept all the aforementioned construction parameters for the Transformer block, as well as these additional parameters:

`vocab_size: int` The size of the vocabulary, necessary for determining the dimensionality of the token embedding matrix.

`context_length: int` The maximum context length, necessary for determining the dimensionality of the position embedding matrix.

`num_layers: int` The number of Transformer blocks to use.

To test your implementation against our provided tests, you will first need to implement the test adapter at [adapters.run_transformer_lm]. Then, run `uv run pytest -k test_transformer_lm` to test your implementation.

Deliverable: A Transformer LM module that passes the above tests.

Resource accounting. It is useful to be able to understand how the various parts of the Transformer consume compute and memory. We will go through the steps to do some basic "FLOPs accounting." The *vast* majority of FLOPS in a Transformer are matrix multiplies, so our core approach is simple:

1. Write down all the matrix multiplies in a Transformer forward pass.
2. Convert each matrix multiply into FLOPs required.

For this second step, the following fact will be useful:

Rule: Given $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the matrix-matrix product AB requires $2mnp$ FLOPs.

To see this, note that $(AB)[i, j] = A[i, :] \cdot B[:, j]$, and that this dot product requires n additions and n multiplications ($2n$ FLOPs). Then, since the matrix-matrix product AB has $m \times p$ entries, the total number of FLOPS is $(2n)(mp) = 2mnp$.

Now, before you do the next problem, it can be helpful to go through each component of your Transformer block and Transformer LM, and list out all the matrix multiplies and their associated FLOPs costs.

Problem (`transformer_accounting`): Transformer LM resource accounting (5 points)

(a) Consider GPT-2 XL, which has the following configuration:

`vocab_size` : 50,257

`context_length` : 1,024

`num_layers` : 48

`d_model` : 1,600

为测试您的实现，请实现适配器[adapters.run_transformer_block]。然后运行uv run pytest -k test_transformer_block以测试您的实现。

可交付成果：通过指定测试的Transformer模块代码。

现在我们按照图1中的高层图将这些块组合起来。按照我们3.1.1节中对嵌入的描述，将这些块输入到num_layers Transformer块中，然后将这些块输入到三个输出层中，从而获得一个词汇表上的分布。

问题（Transformer LM）：实现Transformer LM（3分）

现在是时候把它们组合起来了！按照第3.1节所述和图1所示，实现Transformer语言模型。至少，你的实现应该接受Transformer块的所有上述构造参数，以及这些附加参数：

vocab_size: int 词表大小，用于确定标记嵌入矩阵的维度。

context_length: int 最大上下文长度，用于确定位置嵌入矩阵的维度。

num_layers: int Transformer模块的数量。

要通过我们提供的测试来验证您的实现，您需要先在[adapters.run_transformer_lm]处实现测试适配器。然后，运行uv run pytest -k test_transformer_lm来测试您的实现。

可交付成果：一个通过上述测试的Transformer LM模块。

资源核算。 了解Transformer各部分如何消耗计算和内存很有用。我们将通过一些基本的“FLOPs核算”步骤来说明。Transformer中*绝大多数*的FLOPs是矩阵乘法，所以我们的核心方法很简单:

1. 在Transformer前向传播过程中记录所有矩阵乘法运算。
2. 将每个矩阵乘法运算转换为所需浮点运算次数（FLOPs）。

在第二步操作中，以下事实将具有重要参考价值：

规则： 给定 $A \in \mathrm{R}^{m\times n}$ 和 $B \in \mathrm{R}^{n\times p}$，矩阵乘积 AB 需要 $2\,mnp$ 次浮点运算。

要理解这一点，注意 $(AB)[i, j] = A[i, :] \cdot B[:, j]$，且该点积需要 n 次加法和 n 次乘法（$2n$ 次浮点运算）。由于矩阵乘积 AB 有 $m \times p$ 个元素，总浮点运算次数为 $(2n)(mp) = 2\,mnp$。

在开始下一个问题之前，建议先逐项检查Transformer模块和Transformer LM的各个组件，列出所有矩阵乘法运算及其对应的浮点运算成本（FLOPs）。

问题（变压器核算）：变压器LM资源核算（5分）

(a) 以GPT-2 XL为例，其配置如下：

词汇量: 50,257
上下文长度: 1,024
层数: 48
d模型：1,600

`num_heads` : 25

`d_ff` : 6,400

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Deliverable: A one-to-two sentence response.

(b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has `context_length` tokens.

Deliverable: A list of matrix multiplies (with descriptions), and the total number of FLOPs required.

(c) Based on your analysis above, which parts of the model require the most FLOPs?

Deliverable: A one-to-two sentence response.

(d) Repeat your analysis with GPT-2 small (12 layers, 768 `d_model`, 12 heads), GPT-2 medium (24 layers, 1024 `d_model`, 16 heads), and GPT-2 large (36 layers, 1280 `d_model`, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Deliverable: For each model, provide a breakdown of model components and its associated FLOPs (as a proportion of the total FLOPs required for a forward pass). In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

(e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How do the relative contribution of FLOPs of the model components change?

Deliverable: A one-to-two sentence response.

头部数量: 25

`d_ff` ：6,400

假设我们使用该配置构建模型，该模型将包含多少可训练参数？若每个参数均采用单精度浮点数表示，仅加载该模型所需的内存容量是多少？

交付物：一至两句话的答复。

(b) 请确定完成GPT-2 XL模型前向传播所需的矩阵乘法运算。这些矩阵乘法总共需要多少浮点运算次数（FLOPs）？假设输入序列包含context_length个标记。

可交付成果：矩阵乘法运算列表（含说明）及所需浮点运算次数（FLOPs）总计。

(c) 根据您上述分析，该模型中哪些部分需要最多的浮点运算次数（FLOPs）？

交付物：一至两句话的答复。

(d) 使用GPT-2小模型（12层，768 d_model，12个头）、中模型（24层，1024 d_model，16个头）和大模型（36层，1280 d_model，20个头）重复分析。随着模型规模的增大，Transformer LM的哪些部分会按比例占据更多或更少的总浮点运算次数（FLOPs）？

可交付成果：针对每个模型，需详细说明各组件构成及其对应的浮点运算次数（FLOPs）占比（占前向传播所需总FLOPs的比例）。此外，需用一至两句话简要说明：调整模型规模时，各组件的FLOPs占比会如何变化。

(e) 以GPT-2 XL为例，将上下文长度增加至16,384。一次前向传播所需的总浮点运算次数（FLOPs）将如何变化？模型各组件FLOPs的相对贡献度将如何改变？

交付物：一至两句话的答复。

4 Training a Transformer LM

We now have the steps to preprocess the data (via tokenizer) and the model (Transformer). What remains is to build all of the code to support training. This consists of the following:

- **Loss:** we need to define the loss function (cross-entropy).
- **Optimizer:** we need to define the optimizer to minimize this loss (AdamW).
- **Training loop:** we need all the supporting infrastructure that loads data, saves checkpoints, and manages training.

4.1 Cross-entropy loss

Recall that the Transformer language model defines a distribution $p_\theta(x_{i+1} \mid x_{1:i})$ for each sequence x of length $m+1$ and $i = 1, \ldots, m$. Given a training set D consisting of sequences of length m, we define the standard cross-entropy (negative log-likelihood) loss function:

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^{m} -\log p_\theta(x_{i+1} \mid x_{1:i}). \tag{16}$$

(Note that a single forward pass in the Transformer yields $p_\theta(x_{i+1} \mid x_{1:i})$ for *all* $i = 1, \ldots, m$.)

In particular, the Transformer computes logits $o_i \in \mathbb{R}^{\texttt{vocab_size}}$ for each position i, which results in:[6]

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\texttt{vocab_size}} \exp(o_i[a])}. \tag{17}$$

The cross entropy loss is generally defined with respect to the vector of logits $o_i \in \mathbb{R}^{\texttt{vocab_size}}$ and target x_{i+1}.[7]

Implementing the cross entropy loss requires some care with numerical issues, just like in the case of softmax.

Problem (`cross_entropy`): Implement Cross entropy

Deliverable: Write a function to compute the cross entropy loss, which takes in predicted logits (o_i) and targets (x_{i+1}) and computes the cross entropy $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$. Your function should handle the following:

- Subtract the largest element for numerical stability.
- Cancel out log and exp whenever possible.
- Handle any additional batch dimensions and return the *average* across the batch. As with section 3.3, we assume batch-like dimensions always come first, before the vocabulary size dimension.

Implement `[adapters.run_cross_entropy]`, then run `uv run pytest -k test_cross_entropy` to test your implementation.

Perplexity Cross entropy suffices for training, but when we evaluate the model, we also want to report perplexity. For a sequence of length m where we suffer cross-entropy losses $\ell_1, \ldots, \ell_m$:

$$\text{perplexity} = \exp\left(\frac{1}{m} \sum_{i=1}^{m} \ell_i\right). \tag{18}$$

[6] Note that $o_i[k]$ refers to value at index k of the vector o_i.

[7] This corresponds to the cross entropy between the Dirac delta distribution over x_{i+1} and the predicted $\text{softmax}(o_i)$ distribution.

4 训练一个Transformer语言模型

目前我们已具备数据预处理（通过分词器）和模型（Transformer）的步骤。接下来需要构建支持训练的全部代码，具体包括以下内容：

- **损失函数：** 需要定义损失函数（交叉熵）。
- **优化器：** 需要定义用于最小化该损失的优化器（AdamW）。
- **训练循环：** 我们需要所有支持基础设施，包括数据加载、检查点保存和训练管理。

4.1 交叉熵损失

回想一下，Transformer语言模型为每个长度为$m+1$且$i=1, \ldots, m$的序列x定义了一个分布$p_\theta(x_{i+1} \mid x_{1:i})$。给定一个由长度为$m$的序列组成的训练集$D$，我们定义了标准交叉熵（负对数似然）损失函数：

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^{m} -\log p_\theta(x_{i+1} \mid x_{1:i}). \tag{16}$$

（注意，Transformer中单次前向传递会产生$p_\theta(x_{i+1} \mid x_{1:i})$，适用于*所有*$i=1, \ldots, m$。）具体而言，Transformer为每个位置$i$计算logits $o_i \in \mathbb{R}^{\texttt{vocab_size}}$，从而得到：[6]

$$p(x_{i+1} \mid x_{1:i}) = \mathrm{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\texttt{vocab_size}} \exp(o_i[a])}. \tag{17}$$

交叉熵损失通常定义为相对于向量logits $o_i \in \mathbb{R}^{\texttt{vocab_size}}$和目标[7] x_{i+1}.

实现交叉熵损失时需注意数值问题，这与softmax的情况类似。

问题（交叉熵）：实现交叉熵

可交付成果：编写一个计算交叉熵损失的函数，该函数接收预测的逻辑值（o_i）和目标值（x_{i+1}），并计算交叉熵 $l_i = -\log \mathrm{softmax}(o_i)[x_{i+1}]$。你的函数应处理以下内容：

- 为确保数值稳定性，需减去最大元素。
- 尽可能取消日志和专家的记录。
- 处理任何其他批处理维度，并返回批处理中的*平均值*。与第3.3节一样，我们假设批处理维度总是排在词汇量维度之前。

实现[adapters.run_cross_entropy]，然后运行uv run pytest -k test_cross_entropy来测试你的实现。

困惑度 交叉熵足以用于训练，但在评估模型时，我们还需要报告困惑度。对于长度为m的序列，我们计算交叉熵损失$l_1, \ldots, l_m$：

$$\text{perplexity} = \exp\left(\frac{1}{m}\sum_{i=1}^{m} \ell_i\right). \tag{18}$$

[6] 注意，$o_i[k]$指向向量o_i中索引k处的值。

[7] 这对应于x_{i+1}上的狄拉克δ分布与预测的softmax(o_i)分布之间的交叉熵。

4.2 The SGD Optimizer

Now that we have a loss function, we will begin our exploration of optimizers. The simplest gradient-based optimizer is Stochastic Gradient Descent (SGD). We start with randomly initialized parameters θ_0. Then for each step $t = 0, \ldots, T-1$, we perform the following update:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \tag{19}$$

where B_t is a random batch of data sampled from the dataset D, and the *learning rate* α_t and *batch size* $|B_t|$ are hyperparameters.

4.2.1 Implementing SGD in PyTorch

To implement our optimizers, we will subclass the PyTorch `torch.optim.Optimizer` class. An `Optimizer` subclass must implement two methods:

`def __init__(self, params, ...)` should initialize your optimizer. Here, `params` will be a collection of parameters to be optimized (or parameter groups, in case the user wants to use different hyperparameters, such as learning rates, for different parts of the model). Make sure to pass `params` to the `__init__` method of the base class, which will store these parameters for use in `step`. You can take additional arguments depending on the optimizer (e.g., the learning rate is a common one), and pass them to the base class constructor as a dictionary, where keys are the names (strings) you choose for these parameters.

`def step(self)` should make one update of the parameters. During the training loop, this will be called after the backward pass, so you have access to the gradients on the last batch. This method should iterate through each parameter tensor `p` and modify them *in place*, i.e. setting `p.data`, which holds the tensor associated with that parameter based on the gradient `p.grad` (if it exists), the tensor representing the gradient of the loss with respect to that parameter.

The PyTorch optimizer API has a few subtleties, so it's easier to explain it with an example. To make our example richer, we'll implement a slight variation of SGD where the learning rate decays over training, starting with an initial learning rate α and taking successively smaller steps over time:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \tag{20}$$

Let's see how this version of SGD would be implemented as a PyTorch `Optimizer`:

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"]  # Get the learning rate.
```

4.2 SGD优化器

现在我们有了一个损失函数，我们将开始探索优化器。最简单的基于梯度的优化器是随机梯度下降（SGD）。我们从随机初始化的参数θ_0开始。然后对于每个步骤t=0，... ，T-1，我们执行以下更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \tag{19}$$

其中B_t是从数据集D中随机抽取的数据批次，*学习率* α_t和*批次大小*$|B_t|$是超参数。

4.2.1 在PyTorch中实现SGD

为实现我们的优化器，我们将对PyTorch的torch optimizer Optimizer类进行子类化。优化器子类必须实现以下两个方法：

def__init__(self，params，...) 方法应初始化优化器。此处的params将包含待优化的参数集合（若用户需要为模型不同部分使用不同超参数，例如学习率，可采用参数组）。请务必将params传递给基类的__ init__ 方法，该方法会存储这些参数以供后续步骤使用。根据优化器类型可添加其他参数（例如学习率是常见参数），并将这些参数作为字典形式传递给基类构造函数，其中键值对应参数名称（字符串）。

def step (self) 方法需要对参数进行一次更新。在训练循环中，该方法会在反向传播后被调用，因此可以获取上一批次的梯度信息。该方法需要遍历每个参数张量p，并 *在原地* 修改参数，即通过设置p.data来实现——该张量根据梯度p.grad（如果存在）存储与该参数相关的张量，也就是表示损失函数对该参数梯度的张量。

PyTorch优化器API有一些微妙之处，所以用一个例子来解释它更容易。为了使我们的例子更丰富，我们将实现一个SGD的轻微变体，其中学习率在训练过程中衰减，从初始学习率α开始，并随着时间的推移采取越来越小的步骤:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \tag{20}$$

我们来看看这个版本的SGD如何作为PyTorch优化器实现：

```
从集合 . abc 导入 可调用的、可迭代的 从类型导入 可选
进口火炬
输入数学

class SGD（torch . optim . Optimizer）：
    def __init__ (self ， params， lr =1e-3 ):
        如果 lr < 0 :
            提高 ValueError ( f “学习率无效: { lr } ”  )
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step (self ， closure: Optional[Callable] = None ):
        loss = 无，若 关闭 为无，否则 关闭()
        用于 组 在 自身中 param_groups:
            lr = group[ “lr” ] # 获取学习率。
```

```
        for p in group["params"]:
            if p.grad is None:
                continue

            state = self.state[p]  # Get state associated with p.
            t = state.get("t", 0)  # Get iteration number from the state, or initial value.
            grad = p.grad.data  # Get the gradient of loss with respect to p.
            p.data -= lr / math.sqrt(t + 1) * grad  # Update weight tensor in-place.
            state["t"] = t + 1  # Increment iteration number.

    return loss
```

In `__init__`, we pass the parameters to the optimizer, as well as default hyperparameters, to the base class constructor (the parameters might come in groups, each with different hyperparameters). In case the parameters are just a single collection of `torch.nn.Parameter` objects, the base constructor will create a single group and assign it the default hyperparameters. Then, in `step`, we iterate over each parameter group, then over each parameter in that group, and apply Eq 20. Here, we keep the iteration number as a state associated with each parameter: we first read this value, use it in the gradient update, and then update it. The API specifies that the user might pass in a callable `closure` to re-compute the loss before the optimizer step. We won't need this for the optimizers we'll use, but we add it to comply with the API.

To see this working, we can use the following minimal example of a *training loop*:

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad()  # Reset the gradients for all learnable parameters.
    loss = (weights**2).mean() # Compute a scalar loss value.
    print(loss.cpu().item())
    loss.backward() # Run backward pass, which computes gradients.
    opt.step() # Run optimizer step.
```

This is the typical structure of a training loop: in each iteration, we will compute the loss and run a step of the optimizer. When training language models, our learnable parameters will come from the model (in PyTorch, `m.parameters()` gives us this collection). The loss will be computed over a sampled batch of data, but the basic structure of the training loop will be the same.

Problem (`learning_rate_tuning`): Tuning the learning rate (1 point)

As we will see, one of the hyperparameters that affects training the most is the learning rate. Let's see that in practice in our toy example. Run the SGD example above with three other values for the learning rate: 1e1, 1e2, and 1e3, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge (i.e., increase over the course of training)?

Deliverable: A one-two sentence response with the behaviors you observed.

4.3 AdamW

Modern language models are typically trained with more sophisticated optimizers, instead of SGD. Most optimizers used recently are derivatives of the Adam optimizer [Kingma and Ba, 2015]. We will use AdamW [Loshchilov and Hutter, 2019], which is in wide use in recent work. AdamW proposes a modification to Adam that improves regularization by adding *weight decay* (at each iteration, we pull the parameters towards 0),

```
用于 p 在 group[ "params" ]中：
    如果p.grad 为 None :
        持续

    state = self . state[p] # 获取与p关联的状态。
    t = state .get ( "t" , 0) # 从状态中获取迭代次数，或初始值。
    grad = p.grad.data # 获取损失函数相对于p 的梯度。
    p.data - = lr / math . sqrt (t + 1) * grad # 在原地更新权重张量。
    state[ "t" ] = t + 1 # 增加迭代次数。

损失
```

在初始化阶段，我们将参数及默认超参数传递给优化器，并将其传入基类构造函数（这些参数可能以组形式存在，每组包含不同的超参数）。若参数仅为torch.nn.Parameter对象的单一集合，基类构造函数会创建一个独立组并赋予默认超参数。随后在步骤中，我们遍历每个参数组，再逐个遍历组内参数，并应用公式20。此处我们将迭代次数作为与每个参数关联的状态变量：首先读取该数值，将其用于梯度更新，最后更新该数值。

该API规定用户可在优化器步骤前传递可调用闭包以重新计算损失。虽然我们使用的优化器无需此功能，但为符合API规范，我们仍添加了该特性。

要看到这个工作，我们可以使用以下最小的*训练循环*示例:

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

对于 t 在 范围 (100) 内：
    opt零梯度() # 重置所有可学习参数的梯度。
    loss = (weights**2) .mean() # 计算标量损失值。
    print(loss.cpu().item())
    loss .backward() # 运行反向传播，计算梯度。
    opt. step() # 运行优化器步骤。
```

训练循环的典型结构如下：每次迭代时，我们都会计算损失值并执行优化器的一步操作。在训练语言模型时，可学习参数将直接来自模型（在PyTorch中，通过m.parameters()函数获取该参数集合）。虽然损失值的计算基于采样数据批次，但训练循环的基本架构保持一致。

问题（学习率调整）：调整学习率（1分）

正如我们将要看到的，影响训练效果最显著的超参数之一是学习率。让我们通过一个简单的示例来具体分析。在仅进行10次训练迭代的情况下，使用上述SGD示例中的三个不同学习率值：1e1、1e2和1e3。这些学习率对应的损失函数会呈现怎样的变化趋势？是会更快衰减、更缓慢下降，还是在训练过程中逐渐增大（即发散）？

可交付成果：用一两句话描述你观察到的行为。

4.3 亚当·W

现代语言模型通常使用更复杂的优化器进行训练，而非SGD。近期使用的大多数优化器都是Adam优化器的衍生版本[Kingma和Ba， 2015]。我们将采用AdamW[Loshchilov和Hutter， 2019]，该方法在近期研究中被广泛使用。AdamW对Adam进行了改进，通过添加*权重衰减*（在每次迭代中，我们将参数拉向0）来增强正则化效果，

in a way that is decoupled from the gradient update. We will implement AdamW as described in algorithm 2 of Loshchilov and Hutter [2019].

AdamW is *stateful*: for each parameter, it keeps track of a running estimate of its first and second moments. Thus, AdamW uses additional memory in exchange for improved stability and convergence. Besides the learning rate α, AdamW has a pair of hyperparameters (β_1, β_2) that control the updates to the moment estimates, and a weight decay rate λ. Typical applications set (β_1, β_2) to $(0.9, 0.999)$, but large language models like LLaMA [Touvron et al., 2023] and GPT-3 [Brown et al., 2020] are often trained with $(0.9, 0.95)$. The algorithm can be written as follows, where ϵ is a small value (e.g., 10^{-8}) used to improve numerical stability in case we get extremely small values in v:

Algorithm 1 AdamW Optimizer

init(θ) (Initialize learnable parameters)
$m \leftarrow 0$ (Initial value of the first moment vector; same shape as θ)
$v \leftarrow 0$ (Initial value of the second moment vector; same shape as θ)
for $t = 1, \ldots, T$ **do**
  Sample batch of data B_t
  $g \leftarrow \nabla_\theta \ell(\theta; B_t)$ (Compute the gradient of the loss at the current time step)
  $m \leftarrow \beta_1 m + (1 - \beta_1) g$ (Update the first moment estimate)
  $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$ (Update the second moment estimate)
  $\alpha_t \leftarrow \alpha \frac{\sqrt{1-(\beta_2)^t}}{1-(\beta_1)^t}$ (Compute adjusted α for iteration t)
  $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v}+\epsilon}$ (Update the parameters)
  $\theta \leftarrow \theta - \alpha \lambda \theta$ (Apply weight decay)
end for

Note that t starts at 1. You will now implement this optimizer.

Problem (`adamw`): Implement AdamW (2 points)

Deliverable: Implement the AdamW optimizer as a subclass of `torch.optim.Optimizer`. Your class should take the learning rate α in `__init__`, as well as the β, ϵ and λ hyperparameters. To help you keep state, the base `Optimizer` class gives you a dictionary `self.state`, which maps `nn.Parameter` objects to a dictionary that stores any information you need for that parameter (for AdamW, this would be the moment estimates). Implement `[adapters.get_adamw_cls]` and make sure it passes `uv run pytest -k test_adamw`.

Problem (`adamwAccounting`): Resource accounting for training with AdamW (2 points)

Let us compute how much memory and compute running AdamW requires. Assume we are using float32 for every tensor.

(a) How much peak memory does running AdamW require? Decompose your answer based on the memory usage of the parameters, activations, gradients, and optimizer state. Express your answer in terms of the `batch_size` and the model hyperparameters (`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`). Assume `d_ff` $= 4 \times$ `d_model`.

For simplicity, when calculating memory usage of activations, consider only the following components:

- Transformer block
  - RMSNorm(s)

以一种与梯度更新解耦的方式。我们将按照Loshchilov和Hutter [2019]算法2中所述的方式实现AdamW。

AdamW是*有状态*的：对于每个参数，它会跟踪其一阶和二阶矩的实时估计值。因此，AdamW通过使用额外内存来换取更高的稳定性和收敛性。除了学习率α外，AdamW还有一对控制矩估计更新的超参数（β_1，β_2），以及一个权重衰减率λ。典型应用设置（β_1，β_2）为（0.9，0.999），但像LLaMA[Touvron等人，2023]和GPT-3[Brown等人，2020]这样的大型语言模型通常使用（0.9，0.95）进行训练。该算法可表示为：其中ϵ是一个小值（例如10^{-8}），用于在v出现极小值时提升数值稳定性：

算法1 AdamW优化器

init (θ)（初始化可学习参数）
$m \leftarrow 0$（第一矩向量的初始值；形状与θ 相同）
$v \leftarrow 0$（二阶矩向量的初始值；形状与θ相同）**对于** $t = 1, \ldots, T$ **do**
数据样本批次B_t
$g \leftarrow \nabla_\theta \ell(\theta; B_t)$（计算当前时间步损失的梯度）$m \leftarrow \beta_1 m + (1-\beta_1) g$（更新一阶矩估计）
$v \leftarrow \beta_2 v + (1-\beta_2) g^2$（更新二阶矩估计）
$\alpha_t \leftarrow \alpha \frac{\sqrt{1-(\beta_2)^t}}{1-(\beta_1)^t}$ (计算迭代t的调整α)
$\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v}+\epsilon}$ （更新参数）
$\theta \leftarrow \theta - \alpha\lambda\theta$（应用权重衰减）
结束

注意t从1开始。现在您将实现这个优化器。

问题（AdamW）：实现AdamW（2分）

可交付成果：将AdamW优化器实现为torch优化器的子类。该类应在init方法中接收学习率α，以及β、ϵ和λ等超参数。为便于维护状态，基础优化器类提供了一个self.state字典，该字典将nn.参数对象映射到存储该参数所需信息（对于AdamW而言，即为时刻估计值）的字典中。实现[adapters.get_adamw_cls]并确保其通过uv run pytest -k test_adamw。

问题（adamwAccounting）：AdamW培训资源核算（2分）

我们来计算运行AdamW算法所需的内存和计算资源。假设每个张量都使用浮点32位精度（float32）。

(a) 运行AdamW需要多少峰值内存？请根据参数、激活值、梯度和优化器状态的内存使用情况分解答案。用batch_size和模型超参数（vocab_size、context_length、num_layers、d_model、num_heads）来表达答案。假设d_ff = 4 × d_model。

为简化计算，激活值的内存使用量仅需考虑以下组成部分：

- 变压器块
  - RMSNorm(s)

- Multi-head self-attention sublayer: QKV projections, $Q^\top K$ matrix multiply, softmax, weighted sum of values, output projection.
- Position-wise feed-forward: W_1 matrix multiply, SiLU, W_2 matrix multiply

- final RMSNorm
- output embedding
- cross-entropy on logits

Deliverable: An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

(b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on the `batch_size`. What is the maximum batch size you can use and still fit within 80GB memory?

Deliverable: An expression that looks like $a \cdot \texttt{batch_size} + b$ for numerical values a, b, and a number representing the maximum batch size.

(c) How many FLOPs does running one step of AdamW take?

Deliverable: An algebraic expression, with a brief justification.

(d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput (tokens per second) relative to the hardware's theoretical peak FLOP throughput [Chowdhery et al., 2022]. An NVIDIA A100 GPU has a theoretical peak of 19.5 teraFLOP/s for float32 operations. Assuming you are able to get 50% MFU, how long would it take to train a GPT-2 XL for 400K steps and a batch size of 1024 on a single A100? Following Kaplan et al. [2020] and Hoffmann et al. [2022], assume that the backward pass has twice the FLOPs of the forward pass.

Deliverable: The number of days training would take, with a brief justification.

4.4 Learning rate scheduling

The value for the learning rate that leads to the quickest decrease in loss often varies during training. In training Transformers, it is typical to use a learning rate *schedule*, where we start with a bigger learning rate, making quicker updates in the beginning, and slowly decay it to a smaller value as the model trains[8] In this assignment, we will implement the cosine annealing schedule used to train LLaMA [Touvron et al., 2023].

A scheduler is simply a function that takes the current step t and other relevant parameters (such as the initial and final learning rates), and returns the learning rate to use for the gradient update at step t. The simplest schedule is the constant function, which will return the same learning rate given any t.

The cosine annealing learning rate schedule takes (i) the current iteration t, (ii) the maximum learning rate $\alpha_{\max}$, (iii) the minimum (final) learning rate $\alpha_{\min}$, (iv) the number of *warm-up* iterations T_w, and (v) the number of cosine annealing iterations T_c. The learning rate at iteration t is defined as:

(Warm-up) If $t < T_w$, then $\alpha_t = \frac{t}{T_w}\alpha_{\max}$.

(Cosine annealing) If $T_w \le t \le T_c$, then $\alpha_t = \alpha_{\min} + \frac{1}{2}\left(1 + \cos\left(\frac{t-T_w}{T_c-T_w}\pi\right)\right)(\alpha_{\max} - \alpha_{\min})$.

(Post-annealing) If $t > T_c$, then $\alpha_t = \alpha_{\min}$.

[8] It's sometimes common to use a schedule where the learning rate rises back up (restarts) to help get past local minima.

— 多头自注意力子层：QKV 投影，$Q^\top K$ 矩阵乘法，softmax，加权求和，输出投影。

— 位置前馈：W_1 矩阵乘法，SiLU，W_2 矩阵乘法

- 最终RMS归一化
- 输出嵌入
- 对数交叉熵

可交付成果：各参数、激活值、梯度及优化器状态的代数表达式，以及总和。

(b) 为GPT-2 XL模型实例化你的答案，得到仅依赖batch_size的表达式。在80GB内存限制下，你能使用的最大batch size是多少？

可交付成果：一个表达式，其形式为 $a \cdot \text{batch_size} + b$，其中 a，b 为数值，a 代表最大批量大小。

(c) 运行AdamW的一个步骤需要多少浮点运算次数（FLOPs）？

交付物：代数表达式，附简要说明。

(d) 浮点运算模型利用率（MFU）定义为观测吞吐量（每秒令牌数）与硬件理论峰值浮点运算吞吐量的比值[Chowdhery等人，2022]。 NVIDIA A100 GPU的理论峰值浮点运算吞吐量为19.5万亿次浮点运算/秒（浮点32位运算）。假设你能获得50%的 MFU，那么在单个A100上训练一个GPT-2 XL模型40万步、批量大小为1024需要多长时间？参考Kaplan等人[2020] 以及Hoffmann等人[2022]假设反向传播的浮点运算次数是前向传播的两倍。

可交付成果：培训所需天数，并附简要说明。

4.4 学习速率调度

导致损失最快下降的学习率值通常在训练过程中会变化。在训练Transformer时，通常采用学习率*调度*，即从较大的学习率开始，初期进行快速更新，随着模型训练逐渐衰减至较小值[8]。在本任务中，我们将实现用于训练LLaMA的余弦退火调度[Touvron等人，2023]。

调度器就是个函数，它接收当前步骤t和其他相关参数（比如初始和最终学习率），然后返回步骤t的梯度更新学习率。最简单的调度器是常数函数，它会返回相同的学习率，无论t是什么。

余弦退火学习率调度采用(i)当前迭代t，（ii）最大学习率$\alpha_{\max}$，（iii）最小（最终）学习率$\alpha_{\min}$，（iv）*热身*迭代次数T_w，以及(v)余弦退火迭代次数T_c。迭代t的学习率定义为：

（预热）如果 $t < T_w$，则 $\alpha_t = \frac{t}{T_w}\alpha_{\max}$.

（余弦退火）若 Tw ≤ t ≤ Tc，则 αt = αmin + $\frac{1}{2}\left(1+\cos\left(\frac{t-T_w}{T_c-T_w}\pi\right)\right)(\alpha_{\max}-\alpha_{\min})$.

（退火后）如果 $t > T_c$，则 $\alpha_t = \alpha_{\min}$。

[8]在优化过程中，采用学习率逐步回升（重启）的调度策略来突破局部极小值，这种做法在实际应用中较为常见。

Problem (`learning_rate_schedule`): Implement cosine learning rate schedule with warmup

Write a function that takes t, $\alpha_{\max}$, $\alpha_{\min}$, T_w and T_c, and returns the learning rate α_t according to the scheduler defined above. Then implement `[adapters.get_lr_cosine_schedule]` and make sure it passes `uv run pytest -k test_get_lr_cosine_schedule`.

4.5 Gradient clipping

During training, we can sometimes hit training examples that yield large gradients, which can destabilize training. To mitigate this, one technique often employed in practice is *gradient clipping*. The idea is to enforce a limit on the norm of the gradient after each backward pass before taking an optimizer step.

Given the gradient (for all parameters) g, we compute its ℓ_2-norm $\|g\|_2$. If this norm is less than a maximum value M, then we leave g as is; otherwise, we scale g down by a factor of $\frac{M}{\|g\|_2+\epsilon}$ (where a small ϵ, like 10^{-6}, is added for numeric stability). Note that the resulting norm will be just under M.

Problem (`gradient_clipping`): Implement gradient clipping (1 point)

Write a function that implements gradient clipping. Your function should take a list of parameters and a maximum ℓ_2-norm. It should modify each parameter gradient in place. Use $\epsilon = 10^{-6}$ (the PyTorch default). Then, implement the adapter `[adapters.run_gradient_clipping]` and make sure it passes `uv run pytest -k test_gradient_clipping`.

问题（learning_rate_schedule）：实现带有预热阶段的余弦学习率调度

编写一个函数，接收t，$\alpha_{\max}$，$\alpha_{\min}$，T_w和T_c，并根据上述定义的调度器返回学习率α_t。然后实现[adapters.get_lr_cosine_schedule]，并确保其通过uv run pytest -k test_get_lr_cosine_schedule。

4.5 梯度截断

在训练过程中，我们有时会遇到产生大梯度的训练样本，这可能会导致训练不稳定。为了缓解这种情况，实践中常用的一种技术是*梯度剪切*。其原理是在每次反向传播后、优化器执行步骤之前，对梯度的范数施加限制。

给定梯度（所有参数）g，我们计算其ℓ_2-范数$\|g\|_2$。若该范数小于最大值M，则保持g不变；否则，将g以一个因子进行缩放$\frac{M}{\|g\|_2+\epsilon}$（此处添加一个小$\epsilon$，例如$10^{-6}$，以确保数值稳定性）。请注意，最终的范数将略小于$M$。

问题（梯度剪切）：实现梯度剪切（1分）

编写一个实现梯度剪切的函数。该函数应接受一个参数列表和一个最大ℓ_2范数。它应在原地修改每个参数的梯度。使用$\epsilon=10^{-6}$（PyTorch默认值）。然后，实现适配器[adapters.run_gradient_clipping]并确保其通过uv run pytest -k test_gradient_clipping。

5 Training loop

We will now finally put together the major components we've built so far: the tokenized data, the model, and the optimizer.

5.1 Data Loader

The tokenized data (e.g., that you prepared in `tokenizer_experiments`) is a single sequence of tokens $x = (x_1, \ldots, x_n)$. Even though the source data might consist of separate documents (e.g., different web pages, or source code files), a common practice is to concatenate all of those into a single sequence of tokens, adding a delimiter between them (such as the `<|endoftext|>` token).

A *data loader* turns this into a stream of *batches*, where each batch consists of B sequences of length m, paired with the corresponding next tokens, also with length m. For example, for $B = 1, m = 3$, $([x_2, x_3, x_4], [x_3, x_4, x_5])$ would be one potential batch.

Loading data in this way simplifies training for a number of reasons. First, any $1 \leq i < n - m$ gives a valid training sequence, so sampling sequences are trivial. Since all training sequences have the same length, there's no need to pad input sequences, which improves hardware utilization (also by increasing batch size B). Finally, we also don't need to fully load the full dataset to sample training data, making it easy to handle large datasets that might not otherwise fit in memory.

Problem (`data_loading`): Implement data loading (2 points)

Deliverable: Write a function that takes a numpy array x (integer array with token IDs), a `batch_size`, a `context_length` and a PyTorch device string (e.g., `'cpu'` or `'cuda:0'`), and returns a pair of tensors: the sampled input sequences and the corresponding next-token targets. Both tensors should have shape (`batch_size`, `context_length`) containing token IDs, and both should be placed on the requested device. To test your implementation against our provided tests, you will first need to implement the test adapter at `[adapters.run_get_batch]`. Then, run `uv run pytest -k test_get_batch` to test your implementation.

Low-Resource/Downscaling Tip: Data loading on CPU or Apple Silicon

If you are planning to train your LM on CPU or Apple Silicon, you need to move your data to the correct device (and similarly, you should use the same device for your model later on).

If you are on CPU, you can use the `'cpu'` device string, and on Apple Silicon (M* chips), you can use the `'mps'` device string.

For more on MPS, checkout these resources:

- https://developer.apple.com/metal/pytorch/
- https://pytorch.org/docs/main/notes/mps.html

What if the dataset is too big to load into memory? We can use a Unix systemcall named `mmap` which maps a file on disk to virtual memory, and lazily loads the file contents when that memory location is accessed. Thus, you can "pretend" you have the entire dataset in memory. Numpy implements this through `np.memmap` (or the flag `mmap_mode='r'` to `np.load`, if you originally saved the array with `np.save`), which will return a numpy array-like object that loads the entries on-demand as you access them. **When sampling from your dataset (i.e., a numpy array) during training, be sure load the dataset in memory-mapped mode** (via `np.memmap` or the flag `mmap_mode='r'` to `np.load`, depending on how you saved the array). Make sure you also specify a `dtype` that matches the array that you're loading. It may be helpful to explicitly verify that the memory-mapped data looks correct (e.g., doesn't contain values beyond the expected vocabulary size).

5 训练循环

现在我们将最终整合迄今为止构建的主要组件：标记化数据、模型和优化器。

5.1 数据加载程序

分词后的数据（例如在tokenizer_experiments中准备的数据）是一个单一的分词序列$x = (x_1, \ldots, x_n)$。尽管源数据可能由独立的文档组成（例如不同的网页或源代码文件），但常见的做法是将所有这些文档连接成一个单一的分词序列，并在它们之间添加分隔符（例如（内文）分词符）。

一个*数据加载器*将此转换为*批次*的流，其中每个批次由长度为m的B个序列组成，并与长度同样为m的相应下一个标记配对。例如，当B=1，m=3时，$([x_2, x_3, x_4], [x_3, x_4, x_5])$将构成一个潜在批次。

采用这种数据加载方式能简化训练流程，主要基于以下优势：首先，任意$1 \leq i < n-m$都能生成有效训练序列，因此采样序列的生成变得简单。由于所有训练序列长度相同，无需对输入序列进行填充处理，这能提升硬件利用率（同时通过增加批量处理规模B实现）。最后，我们无需完整加载整个数据集即可进行训练数据采样，这使得处理那些原本可能超出内存容量的大型数据集变得轻而易举。

问题（data_loading）：实现数据加载（2分）

交付成果：编写一个函数，该函数接收一个numpy数组x（包含标记ID的整数数组）、batch_size、context_length以及PyTorch设备字符串（例如‘cpu’或‘cuda:0’），并返回两个张量对：采样输入序列和对应的下一个标记目标。两个张量的形状应为(batch_size，context_length)且包含标记ID，且需部署在指定设备上。为通过我们提供的测试验证实现效果，您需先在[adapters.run_get_batch]中实现测试适配器，随后运行uv run pytest -k test_get_batch以测试您的实现。

低资源/降级提示：数据加载于CPU或Apple Silicon

若计划在CPU或苹果硅芯片上训练语言模型（LM），需将数据迁移至目标设备（后续模型训练亦应使用同一设备）。

若使用CPU架构，可采用‘cpu’设备字符串；若采用苹果硅（M*芯片）架构，则可使用‘mps’设备字符串。

如需了解更多关于MPS的信息，请查阅以下资源：

- https://developer.apple.com/metal/pytorch/
- https://pytorch.org/docs/main/notes/mps.html

如果数据集太大而无法全部加载到内存中怎么办？我们可以使用名为mmap的Unix系统调用，它将磁盘文件映射到虚拟内存，并在访问该内存位置时惰性加载文件内容。这样一来，你就能“假装”整个数据集都已加载到内存中。Numpy通过np.memmap实现这一功能（若最初使用np.save保存数组，则需在np.load中设置mmap_mode=‘r’），该方法会返回一个类似numpy数组的对象，按需加载条目。**在训练过程中从数据集（即numpy数组）中采样时，请务必以内存映射模式加载数据集**（根据数组保存方式，使用np.memmap或np.load的mmap_mode=‘r’）。同时确保指定的数据类型与要加载的数组相匹配。建议显式验证内存映射数据是否正确（例如，是否包含超出预期词汇表大小的值）。

5.2 Checkpointing

In addition to loading data, we will also need to save models as we train. When running jobs, we often want to be able to resume a training run that for some reason stopped midway (e.g., due to your job timing out, machine failure, etc). Even when all goes well, we might also want to later have access to intermediate models (e.g., to study training dynamics post-hoc, take samples from models at different stages of training, etc).

A checkpoint should have all the states that we need to resume training. We of course want to be able to restore model weights at a minimum. If using a stateful optimizer (such as AdamW), we will also need to save the optimizer's state (e.g., in the case of AdamW, the moment estimates). Finally, to resume the learning rate schedule, we will need to know the iteration number we stopped at. PyTorch makes it easy to save all of these: every `nn.Module` has a `state_dict()` method that returns a dictionary with all learnable weights; we can restore these weights later with the sister method `load_state_dict()`. The same goes for any `nn.optim.Optimizer`. Finally, `torch.save(obj, dest)` can dump an object (e.g., a dictionary containing tensors in some values, but also regular Python objects like integers) to a file (path) or file-like object, which can then be loaded back into memory with `torch.load(src)`.

Problem (`checkpointing`): Implement model checkpointing (1 point)

Implement the following two functions to load and save checkpoints:

`def save_checkpoint(model, optimizer, iteration, out)` should dump all the state from the first three parameters into the file-like object `out`. You can use the `state_dict` method of both the model and the optimizer to get their relevant states and use `torch.save(obj, out)` to dump `obj` into out (PyTorch supports either a path or a file-like object here). A typical choice is to have `obj` be a dictionary, but you can use whatever format you want as long as you can load your checkpoint later.

This function expects the following parameters:

```
model: torch.nn.Module
optimizer: torch.optim.Optimizer
iteration: int
out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
```

`def load_checkpoint(src, model, optimizer)` should load a checkpoint from `src` (path or file-like object), and then recover the model and optimizer states from that checkpoint. Your function should return the iteration number that was saved to the checkpoint. You can use `torch.load(src)` to recover what you saved in your `save_checkpoint` implementation, and the `load_state_dict` method in both the model and optimizers to return them to their previous states.

This function expects the following parameters:

```
src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
model: torch.nn.Module
optimizer: torch.optim.Optimizer
```

Implement the `[adapters.run_save_checkpoint]` and `[adapters.run_load_checkpoint]` adapters, and make sure they pass `uv run pytest -k test_checkpointing`.

5.2 检查点

除数据加载外，训练过程中还需保存模型。运行作业时，我们通常希望能够恢复因故中断的训练流程（例如作业超时、机器故障等）。即使一切顺利，我们也可能需要后续访问中间模型（例如事后研究训练动态、从不同训练阶段的模型中抽取样本等）。

检查点应当包含所有用于恢复训练的状态信息。我们当然需要能够至少恢复模型权重。如果使用有状态优化器（如AdamW），还需要保存优化器的状态（例如AdamW的矩估计）。最后，要恢复学习率调度，我们需要知道停止时的迭代次数。PyTorch提供了便捷的保存方式：每个nn模块都有state_dict()方法，可返回包含所有可学习权重的字典；后续可通过同名方法load_state_dict()恢复这些权重。其他nn优化器也遵循相同规则。最后，torch.save(object，dest)可将对象（例如包含张量值的字典，或整数等常规Python对象）保存至文件（路径）或类文件对象，之后可通过torch.load(src)重新加载回内存。

问题（检查点机制）：实施模型检查点机制（1分）

实现以下两个功能以加载和保存检查点：

def saveCheckpoint (model，optimizer，iteration，out)函数需要将模型、优化器和迭代过程的全部状态信息写入类文件对象out。具体实现时，可调用模型和优化器的state_dict方法获取对应状态，再通过torch.save(obj，out)将状态对象写入out（PyTorch支持路径或类文件对象）。虽然通常建议将状态对象设为字典形式，但只要后续能加载检查点数据，任何格式均可使用。

该函数需接受以下参数：

`model: torch.nn.Module`

优化器: torch . optim . Optimizer

迭代: int

输出: 字符串 | 操作系统 . 路径类型 | 类型 . 二进制输入输出 | 类型 . 输入输出[字节]

def load_checkpoint (src，model，optimizer)函数应从src（路径或类文件对象）加载检查点，随后从该检查点恢复模型和优化器的状态。该函数需返回保存至检查点的迭代次数。可通过torch.load(src)调用保存Checkpoint实现中的数据，同时调用模型和优化器的load_state_dict方法将其恢复至先前状态。

该函数需接受以下参数：

src: str | 操作系统 . PathLike | 类型 . BinaryIO | 类型 . IO[字节]

`model: torch.nn.Module`

优化器: torch . optim . Optimizer

实现 [adapters.run_save_checkpoint] 和 [adapters.run_load_checkpoint] 适配器，并确保它们通过 uv run pytest -k testCheckpointing。

5.3 Training loop

Now, it's finally time to put all of the components you implemented together into your main training script. It will pay off to make it easy to start training runs with different hyperparameters (e.g., by taking them as command-line arguments), since you will be doing these many times later to study how different choices impact training.

Problem (`training_together`): Put it together (4 points)

Deliverable: Write a script that runs a training loop to train your model on user-provided input. In particular, we recommend that your training script allow for (at least) the following:

- Ability to configure and control the various model and optimizer hyperparameters.
- Memory-efficient loading of training and validation large datasets with `np.memmap`.
- Serializing checkpoints to a user-provided path.
- Periodically logging training and validation performance (e.g., to console and/or an external service like Weights and Biases).[a]

[a] wandb.ai

5.3 训练循环

现在，终于到了将你实现的所有组件整合到主训练脚本中的时刻。通过将超参数作为命令行参数输入，可以轻松启动不同训练配置的运行，这将非常实用，因为后续你将多次进行此类操作，以研究不同参数选择对训练效果的影响。

问题（联合训练）：整合（4分）

可交付成果：编写一个脚本，通过训练循环使用用户提供的输入数据训练模型。特别建议训练脚本至少包含以下功能：

- 配置和控制各类模型及优化器超参数的能力。
- 使用np.memmap高效加载训练与验证大型数据集
- 将检查点序列化至用户指定路径。
- 定期记录训练和验证性能（例如，向控制台和/或外部服务（如权重与偏置））。[a]

[a] wandb.ai

6 Generating text

Now that we can train models, the last piece we need is the ability to generate text from our model. Recall that a language model takes in a (possibly batched) integer sequence of length (`sequence_length`) and produces a matrix of size (`sequence_length` $\times$ `vocab size`), where each element of the sequence is a probability distribution predicting the next word after that position. We will now write a few functions to turn this into a sampling scheme for new sequences.

Softmax By standard convention, the language model output is the output of the final linear layer (the "logits") and so we have to turn this into a normalized probability via the *softmax* operation, which we saw earlier in Eq 10.

Decoding To generate text (decode) from our model, we will provide the model with a sequence of prefix tokens (the "prompt"), and ask it to produce a probability distribution over the vocabulary that predicts the next word in the sequence. Then, we will sample from this distribution over the vocabulary items to determine the next output token.

Concretely, one step of the decoding process should take in a sequence $x_{1\ldots t}$ and return a token x_{t+1} via the following equation,

$$P(x_{t+1} = i \mid x_{1\ldots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}$$
$$v = \text{TransformerLM}(x_{1\ldots t})_t \in \mathbb{R}^{\texttt{vocab_size}}$$

where TransformerLM is our model which takes as input a sequence of `sequence_length` and produces a matrix of size (`sequence_length` $\times$ `vocab_size`), and we take the last element of this matrix, as we are looking for the next word prediction at the t-th position.

This gives us a basic decoder by repeatedly sampling from these one-step conditionals (appending our previously-generated output token to the input of the next decoding timestep) until we generate the end-of-sequence token `<|endoftext|>` (or a user-specified maximum number of tokens to generate).

Decoder tricks We will be experimenting with small models, and small models can sometimes generate very low quality texts. Two simple decoder tricks can help fix these issues. First, in *temperature scaling* we modify our softmax with a temperature parameter τ, where the new softmax is

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{|\text{vocab_size}|} \exp(v_j/\tau)}. \tag{24}$$

Note how setting $\tau \to 0$ makes it so that the largest element of v dominates, and the output of the softmax becomes a one-hot vector concentrated at this maximal element.

Second, another trick is *nucleus* or *top-p* sampling, where we modify the sampling distribution by truncating low-probability words. Let q be a probability distribution that we get from a (temperature-scaled) softmax of size (`vocab_size`). Nucleus sampling with hyperparameter p produces the next token according to the equation

$$P(x_{t+1} = i|q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{if } i \in V(p) \\ 0 & \text{otherwise} \end{cases}$$

where $V(p)$ is the *smallest* set of indices such that $\sum_{j \in V(p)} q_j \geq p$. You can compute this quantity easily by first sorting the probability distribution q by magnitude, and selecting the largest vocabulary elements until you reach the target level of α.

6 正在生成文本

既然我们已经能够训练模型，接下来需要的关键能力就是让模型生成文本。需要说明的是，语言模型接收长度为sequence_length的整数序列（可能为批量输入），并输出一个大小为sequence_length ×词汇量的矩阵。该矩阵的每个元素都是对后续位置单词的预测概率分布。现在我们将编写几个函数，将这个概率分布转化为生成新序列的采样方案。

Softmax按照标准惯例，语言模型的输出是最终线性层的输出（“logits”），因此我们必须通过*softmax*操作将其转换为归一化概率，我们之前在公式10中已经看到过。

解码 为从我们的模型生成文本（解码），我们将向模型提供一系列前缀标记（即“提示”），并要求其生成一个预测序列中下一个单词的词汇概率分布。随后，我们将从该词汇分布中采样，以确定下一个输出标记。

具体来说，解码过程的一个步骤应接收序列$x_{1\ldots}$，并通过以下方程返回令牌x_{t+1}，

$$P(x_{t+1} = i \mid x_{1\ldots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}$$
$$v = \text{TransformerLM}(x_{1\ldots t})_t \in \mathbb{R}^{\texttt{vocab_size}}$$

其中TransformerLM是我们的模型，它以sequence_length的序列作为输入，并生成一个大小为(sequence_length × vocab_size)的矩阵，我们取该矩阵的最后一个元素，因为我们正在寻找第t个位置的下一个单词预测。

该方法通过反复采样这些单步条件变量（即将先前生成的输出标记附加至下一解码时间步的输入），逐步构建基础解码器，直至生成序列结束标记（endoftext）或用户指定的最大生成标记数。

解码器技巧 我们将尝试使用小型模型，而小型模型有时会产生质量非常低的文本。两个简单的解码器技巧可以帮助解决这些问题。首先，在*温度缩放*中，我们使用温度参数τ来修改我们的softmax，其中新的softmax是

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{|\text{vocab_size}|} \exp(v_j/\tau)}. \tag{24}$$

注意，将$\tau \to 0$设置为0时，v的最大元素会占据主导地位，而softmax的输出
形成一个以该最大元素为中心的独热向量。

其次，另一个技巧是*核抽样*或*顶抽样*，我们通过截断低概率词来修改采样分布。设q为从大小为vocab_size的（温度缩放）softmax中获得的概率分布。使用超参数p的核抽样根据以下方程生成下一个标记：

$$P(x_{t+1} = i|q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{if } i \in V(p) \\ 0 & \text{otherwise} \end{cases}$$

其中$V(p)$是满足$\sum_{j \in V(p)} q_j \geq p$的*最小*索引集合。你可以通过首先按概率分布$q$的大小进行排序，并选择最大的词汇元素直到达到目标水平α来轻松计算该量。

Problem (`decoding`): Decoding (3 points)

Deliverable: Implement a function to decode from your language model. We recommend that you support the following features:

- Generate completions for a user-provided prompt (i.e., take in some $x_{1\ldots t}$ and sample a completion until you hit an `<|endoftext|>` token).
- Allow the user to control the maximum number of generated tokens.
- Given a desired temperature value, apply softmax temperature scaling to the predicted next-word distributions before sampling.
- Top-p sampling (Holtzman et al., 2020; also referred to as nucleus sampling), given a user-specified threshold value.

问题（解码）：解码（3分）

可交付成果：实现从语言模型解码的功能。建议支持以下功能：

- 为用户提供的提示生成补全内容（即输入一些$x_{1\ldots}$并采样补全内容，直到遇到（内文）标记）。
- 允许用户控制生成令牌的最大数量。
- 给定目标温度值，在采样前对预测的下词分布应用softmax温度缩放。
- Top-p抽样（Holtzman等人，2020；亦称核抽样），给定用户指定的阈值。

7 Experiments

Now it is time to put everything together and train (small) language models on a pretaining dataset.

7.1 How to Run Experiments and Deliverables

The best way to understand the rationale behind the architectural components of a Transformer is to actually modify it and run it yourself. There is no substitute for hands-on experience.

To this end, it's important to be able to experiment **quickly, consistently, and keep records** of what you did. To experiment quickly, we will be running many experiments on a small scale model (17M parameters) and simple dataset (TinyStories). To do things consistently, you will ablate components and vary hyperparameters in a systematic way, and to keep records we will ask you to submit a log of your experiments and learning curves associated with each experiment.

To make it possible to submit loss curves, **make sure to periodically evaluate validation losses and record both the number of steps and wallclock times**. You might find logging infrastructure such as Weights and Biases helpful.

Problem (`experiment_log`): Experiment logging (3 points)

For your training and evaluation code, create experiment tracking infrastructure that allows you to track your experiments and loss curves with respect to gradient steps and wallclock time.

Deliverable: Logging infrastructure code for your experiments and an experiment log (a document of all the things you tried) for the assignment problems below in this section.

7.2 TinyStories

We are going to start with a very simple dataset (TinyStories; Eldan and Li, 2023) where models will train quickly, and we can see some interesting behaviors. The instructions for getting this dataset is at section 1. An example of what this dataset looks like is below.

Example (`tinystories_example`): One example from TinyStories

Once upon a time there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, "Wow, that is a really amazing vase! Can I buy it?" The shopkeeper smiled and said, "Of course you can. You can take it home and show all your friends how amazing it is!" So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn't believe how lucky Ben was. And that's how Ben found an amazing vase in the store!

Hyperparameter tuning We will tell you some very basic hyperparameters to start with and ask you to find some settings for others that work well.

`vocab_size` 10000. Typical vocabulary sizes are in the tens to hundreds of thousands. You should vary this and see how the vocabulary and model behavior changes.

`context_length` 256. Simple datasets such as TinyStories might not need long sequence lengths, but for the later OpenWebText data, you may want to vary this. Try varying this and seeing the impact on both the per-iteration runtime and the final perplexity.

7 实验

现在是时候将所有要素整合起来，并在预训练数据集上训练（小型）语言模型了。

7.1 如何进行实验和交付成果

要真正理解Transformer架构组件背后的原理，最好的方法就是亲自修改并运行它。毕竟，没有什么能替代亲身体验。

为此，我们需要具备 **快速实验能力、保持实验一致性并做好记录** 。为实现快速实验，我们将使用小型模型（1700万参数）和简单数据集（TinyStories）进行大量实验。为确保实验一致性，您需要系统性地移除组件并调整超参数。为做好记录，我们将要求您提交每次实验的日志及对应的学习曲线。

为便于提交损失曲线， **请定期评估验证损失，并记录步数和实际运行时间** 。您可能会发现使用权重和偏置等日志记录工具会很有帮助。

问题（实验日志）：实验日志记录（3分）

针对您的训练与评估代码，需构建实验追踪基础设施，以便根据梯度步长和实际运行时间追踪实验过程及损失曲线。

交付成果 ：为本节后续的作业问题，提交实验日志（记录所有尝试操作的文档）及实验基础设施代码。

7.2 小故事

我们将从一个非常简单的数据集(TinyStories； Eldan and Li ，2023)开始，模型将快速训练，我们可以看到一些有趣的行为。获取该数据集的说明见第1节。该数据集的示例如下所示。

示例（tinystories_example）：来自TinyStories的一个示例

从前有个叫本的小男孩，他总爱探索周遭世界。某天在商店里，他看到许多令人惊叹的展品，比如那些精美的花瓶。正当本在店里闲逛时，突然发现一个特别的花瓶。他惊呼道：“哇，这花瓶太棒了！能买下它吗？”店主微笑着回答：“当然可以，你可以带回家给朋友们看它有多神奇！”本把花瓶带回了家，心里美滋滋的。他召集朋友们来家里展示这件宝贝，大家都觉得这花瓶美得让人窒息，简直不敢相信本这么幸运。就这样，本在商店里找到了那个神奇的花瓶！

超参数调优 我们将首先介绍一些基础超参数，并指导您为其他参数找到合适的设置方案。

vocab_size 10000。典型词汇量范围在数万到数十万之间。建议调整该数值，观察词汇量与模型行为的变化。

context_length 256。像TinyStories这样的简单数据集可能不需要较长的序列长度，但对于后续的OpenWebText数据，你可能需要调整这个参数。尝试调整这个参数，观察其对每次迭代运行时间和最终困惑度的影响。

`d_model` 512. This is slightly smaller than the 768 dimensions used in many small Transformer papers, but this will make things faster.

`d_ff` 1344. This is roughly $\frac{8}{3}$`d_model` while being a multiple of 64, which is good for GPU performance.

`RoPE theta parameter` Θ 10000.

`number of layers and heads` 4 layers, 16 heads. Together, this will give about 17M non-embedding parameters which is a fairly small Transformer.

`total tokens processed` 327,680,000 (your batch size $\times$ total step count $\times$ context length should equal roughly this value).

You should do some trial and error to find good defaults for the following other hyperparameters: `learning rate`, `learning rate warmup`, `other AdamW hyperparameters` $(\beta_1, \beta_2, \epsilon)$, and `weight decay`. You can find some typical choices of such hyperparameters in Kingma and Ba [2015].

Putting it together Now you can put everything together by getting a trained BPE tokenizer, tokenizing the training dataset, and running this in the training loop that you wrote. **Important note:** If your implementation is correct and efficient, the above hyperparameters should result in a roughly 30-40 minute runtime on 1 H100 GPU. If you have runtimes that are much longer, please check and make sure your dataloading, checkpointing, or validation loss code is not bottlenecking your runtimes and that your implementation is properly batched.

Tips and tricks for debugging model architectures We highly recommend getting comfortable with your IDE's built-in debugger (e.g., VSCode/PyCharm), which will save you time compared to debugging with print statements. If you use a text editor, you can use something more like `pdb`. A few other good practices when debugging model architectures are:

- A common first step when developing any neural net architecture is to overfit to a single minibatch. If your implementation is correct, you should be able to quickly drive the training loss to near-zero.
- Set debug breakpoints in various model components, and inspect the shapes of intermediate tensors to make sure they match your expectations.
- Monitor the norms of activations, model weights, and gradients to make sure they are not exploding or vanishing.

Problem (`learning_rate`): Tune the learning rate (3 points) (4 H100 hrs)

The learning rate is one of the most important hyperparameters to tune. Taking the base model you've trained, answer the following questions:

(a) Perform a hyperparameter sweep over the learning rates and report the final losses (or note divergence if the optimizer diverges).

Deliverable: Learning curves associated with multiple learning rates. Explain your hyperparameter search strategy.

Deliverable: A model with validation loss (per-token) on TinyStories of at most 1.45

d_model 512。这个维度比许多小型Transformer论文使用的768略小，但能显著提升运算速度。

d_ff 1344。这大约是$\frac{8}{3}$d_model取64的倍数，有利于GPU性能。 **RoPE theta参数** Θ 10000。

层数与头数 为4层、16个头。合计约1700万非嵌入参数，这属于规模较小的Transformer模型。

处理的总标记数为327,680,000（您的批量大小×总步数×上下文长度应大致等于该值）。

你应该通过试错法找到以下其他超参数的良好默认值：学习率、学习率预热、其他AdamW超参数（β_1，β_2，ϵ）以及权重衰减。你可以在Kingma和Ba [2015] 中找到这些超参数的一些典型选择。

整合流程 现在您可以通过获取训练好的 BPE 分词器、对训练数据集进行分词处理，并在您编写的训练循环中运行这些步骤，将所有环节整合起来。**重要提示：** 若实现正确且高效，上述超参数配置应在1个H100 GPU上实现约30-40分钟的运行时。若实际运行时间显著延长，请检查数据加载、检查点保存或验证损失代码是否成为性能瓶颈，并确认批处理机制是否正确配置。

模型架构调试技巧与窍门 我们强烈建议您熟练掌握IDE内置调试器（如VSCode/PyCharm），相比使用打印语句调试，这能为您节省大量时间。若使用文本编辑器，推荐使用pdb等工具。调试模型架构时的其他实用技巧包括：

- 在开发任何神经网络架构时，常见的第一步是使用单个小批量数据进行过拟合。若实现正确，应能快速将训练损失驱动至接近零的水平。
- 在不同模型组件中设置调试断点，并检查中间张量的形状以确保其符合预期。
- 监测激活值、模型权重及梯度的规范性，确保其不会出现爆炸或消失现象。

问题（学习率）：调整学习率（3分）（4 H100小时）

学习率是需要调整的最重要超参数之一。基于您已训练的基础模型，请回答以下问题：

(a) 对学习率进行超参数扫描，并报告最终损失值（若优化器发散则记录发散情况）。

可交付成果：与多种学习率相关联的学习曲线。阐述您的超参数搜索策略。

可交付成果：一个在 TinyStories 上具有最多 1.45 个验证损失（按标记计）的模型

Low-Resource/Downscaling Tip: Train for few steps on CPU or Apple Silicon

If you are running on `cpu` or `mps`, you should instead reduce the total tokens processed count to $40,000,000$, which will be sufficient to produce reasonably fluent text. You may also increase the target validation loss from 1.45 to 2.00.

Running our solution code with a tuned learning rate on an M3 Max chip and 36 GB of RAM, we use batch size $\times$ total step count $\times$ context length $= 32 \times 5000 \times 256 = 40,960,000$ tokens, which takes 1 hour and 22 minutes on `cpu` and 36 minutes on `mps`. At step 5000, we achieve a validation loss of 1.80.

Some additional tips:

- When using X training steps, we suggest adjusting the cosine learning rate decay schedule to terminate its decay (i.e., reach the minimum learning rate) at precisely step X.
- When using `mps`, do **not** use TF32 kernels, i.e., do **not** set

  ```
  torch.set_float32_matmul_precision('high')
  ```

  as you might with `cuda` devices. We tried enabling TF32 kernels with `mps` (`torch` version 2.6.0) and found the backend will use silently broken kernels that cause unstable training.
- You can speed up training by JIT-compiling your model with `torch.compile`. Specifically:
  - On `cpu`, compile your model with

    ```
    model = torch.compile(model)
    ```
  - On `mps`, you can somewhat optimize the backward pass using

    ```
    model = torch.compile(model, backend="aot_eager")
    ```

    Compilation with Inductor is not supported on `mps` as of `torch` version 2.6.0.

(b) Folk wisdom is that the best learning rate is "at the edge of stability." Investigate how the point at which learning rates diverge is related to your best learning rate.

Deliverable: Learning curves of increasing learning rate which include at least one divergent run and an analysis of how this relates to convergence rates.

Now let's vary the batch size and see what happens to training. Batch sizes are important – they let us get higher efficiency from our GPUs by doing larger matrix multiplies, but is it true that we always want batch sizes to be large? Let's run some experiments to find out.

Problem (`batch_size_experiment`): Batch size variations (1 point) (2 H100 hrs)

Vary your batch size all the way from 1 to the GPU memory limit. Try at least a few batch sizes in between, including typical sizes like 64 and 128.

Deliverable: Learning curves for runs with different batch sizes. The learning rates should be optimized again if necessary.

Deliverable: A few sentences discussing of your findings on batch sizes and their impacts on training.

With your decoder in hand, we can now generate text! We will generate from the model and see how good it is. As a reference, you should get outputs that look at least as good as the example below.

低资源/降级提示：在CPU或Apple Silicon上进行少量步骤训练

如果你在cpu或mps上运行，你应该将处理的总标记数减少到40， 000， 000，这将足以产生相当流畅的文本。你也可以将目标验证损失从1.45增加到2.00。

在M3 Max芯片和36GB内存上使用调优的学习率运行我们的解决方案代码时，我们采用批量大小×总步数×上下文长度=32 × 5000 × 256=40，960，000个标记，这在CPU上需要1小时22分钟，在mps上需要36分钟。在第5000步时，我们实现了1.80的验证损失。

其他建议：

- 当使用X个训练步骤时，我们建议调整余弦学习率衰减计划，使其在第X步精确终止衰减（即达到最小学习率）。
- 使用mps时，**不要**使用TF32内核，即**不要**设置torch。 set_float32 _ matmul_precision（ 'high' ）

  如同处理CUDA设备时的操作方式，我们尝试在torch 2.6.0版本中启用带mp的TF32内核，结果发现后端会使用存在严重漏洞的内核，导致训练过程不稳定。
- 通过使用torch工具对模型进行即时编译（JIT编译），可显著提升训练速度。具体操作如下：
  - 在CPU上，使用以下命令编译您的模型：

    ```
    model = torch.compile(model)
    ```
  - 在mps中，你可以对反向传播进行一定程度的优化

    ```
    model = torch.compile(model, backend="aot_eager")
    ```

    自torch 2.6.0版本起，mps不再支持与Inductor的编译。

(b) 民间智慧认为最佳学习率应处于"稳定边缘"。探究学习率发散点与最佳学习率之间的关联。

可交付成果：包含至少一次发散运行的学习率递增学习曲线，以及该现象与收敛速率关系的分析。

现在我们来调整批量大小，看看训练效果会如何变化。批量大小至关重要——通过执行更大规模的矩阵乘法运算，能让我们从GPU中获得更高的效率。但是否真的需要批量大小始终保持较大？让我们通过实验来验证。

问题（batch_size_experiment）：批次大小变异（1个点）（2 H100小时）

将批量大小从1逐步调整至GPU内存上限。建议至少尝试几个中间值，包括64和128等常见大小。

可交付成果：不同批量大小运行的学习曲线。必要时应再次优化学习率。

交付成果：用几句话阐述您关于批量大小及其对训练影响的研究发现。

手握解码器后，我们就能生成文本了！我们将通过模型生成内容，看看效果如何。作为参考，输出结果至少要达到下例的水平。

Example (`ts_generate_example`): Sample output from a TinyStories language model

Once upon a time, there was a pretty girl named Lily. She loved to eat gum, especially the big black one. One day, Lily's mom asked her to help cook dinner. Lily was so excited! She loved to help her mom. Lily's mom made a big pot of soup for dinner. Lily was so happy and said, "Thank you, Mommy! I love you." She helped her mom pour the soup into a big bowl. After dinner, Lily's mom made some yummy soup. Lily loved it! She said, "Thank you, Mommy! This soup is so yummy!" Her mom smiled and said, "I'm glad you like it, Lily." They finished cooking and continued to cook together. The end.

Low-Resource/Downscaling Tip: Generate text on CPU or Apple Silicon

If instead you used the low-resource configuration with 40M tokens processed, you should see generations that still resemble English but are not as fluent as above. For example, our sample output from a TinyStories language model trained on 40M tokens is below:

Once upon a time, there was a little girl named Sue. Sue had a tooth that she loved very much. It was his best head. One day, Sue went for a walk and met a ladybug! They became good friends and played on the path together.

"Hey, Polly! Let's go out!" said Tim. Sue looked at the sky and saw that it was difficult to find a way to dance shining. She smiled and agreed to help the talking!"

As Sue watched the sky moved, what it was. She

Here is the precise problem statement and what we ask for:

Problem (`generate`): Generate text (1 point)

Using your decoder and your trained checkpoint, report the text generated by your model. You may need to manipulate decoder parameters (temperature, top-p, etc.) to get fluent outputs.

Deliverable: Text dump of at least 256 tokens of text (or until the first `<|endoftext|>` token), and a brief comment on the fluency of this output and at least two factors which affect how good or bad this output is.

7.3 Ablations and architecture modification

The best way to understand the Transformer is to actually modify it and see how it behaves. We will now do a few simple ablations and modifications.

Ablation 1: layer normalization It is often said that layer normalization is important for the stability of Transformer training. But perhaps we want to live dangerously. Let's remove RMSNorm from each of our Transformer blocks and see what happens.

Problem (`layer_norm_ablation`): Remove RMSNorm and train (1 point) (1 H100 hr)

Remove all of the RMSNorms from your Transformer and train. What happens at the previous optimal learning rate? Can you get stability by using a lower learning rate?

Deliverable: A learning curve for when you remove RMSNorms and train, as well as a learning curve for the best learning rate.

Deliverable: A few sentence commentary on the impact of RMSNorm.

示例（ts_generate_example）：TinyStories语言模型的示例输出

从前有个叫莉莉的漂亮姑娘，她最爱嚼口香糖，尤其是那种大大的黑色款。有天妈妈请她帮忙做饭，莉莉兴奋得不得了！她特别喜欢帮妈妈干活。妈妈煮了大锅汤，莉莉开心得直说："谢谢妈妈！我好爱你！"她帮妈妈把汤盛进大碗。饭后妈妈又做了碗香喷喷的汤，莉莉吃得津津有味："谢谢妈妈！这汤真香！"妈妈笑着回答："莉莉，你这么喜欢我就开心啦！"两人吃完饭继续一起做饭，故事就这样圆满收场了。

低资源/降级提示：在CPU或Apple Silicon上生成文本

若改用低资源配置（处理4000万个标记），生成的文本仍保留英语特征，但流畅度不及前文案例。例如，我们基于4000万标记训练的TinyStories语言模型生成的样本输出如下：

从前有个小女孩叫苏，她有一颗特别疼爱的牙齿，那可是她最宝贝的宝贝。有一天，苏出门散步，遇见了一只瓢虫！它们成了好朋友，一起在小路上玩耍。

"嘿，波莉！我们出去玩吧！"蒂姆说。苏抬头望向天空，发现很难找到跳舞的光亮。她微笑着答应帮忙，说："来吧，说话的波莉！"

苏看着天空移动，观察着它的变化。

以下是具体问题陈述及我们的要求：

问题（生成）：生成文本（1分）

使用您的解码器和训练检查点，报告模型生成的文本。可能需要调整解码器参数（温度、top-p等）以获得流畅的输出。

可交付成果：至少256个文本标记的文本转储（或直至第一个（endofctext）标记），以及对该输出流畅性的简要评述，以及至少两个影响该输出优劣的因素。

7.3 消融和结构改造

要真正理解Transformer，最佳方法是对其进行实际修改并观察其行为表现。接下来我们将进行若干简单的消融实验与参数调整。

消融实验1：层归一化 人们常说层归一化对Transformer模型训练的稳定性至关重要。但或许我们该冒险一试——从每个Transformer模块中移除RMSNorm，看看会发生什么。

问题（层归一化消融）：移除RMSNorm并进行训练（1分）（1 H100小时）

从Transformer模型中移除所有RMSNorm并进行训练。在之前的最优学习率下会发生什么？使用较低的学习率能否获得稳定性？

可交付成果：包含移除RMSNorms训练时的学习曲线，以及最佳学习率的学习曲线。

交付成果：关于RMSNorm影响的简短评述。

Let's now investigate another layer normalization choice that seems arbitrary at first glance. *Pre-norm* Transformer blocks are defined as

$$z = x + \text{MultiHeadedSelfAttention}(\text{RMSNorm}(x))$$
$$y = z + \text{FFN}(\text{RMSNorm}(z)).$$

This is one of the few 'consensus' modifications to the original Transformer architecture, which used a *post-norm* approach as

$$z = \text{RMSNorm}(x + \text{MultiHeadedSelfAttention}(x))$$
$$y = \text{RMSNorm}(z + \text{FFN}(z)).$$

Let's revert back to the *post-norm* approach and see what happens.

Problem (`pre_norm_ablation`): Implement post-norm and train (1 point) (1 H100 hr)

Modify your pre-norm Transformer implementation into a post-norm one. Train with the post-norm model and see what happens.

Deliverable: A learning curve for a post-norm transformer, compared to the pre-norm one.

We see that layer normalization has a major impact on the behavior of the transformer, and that even the position of the layer normalization is important.

Ablation 2: position embeddings We will next investigate the impact of the position embeddings on the performance of the model. Specifically, we will compare our base model (with RoPE) with not including position embeddings at all (NoPE). It turns out that decoder-only transformers, i.e., those with a causal mask as we have implemented, can in theory infer relative or absolute position information without being provided with position embeddings explicitly [Tsai et al., 2019, Kazemnejad et al., 2023]. We will now test empirically how NoPE performs compare to RoPE.

Problem (`no_pos_emb`): Implement NoPE (1 point) (1 H100 hr)

Modify your Transformer implementation with RoPE to remove the position embedding information entirely, and see what happens.

Deliverable: A learning curve comparing the performance of RoPE and NoPE.

Ablation 3: SwiGLU vs. SiLU Next, we will follow Shazeer [2020] and test the importance of gating in the feed-forward network, by comparing the performance of SwiGLU feed-forward networks versus feed-forward networks using SiLU activations but no gated linear unit (GLU):

$$\text{FFN}_{\text{SiLU}}(x) = W_2\text{SiLU}(W_1 x). \tag{25}$$

Recall that in our SwiGLU implementation, we set the dimensionality of the inner feed-forward layer to be roughly $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ (while ensuring that $d_{\text{ff}} \mod 64 = 0$, to make use of GPU tensor cores). In your FFN_{SiLU} implementation you should set $d_{\text{ff}} = 4 \times d_{\text{model}}$, to approximately match the parameter count of the SwiGLU feed-forward network (which has three instead of two weight matrices).

Problem (`swiglu_ablation`): SwiGLU vs. SiLU (1 point) (1 H100 hr)

Deliverable: A learning curve comparing the performance of SwiGLU and SiLU feed-forward networks, with approximately matched parameter counts.

现在我们来探讨另一种看似随意的层归一化选择。 *预归一化* Transformer模块的定义如下：

$$
\begin{aligned}
z &= x + \mathrm{MultiHeadedSelfAttention}(\mathrm{RMSNorm}(x)) \\
y &= z + \mathrm{FFN}(\mathrm{RMSNorm}(z)).
\end{aligned}
$$

这是对原始Transformer架构的少数几个“共识”修改之一，该架构采用了*后归一化*方法

$$
\begin{aligned}
z &= \mathrm{RMSNorm}(x + \mathrm{MultiHeadedSelfAttention}(x)) \\
y &= \mathrm{RMSNorm}(z + \mathrm{FFN}(z)).
\end{aligned}
$$

让我们回到*后规范*方法，看看会发生什么。

问题（预归一化消融）：实施后归一化并训练（1分）（1小时100分钟）

将你的预归一化Transformer模型修改为后归一化模型。使用后归一化模型进行训练，观察其效果。

可交付成果：后归一化变换器与前归一化变换器的学习曲线对比。

我们发现层归一化对Transformer的行为具有重大影响，且层归一化的位置同样重要。

消融实验2：位置嵌入 接下来我们将研究位置嵌入对模型性能的影响。具体而言，我们将比较基础模型（使用RoPE）与完全不包含位置嵌入（NoPE）的情况。事实证明，仅使用解码器的Transformer模型（即我们实现的因果掩码模型）理论上无需显式提供位置嵌入即可推断相对或绝对位置信息[Tsai等人，2019，Kazemnejad等人，2023]。现在我们将通过实验验证NoPE与RoPE的性能差异。

问题（无位置嵌入）：实现无位置嵌入（NoPE）（1分）（1 H100小时）

使用RoPE修改Transformer实现，完全移除位置嵌入信息，观察其效果。

交付成果：比较RoPE与NoPE性能的学习曲线。

Ablation 3: SwiGLU vs. SiLU 接下来，我们将遵循 Shazeer [2020] 的方法，通过比较使用 SwiGLU 前馈网络与使用 SiLU 激活函数但未使用门控线性单元（GLU）的前馈网络的性能，来测试门控在前馈网络中的重要性：

$$
\mathrm{FFN}_{\mathrm{SiLU}}(x) = W_2 \mathrm{SiLU}(W_1 x). \tag{25}
$$

回想一下，在我们的SwiGLU实现中，我们将内部前馈层的维度设置为大约$d_{\mathrm{ff}} = \frac{8}{3} d_{\text{模型}}$（同时确保 $d_{\mathrm{ff}} \bmod 64 = 0$，以利用GPU张量核心）。在您的 $\mathrm{FFN}_{\mathrm{SiLU}}$ 实现中，应设置 $d_{\mathrm{ff}} = 4 \times d_{\text{模型}}$，以近似匹配SwiGLU前馈网络的参数数量（该网络具有三个而非两个权重矩阵）。

问题（swiglu_ablation）：SwiGLU与SiLU对比（1分）（1 H100小时）

交付成果：比较SwiGLU与SiLU前馈网络性能的学习曲线，参数数量大致匹配。

Deliverable: A few sentences discussing your findings.

Low-Resource/Downscaling Tip: Online students with limited GPU resources should test modifications on TinyStories

In the remainder of the assignment, we will move to a larger-scale, noisier web dataset (OpenWebText), experimenting with architecture modifications and (optionally) making a submission to the course leaderboard.

It takes a long time to train an LM to fluency on OpenWebText, so we suggest that online students with limited GPU access continue testing modifications on TinyStories (using validation loss as a metric to evaluate performance).

7.4 Running on OpenWebText

We will now move to a more standard pretraining dataset created from a webcrawl. A small sample of OpenWebText [Gokaslan et al., 2019] is also provided as a single text file: see section 1 for how to access this file.

Here is an example from OpenWebText. Note how the text is much more realistic, complex, and varied. You may want to look through the training dataset to get a sense of what training data looks like for a webscraped corpus.

Example (`owt_example`): One example from OWT

Baseball Prospectus director of technology Harry Pavlidis took a risk when he hired Jonathan Judge.

Pavlidis knew that, as Alan Schwarz wrote in The Numbers Game, "no corner of American culture is more precisely counted, more passionately quantified, than performances of baseball players." With a few clicks here and there, you can findout that Noah Syndergaard's fastball revolves more than 2,100 times per minute on its way to the plate, that Nelson Cruz had the game's highest average exit velocity among qualified hitters in 2016 and myriad other tidbits that seem ripped from a video game or science fiction novel. The rising ocean of data has empowered an increasingly important actor in baseball's culture: the analytical hobbyist.

That empowerment comes with added scrutiny – on the measurements, but also on the people and publications behind them. With Baseball Prospectus, Pavlidis knew all about the backlash that accompanies quantitative imperfection. He also knew the site's catching metrics needed to be reworked, and that it would take a learned mind – someone who could tackle complex statistical modeling problems – to complete the job.

"He freaks us out." Harry Pavlidis

Pavlidis had a hunch that Judge "got it" based on the latter's writing and their interaction at a site-sponsored ballpark event. Soon thereafter, the two talked over drinks. Pavlidis' intuition was validated. Judge was a fit for the position – better yet, he was a willing fit. "I spoke to a lot of people," Pavlidis said, "he was the only one brave enough to take it on." [...]

Note: You may have to re-tune your hyperparameters such as learning rate or batch size for this experiment.

Problem (`main_experiment`): Experiment on OWT (2 points) (3 H100 hrs)

Train your language model on OpenWebText with the same model architecture and total training iterations as TinyStories. How well does this model do?

Deliverable: A learning curve of your language model on OpenWebText. Describe the difference in losses from TinyStories – how should we interpret these losses?

交付成果：用几句话说明你的发现。

低资源/降级提示：GPU资源有限的在线学生应在TinyStories上测试修改

在本任务的后续部分，我们将转向一个规模更大、噪声更丰富的网络数据集（OpenWebText），尝试进行架构修改，并（可选）向课程排行榜提交作品。

训练语言模型（LM）在OpenWebText数据集上达到流畅性需要较长时间，因此我们建议GPU资源有限的在线学生继续在TinyStories数据集上测试改进方案（以验证损失作为性能评估指标）。

7.4 在OpenWebText上运行

我们现在将转向一个更标准的预训练数据集，该数据集由网络爬虫创建。OpenWebText[Gokaslan等人，2019]的一个小样本也被提供为一个文本文件：如何访问此文件请参见1节。

这里有一个OpenWebText的示例。请注意，该文本更加真实、复杂且多样化。建议您浏览训练数据集，以了解网络抓取语料库的训练数据特征。

示例（owt_example）：来自OWT的一个示例

棒球发展联盟技术总监哈里·帕夫利迪斯在聘请乔纳森·贾奇时，冒了很大风险。

帕夫利迪斯深知，正如艾伦·施瓦茨在《数字游戏》中所言："在美国文化中，没有哪个领域比棒球运动员的表现更被精准统计、更被热切量化。"只需轻点鼠标，你就能发现诺亚·辛德加德的快球在飞向本垒板时每分钟旋转超过2100次，尼尔森·克鲁兹在2016年创下棒球界合格击球手平均出球速度的最高纪录，还有无数其他像是从电子游戏或科幻小说里搬出来的冷知识。数据洪流让棒球文化中一个日益重要的角色崭露头角——数据分析爱好者。

这种赋权伴随着更严格的审查——不仅针对测量指标，也包括其背后的人和出版物。通过Baseball Prospectus，帕夫利迪斯深知量化不完善会引发的反弹。他也明白该网站的接球指标需要重新设计，而完成这项工作需要具备专业素养——即能够解决复杂统计建模问题的人才。

"他让我们感到恐慌。"哈里·帕夫利迪斯

帕夫利迪斯凭直觉认为，法官"懂行"——这既源于法官的著述，也源自两人在某赞助棒球场活动中的互动。不久后，两人边喝边聊。帕夫利迪斯的直觉得到了印证：法官确实适合这个职位，更难得的是，他主动愿意接受。"我跟不少人聊过，"帕夫利迪斯说，"他是唯一一个有胆量接下这个职位的人。"

注： 本实验可能需要重新调整超参数（如学习率或批量大小）。

问题（main_experiment）：OWT实验（2分）（3 H100小时）

使用与TinyStories相同的模型架构和总训练迭代次数，在OpenWebText上训练你的语言模型。该模型表现如何？

可交付成果：您在OpenWebText上的语言模型学习曲线。请说明TinyStories损失函数的差异——我们应如何解读这些损失？

Deliverable: Generated text from OpenWebText LM, in the same format as the TinyStories outputs. How is the fluency of this text? Why is the output quality worse even though we have the same model and compute budget as TinyStories?

7.5 Your own modification + leaderboard

Congratulations on getting to this point. You're almost done! You will now try to improve upon the Transformer architecture, and see how your hyperparameters and architecture stack up against other students in the class.

Rules for the leaderboard There are no restrictions other than the following:

`Runtime` Your submission can run for at most 1.5 hours on an H100. You can enforce this by setting `--time=01:30:00` in your slurm submission script.

`Data` You may only use the OpenWebText training dataset that we provide.

Otherwise, you are free to do whatever your heart desires.

If you are looking for some ideas on what to implement, you can checkout some of these resources:

- State-of-the-art open-source LLM families, such as Llama 3 [Grattafiori et al., 2024] or Qwen 2.5 [Yang et al., 2024].
- The NanoGPT speedrun repository (`https://github.com/KellerJordan/modded-nanogpt`), where community members post many interesting modifications for "speedrunning" small-scale language model pretraining. For example, a common modification that dates back to the original Transformer paper is to tie the weights of the input and output embeddings together (see Vaswani et al. [2017] (Section 3.4) and Chowdhery et al. [2022] (Section 2)). If you do try weight tying, you may have to decrease the standard deviation of the embedding/LM head init.

You will want to test these on either a small subset of OpenWebText or on TinyStories before trying the full 1.5-hour run.

As a caveat, we do note that some of the modifications you may find working well in this leaderboard may not generalize to larger-scale pretraining. We will explore this idea further in the scaling laws unit of the course.

Problem (`leaderboard`): Leaderboard (6 points) (10 H100 hrs)

You will train a model under the leaderboard rules above with the goal of minimizing the validation loss of your language model within 1.5 H100-hour.

Deliverable: The final validation loss that was recorded, an associated learning curve that clearly shows a wallclock-time x-axis that is less than 1.5 hours and a description of what you did. We expect a leaderboard submission to beat at least the naive baseline of a 5.0 loss. Submit to the leaderboard here: https://github.com/stanford-cs336/assignment1-basics-leaderboard.

可交付成果：基于OpenWebText语言模型生成的文本，格式与TinyStories输出一致。该文本的流畅度如何？为何在使用与TinyStories相同的模型和计算预算时，输出质量反而更差？

7.5 你的修改 + 排行榜

恭喜你走到这一步！你已接近完成！接下来你将尝试改进Transformer架构，并观察你的超参数和架构在班级中与其他同学相比的表现。

排行榜规则 除以下内容外，无其他限制：

运行时长 您的提交在H100上最多可运行1.5小时。您可通过在slurm提交脚本中设置--time=01:30:00来强制执行此限制。

数据 您只能使用我们提供的 OpenWebText 训练数据集。

否则，你大可随心所欲。

若您需要获取具体实施方案的参考建议，可查阅以下资源：

- 最先进的开源大语言模型家族，例如Llama 3[Grattafiori等人，2024]或Qwen 2.5[Yang等人，2024]。
- NanoGPT速跑代码库（`https://github.com/KellerJordan/modded-nanogpt`），社区成员在此发布许多有趣的“速跑”小规模语言模型预训练修改方案。例如，一种可追溯至原始Transformer论文的常见修改是将输入和输出嵌入的权重绑定在一起（参见Vaswani等人[原文链接]2017]（第3.4节）和Chowdhery等人[2022] （第2节））。若尝试采用权重绑定法，可能需要降低嵌入/语言模型头部初始化的标准差。

在尝试完整运行1.5小时之前，建议先在OpenWebText的小样本数据集或TinyStories上进行测试。

需要特别说明的是，您在本排行榜中发现效果良好的某些调整，可能无法推广至更大规模的预训练场景。我们将在课程的扩展定律单元中对此进行深入探讨。

问题（排行榜）：排行榜（6分）（10 H100小时）

您将按照上述排行榜规则训练模型，目标是在1.5个H100小时（1.5个百小时）内最小化语言模型的验证损失。

交付成果：记录的最终验证损失、清晰显示时钟时间x轴小于1.5小时的相关学习曲线，以及对工作内容的描述。我们期望排行榜提交的成果至少能超越5.0损失的简单基线。请在此处提交排行榜：https://github.com/stanford-cs336/assignment1-basics-leaderboard。

References

Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English?, 2023. arXiv:2305.07759.

Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. OpenWebText corpus. `http://Skylion007.github.io/OpenWebTextCorpus`, 2019.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proc. of ACL*, 2016.

Changhan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords, 2019. arXiv:1909.03341.

Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, February 1994. ISSN 0898-9788.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training, 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proc. of NeurIPS*, 2017.

Toan Q. Nguyen and Julian Salazar. Transformers without tears: Improving the normalization of self-attention. In *Proc. of IWSWLT*, 2019.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the Transformer architecture. In *Proc. of ICML*, 2020.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. arXiv:1607.06450.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. arXiv:2302.13971.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Proc. of NeurIPS*, 2019.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu

参考文献

Ronen Eldan 与 Yuanzhi Li 。TinyStories：语言模型能小到什么程度仍能说通顺的英语？，2023年。arXiv: 2305.07759。

Aaron Gokaslan、Vanya Cohen、Ellie Pavlick 和 Stefanie Tellex。OpenWebText 语料库。http:// Skylion007.github.io/OpenWebTextCorpus，2019年。

里科·森里奇、巴里·哈多和亚历山德拉·伯奇。基于子词单元的罕见词汇神经机器翻译。收录于*ACL会议论文集*，2016年。

Changhan Wang 、 Kyunghyun Cho 和 Jiatao Gu 。基于字节级子词的神经机器翻译，2019年。arXiv:1909.03341。

菲利普·盖奇。一种新的数据压缩算法。*C Users Journal*，12(2):23-38,1994年2月。ISSN 0898-9788。

Alec Radford、Jeff Wu、Rewon Child、David Luan、Dario Amodei 和 Ilya Sutskever。语言模型是无监督多任务学习器，2019年。

Alec Radford、Karthik Narasimhan、Tim Salimans 和 Ilya Sutskever。通过生成式预训练提升语言理解能力，2018年。

阿希什·瓦斯瓦尼、诺姆·沙泽尔、尼基·帕尔马尔、雅各布·乌什科雷特、利昂·琼斯、艾丹·N·戈麦斯、 Ł ·乌卡什·凯泽和伊利亚·波洛苏欣。注意力就是你需要的一切。发表于*神经信息处理会议论文集*，2017年。

阮全Q.与朱利安·萨拉查。无泪变形金刚：提升自注意力的标准化水平。载于*IWSWLT 会议录*，2019年。

熊瑞斌、杨云昌、何迪、郑凯、郑淑欣、邢晨、张慧帅、兰艳艳、王丽薇、刘铁燕。关于Transformer架构中的层归一化。收录于*ICML会议论文集*，2020年。

Jimmy Lei Ba、 Jamie Ryan Kiros和 Geoffrey E. Hinton . 层归一化，2016年。arXiv:1607.06450。

雨果·图弗龙、蒂博·拉夫里尔、戈蒂埃·伊扎卡尔、泽维尔·马蒂内、玛丽-安妮·拉绍、蒂莫西·拉克鲁瓦、巴普蒂斯特·罗齐耶、纳曼·戈亚尔、埃里克·汉布罗、费萨尔·阿扎尔、奥雷利安·罗德里格斯、阿曼德·茹兰、爱德华·格拉夫和纪尧姆·兰普勒。《Llama：开放高效的基金会语言模型》，2023年。
arXiv:2302.13971.

Biao Zhang 与 Rico Sennrich 。均方根层归一化。 In *Proc* 。*NeurIPS* 2019。

亚伦·格拉塔菲奥里、阿比曼尤·杜贝、阿比纳夫·乔里、阿比纳夫·潘迪、阿比谢克·卡迪安、艾哈迈德·阿尔达赫勒、艾莎·莱特曼、阿基尔·马图尔、艾伦·舍尔滕、亚历克斯·沃恩、艾米·杨、安吉拉·范、阿尼鲁德·戈亚尔、安东尼·哈特肖恩、杨阿波、阿奇·米特拉、阿奇·斯拉万库马尔、阿尔乔姆·科列涅夫、亚瑟·欣斯瓦克、阿伦·拉奥、张阿斯顿、奥雷利安·罗德里格斯、奥斯汀·格雷格森、艾娃·斯帕塔鲁、巴蒂斯特·罗齐耶、贝瑟尼·比龙、唐斌、博比·切尔恩、夏洛特·科谢图、查亚·奈亚克、克洛伊·比、克里斯·马拉、克里斯·麦康奈尔、克里斯蒂安·凯勒、克里斯托夫·图雷、吴春阳、科琳·王、克里斯蒂安·坎顿·费雷尔、塞勒斯·尼科莱迪斯、达米安·阿隆修斯、丹尼尔·宋、丹妮尔·平茨、丹尼·利夫希茨、丹尼·怀亚特、大卫·埃西奥布、德鲁夫·乔杜里、德鲁夫·马哈詹、迭戈·加西亚-奥拉诺、迭戈·佩里诺、迪尤克·赫普克斯、叶戈尔·拉科姆金、伊哈布·阿尔巴达维、埃琳娜·洛巴诺娃、艾米丽·迪南、埃里克·迈克尔·史密斯、菲利普·拉德诺维奇、弗朗西斯科·古兹曼、张弗兰克、加布里埃尔·西纳耶夫、加布里埃尔·李、乔治亚·刘易斯·安德森、戈文德·塔泰、格雷姆·奈尔、格雷戈瓦·米亚隆、关庞、吉列姆·库库雷尔、海莉·阮、汉娜·科雷瓦尔、许虎、雨果·图弗龙、伊利亚·扎罗夫、伊马诺尔·阿里埃塔·伊瓦拉、伊莎贝尔·克洛曼、伊尚·米斯拉、伊万·埃夫蒂莫夫、张杰克、杰德·科佩特、杰温·李ee、Jan Geffert、Jana Vranes、Jason Park、Jay Mahadeokar、Jeet Shah、Jelmer van der Linde、Jennifer Billock、Jenny Hong、Jenya Lee、Jeremy Fu、Jianfeng Chi、Jianyu

Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Moham-

黄佳文、刘佳文、王杰、余杰超、乔安娜·比特顿、乔·斯皮萨克、朴钟秀、约瑟夫·罗卡、约书亚·约翰斯顿、约书亚·萨克斯、贾俊腾、卡利安·瓦苏登·阿尔瓦拉、卡提克·普拉萨德、卡提凯亚·乌帕萨尼、凯特·普拉维亚克、李科、肯尼斯·希菲尔德、凯文·斯通、哈立德·埃尔-阿里尼、克里蒂卡·伊耶尔、克希提兹·马利克、邱昆雷、库纳尔·巴拉、库沙尔·拉科蒂亚、劳伦·兰塔拉-耶里、劳伦斯·范德马滕、陈劳伦斯、谭亮、莉兹·詹金斯、路易斯·马丁、洛维什·马达安、卢博·马洛、卢卡斯·布莱彻、卢卡斯·兰扎特、卢克·德奥利维拉、玛德琳·穆齐、马赫什·帕苏普莱蒂、曼纳特·辛格、马诺哈尔·帕卢里、马尔钦·卡达斯、玛丽亚·辛普科利、马修·奥尔德姆、马蒂厄·里塔、玛雅·帕夫洛娃、梅兰妮·坎巴杜尔、迈克·刘易斯、闵思、米特什·库马尔·辛格、莫娜·哈桑、纳曼·戈亚尔、纳杰斯·托拉比、尼古拉·巴什利科夫、尼古拉·博戈伊切夫、尼拉德里·查特吉、张宁、奥利维尔·杜尚、奥努尔·切莱比、帕特里克·阿尔拉西、张鹏川、李鹏伟、佩塔尔·瓦西奇、彼得·翁、普拉杰瓦尔·巴尔加瓦、普拉提克·杜巴尔、普拉文·克里希南、普尼特·辛格·库拉、徐普欣、何庆、董庆晓、拉加万·斯里尼瓦桑、拉吉·加纳帕蒂、拉蒙·卡尔德雷、里卡多·席尔维拉·卡布拉尔、罗伯特·斯托伊尼克、罗伯塔·莱莱努、罗汉·马赫斯瓦里、罗希特·吉尔达尔、罗希特·帕特尔、罗曼·索维斯特罗尼·波利多罗、罗山·桑巴利、罗斯·泰勒、阮·席尔瓦、侯瑞、王瑞、萨加尔·霍塞尼、萨哈娜·切纳巴萨帕、桑杰·辛格、肖恩·贝尔、金秀贤、谢尔盖·埃杜诺夫、聂少亮、沙兰·纳朗、沙拉特·拉帕蒂、沈胜、万胜业、斯鲁蒂·博萨莱、张顺、西蒙·范登亨德、苏米娅·巴特拉、斯宾塞·惠特曼、斯滕·苏特拉、斯特凡·科洛、苏钦·古鲁兰甘、悉尼·博罗金斯基、塔玛尔·赫尔曼、塔拉·福勒、塔雷克·谢阿沙、托马斯·乔治乌、托马斯·西亚洛姆、托比亚斯·施佩克巴赫、托多尔·米哈伊洛夫、童晓、乌贾瓦尔·卡恩、韦丹努·戈斯瓦米、维博尔·古普塔、维格内什·拉马纳坦、维克托·克尔克兹、文森特·贡盖、维吉妮·杜、维什·沃格蒂、维托尔·阿尔比罗、弗拉丹·彼得罗维奇、朱伟伟、熊文汉、傅文茵、惠特尼·米尔斯、泽维尔·马丁内、王晓东、王晓芳、谭小青·艾伦、夏德·夏、谢新峰、贾旭超、王雪伟、耶尔·戈尔施拉格、亚舍什·高尔、雅斯敏·巴巴伊、易文、易文·宋、张宇晨、李悦、毛宇宁、扎卡里·德尔皮埃尔·库德、郑艳、陈正星、佐伊·帕帕基波斯、阿迪提亚·辛格、阿尤希·斯里瓦斯塔瓦、阿巴·贾因、亚当·凯尔西、亚当·沙恩菲尔德、阿迪提亚·甘吉迪、阿道夫·维多利亚、阿胡瓦·戈尔斯坦德、阿杰·梅农、阿杰·夏尔马、亚历克斯·博森伯格、阿列克谢·巴耶夫斯基、阿尔莉丝·范·费恩斯、阿曼达·卡莱特、阿米特·桑加尼、阿莫斯·特奥、阿南·尤努斯、安德烈·卢普、安德烈斯·阿尔瓦拉多、安德鲁·卡普尔斯、安德鲁·古、安德鲁·何、安德鲁·波尔顿、安德鲁·瑞安、安基特·拉姆昌达尼、安妮·董、安妮·弗兰科、阿努杰·戈亚尔、阿帕拉吉塔·萨拉夫、阿卡班杜·乔杜里、阿什利·加布里埃尔、阿什温·巴拉姆贝、阿萨夫·艾森曼、阿扎德·亚兹丹、博·詹姆斯、本·毛雷尔、本杰明·莱昂哈迪、伯尼·黄、贝丝·洛伊德、贝托·德·帕奥拉、巴尔加维·帕兰贾佩、刘冰、吴波、倪博宇、布雷登·汉考克、布拉姆·瓦斯蒂、布兰登·斯宾塞、布兰尼·斯托伊科维奇、布莱恩·加米多、布里特·蒙塔尔沃、卡尔·帕克、卡莉·伯顿、卡塔利娜·梅希亚、刘策、王长汉、金昌久、周超、胡志诚、朱清祥、蔡克里斯、廷德尔·克里斯、克里斯托夫·费希滕霍费尔、高辛西娅、达蒙·西文、达娜·贝蒂、丹尼尔·克雷默、李丹尼尔、大卫·阿德金斯、徐大卫、达维德·泰斯图吉内、德莉娅·大卫、德维·帕里克、戴安娜·利斯科维奇、迪德姆·福斯、王鼎康、杜克·勒、达斯汀·霍兰德、爱德华·道林、艾萨·贾米尔、艾琳·蒙哥马利、埃莱奥诺拉·普雷萨尼、艾米丽·哈恩、艾米丽·伍德、埃里克-图安·勒、埃里克·布林克曼、埃斯特班·阿尔卡特、埃文·邓巴、埃文·斯莫瑟斯、孙飞、费利克斯·克鲁克、冯天、菲利波斯·科基诺斯、菲拉特·奥兹根内尔、弗朗切斯科·卡乔尼、弗兰克·卡奈特、弗兰克·塞德、加布里埃莱娜·弗洛雷斯、加布里埃拉·施瓦茨、加达·巴迪尔、乔治亚·斯威、吉尔·哈尔珀恩、格兰特·赫尔曼、格里戈里·西佐夫、广义、张、古纳·拉克什米纳拉扬、哈坎·伊南、哈米德·肖贾纳泽里、韩祖、王汉娜、查汉文、哈鲁恩·哈比卜、哈里森·鲁道夫、海伦·苏克、亨利·阿斯佩格伦、亨特·戈德曼、詹鸿源、易卜拉欣·达姆拉杰、伊戈尔·莫利博格、伊戈尔·图法诺夫、伊利亚斯·莱昂蒂亚迪斯、伊琳娜-埃莱娜·维利切、伊泰·加特、杰克·韦斯曼、詹姆斯·格博斯基、詹姆斯·科利、林珍妮、雅弗·阿舍、让-巴蒂斯特·加亚、杰夫·马库斯、杰夫·唐、詹妮弗·陈、珍妮·陈、杰里米·雷岑斯坦、杰里米·特博尔、钟杰西卡、金建、杨静怡、乔·卡明斯、乔恩·卡维尔、乔恩·谢泼德、乔纳森·麦克菲、乔纳森·托雷斯、乔什·金斯伯格、王俊杰、吴凯、侯宇、卡兰·萨克塞纳、卡提卡伊·坎德尔瓦尔、卡泰永·赞德、凯西·马托西奇、考希克·维拉拉加万、凯莉·米凯莱娜、李克谦、基兰·贾加迪什、黄坤、库纳尔·查瓦拉、黄凯、陈莱琳、拉克什亚·加尔格、拉文德·A、莱昂德罗·席尔瓦、李·贝尔、张磊、郭良鹏、余立成、利龙·莫什科维奇、卢卡·韦斯特德特、马迪安·哈布萨、马纳夫·阿瓦拉尼、马尼什·巴特、马蒂纳斯·曼库斯、马坦·哈松、马修·伦尼、马蒂亚斯·雷索、马克西姆·格罗舍夫、马克西姆·瑙莫夫、玛雅·拉蒂、梅根·凯内利苗刘、迈克尔·L·塞尔策、米哈尔·瓦尔科、米歇尔·雷斯特雷波、米希尔·帕特尔、米克·维亚茨科夫、米凯尔·萨姆维利安、迈克·克拉克、迈克·梅西、迈克·王、米克尔·胡贝尔特·埃尔莫索、莫·梅塔纳特、莫哈姆-

mad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024. URL `https://arxiv.org/abs/2407.21783`.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways, 2022. arXiv:2204.02311.

Dan Hendrycks and Kevin Gimpel. Bridging nonlinearities and stochastic regularizers with gaussian error linear units, 2016. arXiv:1606.08415.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, 2017. URL `https://arxiv.org/abs/1702.03118`.

Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks, 2017. URL `https://arxiv.org/abs/1612.08083`.

Noam Shazeer. GLU variants improve transformer, 2020. arXiv:2002.05202.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021.

疯狂的拉斯泰加里、穆尼什·班萨尔、南迪尼·桑塔南、娜塔莎·帕克斯、娜塔莎·怀特、纳维亚塔·巴瓦、纳扬·辛格哈尔、尼克·埃格博、尼古拉·乌苏尼耶、尼基尔·梅塔、尼古拉·帕夫洛维奇·拉普捷夫、宁东、诺曼·程、奥列格·切尔诺古兹、奥利维亚·哈特、奥姆卡尔·萨尔佩卡尔、奥兹莱姆·卡林利、帕金·肯特、帕尔特·帕雷克、保罗·萨布、帕万·巴拉吉、佩德罗·里特纳、菲利普·邦特拉格、皮埃尔·鲁、皮奥特·多拉、波琳娜·兹维亚金娜、普拉山特·拉坦昌达尼、普里蒂什·尤夫拉杰、钱亮、拉查德·阿劳、蕾切尔·罗德里格斯、拉菲·阿尤布、拉戈塔姆·穆尔蒂、拉古·纳亚尼、拉胡尔·米特拉、兰加普拉布·帕尔萨萨拉蒂、雷蒙德·李、丽贝卡·霍根、罗宾·巴蒂、罗基·王、拉塞尔·豪斯、鲁蒂·里诺特、萨钦·梅塔、萨钦·西比、赛·贾耶什·邦杜、萨米亚克·达塔、萨拉·丘格、萨拉·亨特、萨尔贡·迪隆、萨沙·西多罗夫、萨塔德鲁·潘、绍拉布·马哈詹、绍拉布·维尔马、山本诚二、沙拉德·拉马斯瓦米、肖恩·林赛、肖恩·林赛、冯胜、林胜浩、辛欣·辛迪·扎、希希尔·帕蒂尔、希瓦·尚卡尔、张树强、张树强、王思农、斯内哈·阿加瓦尔、索吉·萨朱伊格贝、苏密特·钦塔拉、斯蒂芬妮·麦克斯、斯蒂芬·陈、史蒂夫·基霍、史蒂夫·萨特菲尔德、苏达山·戈文达普拉萨德、苏米特·古普塔、邓夏·邓、赵成民、桑尼·维尔克、苏拉杰·苏布拉马尼安、西·乔杜里悉尼·戈德曼、塔尔·雷梅兹、塔玛尔·格拉瑟、塔玛拉·贝斯特、蒂洛·科勒、托马斯·罗宾逊、李天禾、张天军、蒂姆·马修斯、周天明、祖克·沙克德、瓦伦·冯蒂米塔、维多利亚·阿贾伊、维多利亚·蒙塔涅兹、维贾伊·莫汉、维奈·萨蒂什·库马尔、维沙尔·曼格拉、弗拉德·伊奥内斯库、弗拉德·波纳鲁、弗拉德·蒂贝里乌·米哈伊莱斯库、弗拉基米尔·伊万诺夫、李伟、王文晨、江文文、韦斯·布阿齐兹、威尔·康斯特布尔、唐晓成、吴晓健、王晓兰、吴希伦、高新波、亚尼夫·克莱因曼、陈亚军、胡烨、贾烨、齐烨、李延达、张一林、张颖、约西·阿迪、南永进、余、王、赵宇、郝宇晨、钱云迪、李云路、何玉子、扎克·雷特、扎卡里·德维托、泽夫·罗斯布里克、文兆多、杨振宇、赵志伟和马志宇。2024年模型羊驼3号群。URL https://arxiv.org/abs/2407.21783 。

杨安、杨宝松、张柏辰、惠斌远、郑波、余博文、李成远、刘大一恒、黄飞、魏浩然、林欢、杨健、涂建宏、张建伟、杨建新、杨佳曦、周景仁、林俊阳、邓凯、卢克明、鲍克钦、杨克欣、余乐、李梅、薛明峰、张沛、朱琴、门瑞、林润基、李天浩、夏婷宇、任兴章、任宣诚、范阳、苏阳、张一昌、万宇、刘玉琼、崔泽宇、张振儒、邱志涵。Qwen2.5技术报告。*arXiv预印本 arXiv:2412.15115* ，2024年。

阿坎克莎·乔杜里、沙兰·纳朗、雅各布·德夫林、马滕·博斯马、高拉夫·米什拉、亚当·罗伯茨、保罗·巴勒姆、郑亨元、查尔斯·萨顿、塞巴斯蒂安·格曼、帕克·舒、石健森、萨沙·茨维亚申科、约书亚·梅内兹、阿比谢克·拉奥、帕克·巴恩斯、易泰、诺亚姆·沙泽尔、维诺德库马尔·普拉巴卡兰、艾米丽·雷夫、杜南、本·哈钦森、莱纳·波普、詹姆斯·布拉德伯里、雅各布·奥斯汀、迈克尔·伊萨德、盖伊·古尔-阿里、尹鹏程、杜克·托朱、安塞尔姆·列夫斯卡娅、桑杰·格马瓦特、苏尼帕·德夫、亨里克·米哈莱夫斯基、泽维尔·加西亚、韦丹特·米斯拉、凯文·罗宾逊、利亚姆·费杜斯、周丹尼、达芙妮·伊波利托、大卫·卢安、林惠泰、巴雷特·佐夫、亚历山大·斯皮里多诺夫、瑞安·塞帕西、大卫·多汉、希瓦尼·阿格拉瓦尔、马克·奥默尼克、安德鲁·M·戴、塔努马拉扬·桑卡纳拉亚纳·皮莱、玛丽·佩拉特、艾托尔·莱沃科维奇、埃里卡·莫雷拉、瑞翁·柴尔德、奥列克桑德尔·波洛佐夫、凯瑟琳·李、周宗伟、王雪芝、布伦南·萨埃塔、马克·迪亚兹、奥尔罕·菲拉特、米歇尔·卡塔斯塔、杰森·韦、凯西·迈尔-赫尔斯滕、道格拉斯·埃克、杰夫·迪恩、斯拉夫·彼得罗夫和诺亚·菲德尔。PaLM：基于路径的规模语言建模，2022年。arXiv:2204.02311.

丹·亨德里克斯与凯文·金佩尔。基于高斯误差线性单元的非线性与随机正则化方法的桥梁，2016年。arXiv: 1606.08415。

Stefan Elfwing、Eiji Uchibe与Kenji Doya。强化学习中用于神经网络函数近似的Sigmoid加权线性单元，2017年。网址https://arxiv.org/abs/1702.03118。

Yann N.Angela Fan , Michael Auli 和 David Grangier .基于门控卷积网络的语言建模,2017年.网址https://arxiv.org/abs/1612.08083.

Noam Shazeer . GLU变体改进变压器，2020年。arXiv:2002.05202。

苏建林、陆宇、潘胜峰、温波和刘云峰。Roformer：一种具有旋转位置嵌入的增强型变压器，2021年。

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. of ICLR*, 2015.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proc. of ICLR*, 2019.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Proc. of NeurIPS*, 2020.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. arXiv:2001.08361.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. arXiv:2203.15556.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *Proc. of ICLR*, 2020.

Yao-Hung Hubert Tsai, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, and Ruslan Salakhutdinov. Transformer dissection: An unified understanding for transformer‘s attention via the lens of kernel. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4344–4353, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1443. URL `https://aclanthology.org/D19-1443/`.

Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL `https://openreview.net/forum?id=Drrl2gcjzl`.

Diederik P. Kingma 与 Jimmy Ba 。亚当：一种随机优化方法。《 ICLR In *Proc* 》，2015年。伊利亚·洛什奇洛夫与弗兰克·胡特。解耦权重衰减正则化。收录于《 ICLR 会议录》，2019年。

汤姆·B·布朗、本杰明·曼、尼克·莱德、梅兰妮·苏比亚、贾里德·卡普兰、普拉富拉·达里瓦尔、阿文德·尼拉坎坦、普拉纳夫·希亚姆、吉里什·萨斯特里、阿曼达·阿斯凯尔、桑迪尼·阿加瓦尔、阿里尔·赫伯特-沃斯、格雷琴·克鲁格、汤姆·亨尼根、瑞翁·柴尔德、阿迪提亚·拉梅什、丹尼尔·M·齐格勒、吴杰夫、克莱门斯·温特、克里斯托弗·赫塞、马克·陈、埃里克·西格勒、马特乌什·利特温、斯科特·格雷、本杰明·切斯、杰克·克拉克、克里斯托弗·伯纳、萨姆·麦坎德利什、亚历克·拉德福德、伊利亚·苏茨克韦尔和达里奥·阿莫代。语言模型属于少样本学习器。该研究发表于2020年*《神经信息处理会议论文集》*。

贾里德·卡普兰、萨姆·麦坎德利什、汤姆·海尼根、汤姆·B·布朗、本杰明·切斯、瑞翁·柴尔德、斯科特·格雷、亚历克·拉德福德、吴杰夫和达里奥·阿莫代。神经语言模型的缩放定律，2020年。arXiv:2001.08361。

乔丹·霍夫曼、塞巴斯蒂安·博尔戈多、亚瑟·门施、叶莲娜·布恰茨卡娅、崔特·蔡、伊丽莎·拉瑟福德、迭戈·德拉斯卡萨斯、丽莎·安妮·亨德里克斯、约翰内斯·韦尔布、艾丹·克拉克、汤姆·亨尼根、埃里克·诺兰、凯蒂·米利坎、乔治·范登德里斯切、博格丹·达莫克、奥蕾莉亚·盖伊、西蒙·奥斯因德罗、凯伦·西蒙尼安、埃里希·埃尔森、杰克·W·雷、奥里奥尔·维尼亚尔斯和洛朗·西弗。《训练计算最优大型语言模型》，2022年。arXiv:2203.15556。

阿里·霍尔茨曼、简·布伊斯、李杜、麦克斯韦尔·福布斯和叶金·崔。神经退行性变的奇特案例。载于*ICLR 会议录*，2020年。

姚宏·休伯特·蔡、邵杰·白、山田真、路易-菲利普·莫朗西和鲁斯兰·萨拉胡丁诺夫。Transformer解剖：通过核的视角统一理解Transformer的注意力。见Kentaro Inui、Jing Jiang、Vincent Ng和Xiaojun Wan编，*2019年会议论文集 在自然语言处理的实证方法与第九届国际自然语言处理联合会议（EMNLP−IJCNLP）*，第4344—4353页，中国香港，2019年11月。计算语言学协会。doi: 10.18653/v1/D19-1443。URL `https://aclanthology.org/D19-1443/` 。

Amirhossein Kazemnejad、Inkit Padhi、Karthikeyan Natesan、Payel Das和Siva Reddy。位置编码对变压器长度泛化的影响。在*第三十七届神经信息处理系统会议*，2023年。URL https://openreview.net/forum？id=Drrl2gcjzl。